Mathematical Foundations of Deep Neural Networks

Lecture Notes (Ch. 1–11)

Contents

Chapter 1. Linear Regression and Gradient Descent 8

1 The Regression Problem 8

2 Matrix Calculus Toolkit 8

3 Normal Equations 9

4 Gradient Descent 9

4.1 The algorithm . . . 9

4.2 Convergence on least squares . . . 10

5 Polynomial Features and Generalization 10

5.1 Feature maps . . . 10

5.2 Training error versus generalization . . . 11

6 Proof Supplement: Uniqueness and the Pseudoinverse 11

7 Selected exercises 11

Chapter 2. Probability, Likelihood and Information Theory 14

1 Events and Conditional Probability 14

2 Random Variables and Distributions 14

3 Maximum Likelihood Estimation 15

4 Bayesian Inference and MAP 15

4.1 Beta–Bernoulli conjugacy . . . 16

5 MAP as Regularization 16

5.1 Gaussian mean with a Gaussian prior . . . 16

6 Information and Entropy 17

7 Kullback–Leibler Divergence **18**

8 Mutual Information **18**

9 Cross-Entropy **19**

Chapter 3. Probabilistic Regression, Regularization and Kernel Ridge **20**

1 Gaussian Likelihood Explains Squared Error **20**

2 Ridge Regression **20**

2.1 Definition and closed form 20

2.2 MAP interpretation 20

3 Other Penalties and the Lasso **21**

4 Kernel Ridge Regression **21**

4.1 Feature-space and sample-space forms 21

4.2 Kernels 22

4.3 Worked examples 22

5 Selected exercises **23**

Chapter 4. Classification: Logistic Regression, Softmax and SVM **25**

1 Odds, Logit and the Logistic Model **25**

2 Likelihood, Gradient, Hessian and Convexity **25**

3 Labels in $\{\pm 1\}$, Logistic Loss and BCE **26**

3.1 Surrogate risk bounds 26

4 Multiclass Classification and Softmax **26**

5 Support Vector Machines **27**

5.1 Geometry and the primal problem 27

5.2 Lagrangian, max–min inequality, weak duality 28

5.3 The dual problem 28

6 Selected exercises **29**

Chapter 5. Neural Networks and Backpropagation **31**

1 From Hand-Designed Features to Learned Functions **31**

2 Neurons, Layers and Shapes **31**

3 Why Nonlinearity and Depth **32**

4 Activation Functions and Outputs **32**

5 Computation Graphs and the Scalar Chain Rule **32**

6 Vector Calculus Rules for Backpropagation **33**

6.1 Backpropagation through an MLP 34

7 Worked Results **34**

7.1 Single neurons: MSE versus cross-entropy 34

7.2 Residual MLP 34

8 Selected exercises **35**

Chapter 6. Training and Optimization of Deep Networks **37**

1 Full, Stochastic and Mini-Batch Gradients **37**

2 Activation Functions Revisited **37**

3 Weight Initialization **38**

3.1 Why the scale matters 38

3.2 Zero and symmetric initialization 38

3.3 Xavier and He initialization 38

3.4 Variance of a sigmoid output 39

4 Learning-Rate Schedules **39**

5 Batch Normalization **40**

5.1 Training-time computation 40

5.2 Test time 40

6 Gradient Descent and Taylor's Theorem **41**

7 Smoothness and the Descent Lemma **41**

8 Stochastic Gradient Descent **42**

9 Momentum **42**

9.1 Heavy-ball momentum 42

9.2 Nesterov momentum . . . 43

10 Adaptive Methods **43**

10.1 Exponential moving averages and bias correction . . . 43

10.2 Special cases and invariances . . . 44

Chapter 7. Convolutional Neural Networks and Architectures **45**

1 Why a New Layer **45**

2 Convolution as a Local Dot Product **45**

2.1 A fixed filter: moving average . . . 45

2.2 Single and multiple channels . . . 45

2.3 Parameter counts . . . 46

3 Stride, Padding and Output Size **46**

4 Learning Filters **47**

5 Shift Equivariance **47**

6 1×1 Convolution, Pooling and Global Average Pooling **47**

7 Algebraic Facts about CNN Layers **48**

8 Vocabulary and Costs **48**

9 AlexNet **48**

10 ZFNet and VGG **49**

11 GoogLeNet and the Inception Module **49**

11.1 Naive module . . . 49

11.2 Bottleneck module . . . 50

11.3 Split–transform–merge . . . 50

12 Why Depth Can Fail, and Residual Networks **50**

13 Saliency Maps **51**

Chapter 8. Universal Approximation, Barron's Theorem and Generalization **52**

1 Three Sources of Error **52**

2 Preliminaries from Analysis **52**

2.1 Measures . . . 52
2.2 Normed spaces and density . . . 52

3 Cybenko's Universal Approximation Theorem 53
3.1 The functional-analytic lemmas . . . 53
3.2 Density for discriminatory functions . . . 54
3.3 Continuous sigmoidals are discriminatory . . . 54

4 Barron's Theorem: a Dimension-Free Rate 54
4.1 Fourier preliminaries . . . 54

5 Proof of Barron's Theorem 55
5.1 Convex hulls and Maurey's lemma . . . 55
5.2 Barron functions lie in a convex hull of half-space indicators . . . 56
5.3 Completing the proof of Theorem 1 . . . 58

6 Generalization and Rademacher Complexity 58
6.1 Population and empirical risk . . . 58
6.2 Rademacher complexity . . . 58
6.3 Two-layer networks . . . 59

Chapter 9. Recurrent Neural Networks and LSTM 60

1 Why Recurrent Networks 60

2 The Recurrence 60

3 Backpropagation Through Time 61
3.1 The idea . . . 61
3.2 A worked two-step example . . . 61
3.3 General formulation . . . 62

4 Vanishing and Exploding Gradients 62
4.1 RNNs are very deep networks . . . 62
4.2 Exploding gradients and gradient clipping . . . 63
4.3 Vanishing gradients . . . 63

5 Long Short-Term Memory 63
5.1 The LSTM cell . . . 63
5.2 Gradient flow through the cell . . . 64

6 Applications 65

6.1 Language modeling 65
6.2 Image captioning 66

Chapter 10. Attention and Transformers 67

1 Attention-Based Image Captioning 67

2 Spatial and Channel Attention 68

3 Inductive Bias 68

4 The General Attention Layer 69
4.1 One query 69
4.2 Why divide by $\sqrt{D}$ 69
4.3 Many queries, keys and values 69

5 Permutation Symmetry 70

6 Positional Encoding 70

7 Masked Self-Attention 71

8 Multi-Head Attention 72

9 The Transformer Encoder 72

10 Linearization of the Transformer 73

11 Layer Normalization 74

12 Why Transformers 75

13 Vision Transformer 75
13.1 GELU 76
13.2 ViT versus CNN 76

14 Swin Transformer (Advanced) 76

15 Application: Radar-Based Precipitation Nowcasting 77

Chapter 11. Generative Models and Autoencoders 78

1 Supervised versus Unsupervised Learning 78

2 Generative Modeling 78

3 Autoregressive Models **79**

4 Latent Variable Models **79**

5 Autoencoders **80**

6 Variational Autoencoder **81**

Chapter 1. Linear Regression and Gradient Descent

Regression, normal equations, gradient descent, generalization

1 The Regression Problem

We observe $(\mathbf{x}_i, y_i)$, $i = 1, \dots, n$, with $\mathbf{x}_i \in \mathbb{R}^k$, $y_i \in \mathbb{R}$, and seek a function predicting y from $\mathbf{x}$. The "best line" minimizes the sum of squared errors (SSE): rotating a benchmark line traces an SSE curve whose minimizer is the fit.

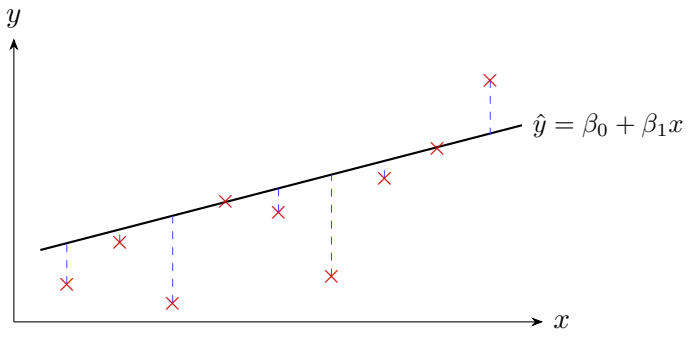
Definition 1.1: linear regression model and design matrix

$$y_i = \beta_0 + \sum_{j=1}^{k} \beta_j x_{ij} + \varepsilon_i, \qquad i = 1, \dots, n.$$

Absorb the intercept with a constant input 1: let $d = k + 1$, $\boldsymbol{\beta} = (\beta_0, \dots, \beta_k)^\top \in \mathbb{R}^d$, and let the **design matrix** $X \in \mathbb{R}^{n \times d}$ have ith row $(1, x_{i1}, \dots, x_{ik})$. Then

$$\mathbf{y} = X\boldsymbol{\beta} + \boldsymbol{\varepsilon}, \qquad f(\boldsymbol{\beta}) = \sum_{i=1}^{n} (y_i - \mathbf{x}_i^\top \boldsymbol{\beta})^2 = \|\mathbf{y} - X\boldsymbol{\beta}\|^2 = (\mathbf{y} - X\boldsymbol{\beta})^\top (\mathbf{y} - X\boldsymbol{\beta}).$$

Rows of X index samples; columns index features.



Residuals $y_i - \hat{y}_i$ (dashed); least squares minimizes the sum of their squares.

2 Matrix Calculus Toolkit

Motivation. A few gradient identities for linear and quadratic forms make every loss computation in the course one line.

Convention: the gradient of a scalar with respect to a vector is a column vector of the same size.

Theorem 1.2: basic gradients

For $\mathbf{a} \in \mathbb{R}^d$ and $A \in \mathbb{R}^{d \times d}$,

$$\nabla_{\boldsymbol{\beta}} (\mathbf{a}^\top \boldsymbol{\beta}) = \mathbf{a}, \qquad \nabla_{\boldsymbol{\beta}} (\boldsymbol{\beta}^\top A \boldsymbol{\beta}) = (A + A^\top)\boldsymbol{\beta}, \qquad \nabla^2_{\boldsymbol{\beta}} (\boldsymbol{\beta}^\top A \boldsymbol{\beta}) = A + A^\top.$$

If A is symmetric the last two become $2A\boldsymbol{\beta}$ and $2A$.

Proof. $\partial(\sum_j a_j\beta_j)/\partial\beta_k = a_k$. For $\boldsymbol{\beta}^\top A\boldsymbol{\beta} = \sum_{i,j} A_{ij}\beta_i\beta_j$ the product rule picks up the terms $i = k$ and $j = k$:

$$\frac{\partial}{\partial\beta_k}\sum_{i,j} A_{ij}\beta_i\beta_j = \sum_j A_{kj}\beta_j + \sum_i A_{ik}\beta_i = (A\boldsymbol{\beta})_k + (A^\top\boldsymbol{\beta})_k.$$

Differentiating again gives $A + A^\top$. ■

3 Normal Equations

Motivation. Least squares has a closed-form solution: projection of $\mathbf{y}$ onto the column space of X.

Theorem 1.3: least squares solution

$\nabla f(\boldsymbol{\beta}) = 2X^\top(X\boldsymbol{\beta} - \mathbf{y})$ and $\nabla^2 f = 2X^\top X \succeq 0$. Every minimizer solves the **normal equations**

$$X^\top X\boldsymbol{\beta} = X^\top\mathbf{y}.$$

If X has full column rank d, then $X^\top X \succ 0$ and the unique minimizer is $\hat{\boldsymbol{\beta}} = (X^\top X)^{-1}X^\top\mathbf{y}$.

Proof. $f = \mathbf{y}^\top\mathbf{y} - 2\mathbf{y}^\top X\boldsymbol{\beta} + \boldsymbol{\beta}^\top X^\top X\boldsymbol{\beta}$, so $\nabla f = -2X^\top\mathbf{y} + 2X^\top X\boldsymbol{\beta}$, $\nabla^2 f = 2X^\top X$. Since $\mathbf{v}^\top X^\top X\mathbf{v} = \|X\mathbf{v}\|^2 \geq 0$, f is convex and stationary points are global minimizers. If the columns of X are independent, $X\mathbf{v} \neq \mathbf{0}$ for $\mathbf{v} \neq \mathbf{0}$, so $X^\top X \succ 0$ is invertible, f is strictly convex and the minimizer is unique. ■

Remark 1.4: geometry and rank

$X^\top(\mathbf{y} - X\hat{\boldsymbol{\beta}}) = \mathbf{0}$: $X\hat{\boldsymbol{\beta}}$ is the orthogonal projection of $\mathbf{y}$ onto $\mathrm{col}(X)$, and residuals sum to zero (first column $\mathbf{1}_n$). With dependent columns ($n > d$ does not prevent this) the pseudoinverse gives the minimum-norm solution; numerically use QR or SVD.

Example 1.5: least squares on three points

Points $(0,1),(1,2),(2,2)$ and model $y = \beta_0 + \beta_1 x$:

$$X = \begin{pmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{pmatrix}, \quad X^\top X = \begin{pmatrix} 3 & 3 \\ 3 & 5 \end{pmatrix}, \quad X^\top\mathbf{y} = \begin{pmatrix} 5 \\ 6 \end{pmatrix}, \quad (X^\top X)^{-1} = \frac{1}{6}\begin{pmatrix} 5 & -3 \\ -3 & 3 \end{pmatrix}.$$

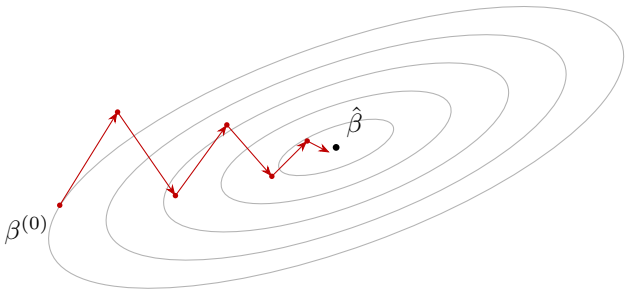
Hence $\hat{\boldsymbol{\beta}} = \frac{1}{6}(25 - 18, -15 + 18)^\top = (7/6,\ 1/2)^\top$; predictions $(7/6, 5/3, 13/6)$, residuals $(-1/6, 1/3, -1/6)$, SSE $= 1/6$.

4 Gradient Descent

4.1 The algorithm

$$\boldsymbol{\beta}_{t+1} = \boldsymbol{\beta}_t - \eta\nabla f(\boldsymbol{\beta}_t) = \boldsymbol{\beta}_t + 2\eta X^\top(\mathbf{y} - X\boldsymbol{\beta}_t),$$

i.e. $\beta_j \leftarrow \beta_j + 2\eta\sum_i(y_i - \hat{y}_i)x_{ij}$ with $x_{i0} = 1$ (the 2 is often absorbed into η); each full-batch step uses all samples. By Cauchy–Schwarz, $\nabla f^\top\mathbf{v}$ over unit $\mathbf{v}$ is minimized at $\mathbf{v} = -\nabla f/\|\nabla f\|$ (steepest descent).



Gradient descent on an elongated quadratic bowl zigzags toward $\hat{\beta}$.

4.2 Convergence on least squares

Let $J(\boldsymbol{\beta}) = \frac{1}{2}\|X\boldsymbol{\beta} - \mathbf{y}\|^2$, $\nabla J = X^\top(X\boldsymbol{\beta} - \mathbf{y})$, $H = X^\top X \succ 0$.

Theorem 1.6: exact convergence condition

Gradient descent $\boldsymbol{\beta}_{t+1} = \boldsymbol{\beta}_t - \eta\nabla J(\boldsymbol{\beta}_t)$ converges to $\hat{\boldsymbol{\beta}}$ for every initial point if and only if

$$0 < \eta < \frac{2}{\lambda_{\max}(X^\top X)}.$$

Proof. Error recursion. $X^\top\mathbf{y} = H\hat{\boldsymbol{\beta}}$, so $\nabla J(\boldsymbol{\beta}_t) = H(\boldsymbol{\beta}_t - \hat{\boldsymbol{\beta}})$ and, with $\mathbf{e}_t = \boldsymbol{\beta}_t - \hat{\boldsymbol{\beta}}$,

$$\mathbf{e}_{t+1} = \mathbf{e}_t - \eta H\mathbf{e}_t = (\mathbf{I} - \eta H)\mathbf{e}_t, \qquad \mathbf{e}_t = (\mathbf{I} - \eta H)^t\mathbf{e}_0.$$

Diagonalize. $H = Q\Lambda Q^\top$, Q orthogonal, $\Lambda = \mathrm{diag}(\lambda_1, \ldots, \lambda_d)$, $\lambda_i > 0$; $\mathbf{I} - \eta H = Q(\mathbf{I} - \eta\Lambda)Q^\top$. With $\mathbf{c}_t = Q^\top\mathbf{e}_t$, $\mathbf{c}_{t+1} = (\mathbf{I} - \eta\Lambda)\mathbf{c}_t$:

$$c_{t,i} = (1 - \eta\lambda_i)^t\, c_{0,i}.$$

As $\|\mathbf{c}_t\| = \|\mathbf{e}_t\|$, $\mathbf{e}_t \to \mathbf{0}$ iff every $c_{t,i} \to 0$.

Sufficiency. If $|1 - \eta\lambda_i| < 1$ for all i, every coordinate tends to zero for every $\mathbf{e}_0$.

Necessity. If $|1 - \eta\lambda_i| \geq 1$ for some i, take $\mathbf{e}_0 = \mathbf{q}_i$: $\|\mathbf{e}_t\| = |1 - \eta\lambda_i|^t \geq 1$.

Solve. $|1 - \eta\lambda_i| < 1 \iff 0 < \eta < 2/\lambda_i$; for all i this is $0 < \eta < 2/\lambda_{\max}$. ■

Remark 1.7: conditioning

$\max_i |1 - \eta\lambda_i|$ is minimized by $\eta^\star = 2/(\lambda_{\max} + \lambda_{\min})$, rate $(\kappa - 1)/(\kappa + 1)$, $\kappa = \lambda_{\max}/\lambda_{\min}$: ill-conditioning slows gradient descent (Chapter 6). At $\eta = 2/\lambda_{\max}$ the top direction oscillates with factor -1, so the inequality is strict. In Example 1.5, $\lambda(X^\top X) = 4 \pm \sqrt{10}$, so $0 < \eta < (4 - \sqrt{10})/3 \approx 0.279$.

5 Polynomial Features and Generalization

5.1 Feature maps

$\boldsymbol{\phi}_M(x) = (1, x, x^2, \ldots, x^M)^\top$, $y(x) = \sum_{j=0}^M \beta_j x^j$: nonlinear in x, linear in $\boldsymbol{\beta}$, with Vandermonde-type design matrix of rows $\boldsymbol{\phi}_M(x_i)^\top$.

5.2 Training error versus generalization

Nested models make the minimal training SSE nonincreasing in M, but validation error can rise: high degree fits noise (overfitting), low degree misses the trend (underfitting). Metric:

$$\mathrm{MSE} = \frac{1}{n}\mathrm{SSE}, \qquad E_{\mathrm{RMS}} = \sqrt{\mathrm{MSE}}.$$

Remark 1.8: roles of the data splits

Training data fit coefficients, validation data choose degree and regularization, a test set estimates final performance. Degree 9 overfits badly with $n = 10$ but not with $n = 100$; "ten points per parameter" is only a heuristic. Regularization (Chapter 3) is the second remedy.

6 Proof Supplement: Uniqueness and the Pseudoinverse

Motivation. When $X^\top X$ is singular, the pseudoinverse picks the minimum-norm solution, the one gradient descent from zero finds.

Theorem 1.9: set of minimizers

The minimizers of $\|\mathbf{y} - X\boldsymbol{\beta}\|^2$ are exactly the solutions of $X^\top X\boldsymbol{\beta} = X^\top \mathbf{y}$; this system is always consistent, and the solution set is $\hat{\boldsymbol{\beta}}_0 + \ker X$ for any particular solution $\hat{\boldsymbol{\beta}}_0$. Among them, $X^+\mathbf{y}$ has minimal Euclidean norm.

Proof. Consistency: $X^\top X\mathbf{v} = \mathbf{0} \Rightarrow \|X\mathbf{v}\|^2 = 0$, so $\mathrm{col}(X^\top)$ and $\mathrm{col}(X^\top X)$ both have complement $\ker X$ and coincide; $X^\top \mathbf{y}$ lies in it. Two solutions differ by $\mathbf{v} \in \ker X$ and give the same fit. With thin SVD $X = U\Sigma V^\top$ (r positive singular values), every solution is $V_r\Sigma_r^{-1}U_r^\top \mathbf{y} + \mathbf{v}$, $\mathbf{v} \perp \mathrm{range}(V_r)$; by Pythagoras the norm is minimal at $\mathbf{v} = \mathbf{0}$, i.e. $X^+\mathbf{y}$. ■

7 Selected exercises

Motivation. Three derivations: the hat matrix, gradient descent's minimum-norm limit, and the optimal step size.

Exercise 1.1 (Hat matrix and the residual sum of squares)

Let $X \in \mathbb{R}^{n\times d}$ have full column rank $d < n$ and put $H = X(X^\top X)^{-1}X^\top$.
(a) Show that H is symmetric and idempotent, $HX = X$ and $\mathrm{tr}\, H = d$; show that $\mathbf{I} - H$ is symmetric and idempotent with $\mathrm{tr}(\mathbf{I} - H) = n - d$.
(b) Show that the fitted values are $H\mathbf{y}$, the residual vector is $\mathbf{e} = (\mathbf{I}-H)\mathbf{y}$, $\mathrm{SSE} = \mathbf{y}^\top(\mathbf{I}-H)\mathbf{y}$, and $\|\mathbf{y}\|^2 = \|H\mathbf{y}\|^2 + \|\mathbf{e}\|^2$.
(c) Let $\mathbf{y} = X\boldsymbol{\beta} + \boldsymbol{\varepsilon}$ with $\mathbb{E}\boldsymbol{\varepsilon} = \mathbf{0}$ and $\mathrm{Cov}(\boldsymbol{\varepsilon}) = \sigma^2\mathbf{I}$. Show that $\mathbb{E}[\mathrm{SSE}] = (n - d)\sigma^2$, so $\mathrm{SSE}/(n - d)$ is unbiased for σ^2.

Solution. [Supplement]

(a) $(X^\top X)^{-1}$ is symmetric, so $H^\top = H$, and

$$H^2 = X(X^\top X)^{-1}(X^\top X)(X^\top X)^{-1}X^\top = H, \qquad HX = X(X^\top X)^{-1}X^\top X = X.$$

$\mathrm{tr}\, H = \mathrm{tr}\,\big((X^\top X)^{-1}X^\top X\big) = \mathrm{tr}\,\mathbf{I}_d = d$. Then $\mathbf{I} - H$ is symmetric, $(\mathbf{I} - H)^2 = \mathbf{I} - 2H + H^2 = \mathbf{I} - H$, $\mathrm{tr}(\mathbf{I} - H) = n - d$.

(b) $X\hat{\boldsymbol{\beta}} = X(X^\top X)^{-1}X^\top \mathbf{y} = H\mathbf{y}$, so $\mathbf{e} = (\mathbf{I} - H)\mathbf{y}$ and by (a)

$$\mathrm{SSE} = \mathbf{e}^\top \mathbf{e} = \mathbf{y}^\top (\mathbf{I} - H)^\top (\mathbf{I} - H)\mathbf{y} = \mathbf{y}^\top (\mathbf{I} - H)\mathbf{y}.$$

$(H\mathbf{y})^\top \mathbf{e} = \mathbf{y}^\top (H - H^2)\mathbf{y} = 0$, so expanding $\|H\mathbf{y} + \mathbf{e}\|^2$ gives the identity.

(c) $(\mathbf{I} - H)X = 0$ gives $\mathbf{e} = (\mathbf{I} - H)\boldsymbol{\varepsilon}$ and $\mathrm{SSE} = \boldsymbol{\varepsilon}^\top (\mathbf{I} - H)\boldsymbol{\varepsilon}$. For fixed A,

$$\mathbb{E}[\boldsymbol{\varepsilon}^\top A \boldsymbol{\varepsilon}] = \mathbb{E}\,\mathrm{tr}(A\boldsymbol{\varepsilon}\boldsymbol{\varepsilon}^\top) = \mathrm{tr}\,(A\,\mathbb{E}[\boldsymbol{\varepsilon}\boldsymbol{\varepsilon}^\top]) = \sigma^2\,\mathrm{tr}\,A.$$

With $A = \mathbf{I} - H$ this is $(n - d)\sigma^2$; hence the MLE SSE/n (Chapter 3) is biased downward. ■

Exercise 1.2 (Gradient descent from zero finds $X^+\mathbf{y}$)

Let $X \neq 0$ have rank r and thin SVD $X = U_r \Sigma_r V_r^\top$ with singular values $s_1, \ldots, s_r > 0$; let the columns of V_0 complete those of V_r to an orthonormal basis of $\mathbb{R}^d$. Run gradient descent on $J(\boldsymbol{\beta}) = \frac{1}{2}\|X\boldsymbol{\beta} - \mathbf{y}\|^2$ with $0 < \eta < 2/\lambda_{\max}(X^\top X)$; $X^\top X$ need not be invertible.
(a) Show that if $\boldsymbol{\beta}_0 = \mathbf{0}$, every iterate lies in $\mathrm{range}(X^\top) = \mathrm{range}(V_r)$.
(b) Show that if $\boldsymbol{\beta}_0 = \mathbf{0}$, then $\boldsymbol{\beta}_t \to X^+\mathbf{y} = V_r \Sigma_r^{-1} U_r^\top \mathbf{y}$.
(c) Show that for a general $\boldsymbol{\beta}_0$ the limit is $X^+\mathbf{y} + V_0 V_0^\top \boldsymbol{\beta}_0$.

Solution. [Supplement]

(a) $\nabla J(\boldsymbol{\beta}) = X^\top(X\boldsymbol{\beta} - \mathbf{y}) \in \mathrm{range}(X^\top)$, so by induction from $\boldsymbol{\beta}_0 = \mathbf{0}$ every $\boldsymbol{\beta}_t$ stays in this subspace. $X^\top = V_r \Sigma_r U_r^\top$ with $\Sigma_r U_r^\top$ of full row rank, so $\mathrm{range}(X^\top) = \mathrm{range}(V_r)$.

(b) *Step 1 (coordinates).* By (a), $\boldsymbol{\beta}_t = V_r \mathbf{c}_t$, $\mathbf{c}_t = V_r^\top \boldsymbol{\beta}_t$. With $X^\top X = V_r \Sigma_r^2 V_r^\top$, $X^\top \mathbf{y} = V_r \Sigma_r U_r^\top \mathbf{y}$,

$$\mathbf{c}_{t+1} = \mathbf{c}_t - \eta(\Sigma_r^2 \mathbf{c}_t - \Sigma_r U_r^\top \mathbf{y}).$$

Step 2 (error recursion). With $\mathbf{c}^\star = \Sigma_r^{-1} U_r^\top \mathbf{y}$, $\mathbf{c}_{t+1} - \mathbf{c}^\star = (\mathbf{I} - \eta\Sigma_r^2)(\mathbf{c}_t - \mathbf{c}^\star)$, diagonal with factors $1 - \eta s_i^2$.

Step 3 (contraction). s_i^2 are the nonzero eigenvalues of $X^\top X$, so $0 < \eta s_i^2 < 2$, $|1 - \eta s_i^2| < 1$, $\mathbf{c}_t \to \mathbf{c}^\star$ and $\boldsymbol{\beta}_t \to V_r \Sigma_r^{-1} U_r^\top \mathbf{y} = X^+\mathbf{y}$.

(c) Write $\boldsymbol{\beta}_t = V_r \mathbf{c}_t + V_0 \mathbf{d}_t$. As $\nabla J \in \mathrm{range}(V_r) \perp V_0$, $\mathbf{d}_t = V_0^\top \boldsymbol{\beta}_0$ for all t. Since $X^\top X V_0 = 0$, $\mathbf{c}_t$ obeys Step 1 from a different start, and Steps 2–3 give $\mathbf{c}_t \to \mathbf{c}^\star$, so $\boldsymbol{\beta}_t \to X^+\mathbf{y} + V_0 V_0^\top \boldsymbol{\beta}_0$: the $\ker X$ component never moves (implicit regularization). ■

Exercise 1.3 (The optimal constant step size)

Let $H = X^\top X \succ 0$ have eigenvalues $0 < \lambda_{\min} = \lambda_d \leq \cdots \leq \lambda_1 = \lambda_{\max}$, $\kappa = \lambda_{\max}/\lambda_{\min}$, and $\rho(\eta) = \max_i |1 - \eta\lambda_i|$ for $\eta > 0$.
(a) Show that $\rho(\eta) = \max\{|1 - \eta\lambda_{\min}|, |1 - \eta\lambda_{\max}|\}$.
(b) Show that ρ is minimized at $\eta^\star = 2/(\lambda_{\max} + \lambda_{\min})$ and $\rho(\eta^\star) = (\kappa - 1)/(\kappa + 1)$.
(c) Deduce $\|\boldsymbol{\beta}_t - \hat{\boldsymbol{\beta}}\| \leq \rho(\eta)^t \|\boldsymbol{\beta}_0 - \hat{\boldsymbol{\beta}}\|$, and compute $\eta^\star$ and $\rho(\eta^\star)$ for Example 1.5.

Solution. [Supplement]

(a) $\lambda \mapsto |1 - \eta\lambda|$ is convex, so its maximum on $[\lambda_{\min}, \lambda_{\max}]$, which contains all λ_i and has eigenvalue endpoints, is at an endpoint.

(b) *Step 1 (lower bound).* By (a), $\rho(\eta) \geq m(\eta) := \max\{1 - \eta\lambda_{\min},\ \eta\lambda_{\max} - 1\}$. The first is decreasing, the second increasing, and they meet at $\eta^\star$, so $m(\eta) \geq m(\eta^\star)$ with

$$m(\eta^\star) = 1 - \frac{2\lambda_{\min}}{\lambda_{\max} + \lambda_{\min}} = \frac{\lambda_{\max} - \lambda_{\min}}{\lambda_{\max} + \lambda_{\min}} = \frac{\kappa - 1}{\kappa + 1}.$$

Step 2 (attained). $1-\eta^\star\lambda_{\min}=\frac{\kappa-1}{\kappa+1}\geq 0$ and $1-\eta^\star\lambda_{\max}=-\frac{\kappa-1}{\kappa+1}$, so $\rho(\eta^\star)=m(\eta^\star)\leq\rho(\eta)$ for all $\eta>0$.

(c) From the convergence proof, $|c_{t,i}|=|1-\eta\lambda_i|^t|c_{0,i}|\leq\rho(\eta)^t|c_{0,i}|$; square, sum, and use $\|\mathbf{c}_t\|=\|\mathbf{e}_t\|$. In Example 1.5, $\lambda_{\max,\min}=4\pm\sqrt{10}$, so $\eta^\star=1/4$ and

$$\rho(\eta^\star)=\frac{\lambda_{\max}-\lambda_{\min}}{\lambda_{\max}+\lambda_{\min}}=\frac{2\sqrt{10}}{8}=\frac{\sqrt{10}}{4}\approx 0.791,$$

only about 21% per step, since $\kappa\approx 8.6$. ■

Chapter 2. Probability, Likelihood and Information Theory

Random variables, maximum likelihood, MAP estimation, Bayesian models; Entropy, KL divergence, mutual information, cross-entropy

1 Events and Conditional Probability

Events are (measurable) subsets of a sample space Ω; the axioms (nonnegativity, $P(\Omega) = 1$, countable additivity) imply

$$P(\emptyset) = 0, \qquad P(E^c) = 1 - P(E), \qquad P(E \cup F) = P(E) + P(F) - P(E \cap F),$$

monotonicity $E \subseteq F \Rightarrow P(E) \le P(F)$, and continuity along increasing or decreasing sequences of events.

Definition 2.1: conditional probability, Bayes' rule, independence

For $P(E) > 0$, $P(F \mid E) = P(F \cap E)/P(E)$. For a partition $E_1, E_2, \dots$ of Ω,

$$P(F) = \sum_i P(F \mid E_i)P(E_i), \qquad P(E_i \mid F) = \frac{P(F \mid E_i)P(E_i)}{\sum_j P(F \mid E_j)P(E_j)}.$$

E, F are independent if $P(E \cap F) = P(E)P(F)$.

Example 2.2: why the prior matters

Let $P(D) = 0.01$, $P(+ \mid D) = 0.9$, $P(+ \mid D^c) = 0.1$. Then

$$P(D \mid +) = \frac{0.9(0.01)}{0.9(0.01) + 0.1(0.99)} \approx 0.0833.$$

A small prior keeps the posterior small. For two fair tosses, "first head" and "second head" (1/2 each, 1/4 jointly) are independent.

2 Random Variables and Distributions

Motivation. Data are samples from an unknown distribution and losses are expectations.

A random variable is a measurable $X : \Omega \to \mathbb{R}$ (e.g. heads in three tosses, $X(HHT) = 2$), with CDF $F_X(t) = P(X \le t)$.

- Discrete: probability mass $p(x) = P(X = x)$, $F_X(t) = \sum_{x \le t} p(x)$.
- Continuous: density $f \ge 0$ with $\int f = 1$, $P(a < X \le b) = \int_a^b f(u)\,du$. A density value is not a probability and may exceed one; $P(X = x) = 0$.

$$\mathbb{E}[g(X)] = \sum_x g(x)p(x) \text{ or } \int g(x)f(x)\,dx, \qquad \mathrm{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}X)^2.$$

Theorem 2.3: tools used throughout the course

(i) Linearity: $\mathbb{E}[aX + bY] = a\mathbb{E}X + b\mathbb{E}Y$ always.

(ii) Independence: $\mathbb{E}[XY] = \mathbb{E}X\,\mathbb{E}Y$; hence $\operatorname{Var}(X+Y) = \operatorname{Var}X + \operatorname{Var}Y$. Uncorrelatedness ($\operatorname{Cov} = 0$) suffices for the variance identity.

(iii) Tower property: $\mathbb{E}[\mathbb{E}[Z \mid X]] = \mathbb{E}Z$.

(iv) $\mathbb{E}[(Z-c)^2] = \operatorname{Var}Z + (\mathbb{E}Z - c)^2 \geq \operatorname{Var}Z$ for every constant c.

Proof. (iv): square $Z - c = (Z - \mathbb{E}Z) + (\mathbb{E}Z - c)$; the cross term has mean 0. The other items are standard. ■

3 Maximum Likelihood Estimation

Definition 2.4: likelihood and MLE

For iid data $x_1, \ldots, x_n$ from $p(x \mid \theta)$,

$$L(\theta) = \prod_{i=1}^{n} p(x_i \mid \theta), \qquad \hat{\theta}_{\mathrm{MLE}} = \arg\max_{\theta} \sum_{i=1}^{n} \log p(x_i \mid \theta).$$

Data fixed, θ varies; L is not a distribution over θ. The log keeps the maximizer.

Example 2.5: Bernoulli, including the boundary

$x_i \in \{0, 1\}$ with $P(x_i = 1) = \theta$; let $s = \sum x_i$, $f = n - s$. Then $\log L = s \log\theta + f \log(1-\theta)$. For $0 < s < n$,

$$\frac{d}{d\theta} \log L = \frac{s}{\theta} - \frac{f}{1-\theta} = 0 \iff \theta = \frac{s}{n}, \qquad \frac{d^2}{d\theta^2} \log L < 0.$$

If $s = 0$, $L = (1-\theta)^n$ is decreasing and the maximizer is 0; if $s = n$ it is 1: both equal s/n, without the derivative argument. Eight heads in ten gives 0.8; two positive reviews of two gives the extreme 1.

Example 2.6: Gaussian

For $x_i \overset{\text{iid}}{\sim} N(\mu, \sigma^2)$, $\log L = -\frac{n}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum(x_i - \mu)^2$, giving $\hat{\mu} = \bar{x}$ and $\hat{\sigma}^2 = \frac{1}{n}\sum(x_i - \bar{x})^2$ (biased; the unbiased version divides by $n - 1$).

4 Bayesian Inference and MAP

Motivation. A prior tempers maximum likelihood on little data; the posterior mode (MAP) is likelihood plus a regularizer.

Definition 2.7: prior, posterior, evidence, MAP

With prior $p(\theta)$ and likelihood $p(D \mid \theta)$,

$$p(\theta \mid D) = \frac{p(D \mid \theta)p(\theta)}{p(D)}, \qquad p(D) = \int p(D \mid \theta)p(\theta)\, d\theta,$$

$$\hat{\theta}_{\mathrm{MAP}} = \arg\max_{\theta} \{\log p(D \mid \theta) + \log p(\theta)\}.$$

The evidence is free of θ; MAP is the posterior mode, not mean.

4.1 Beta–Bernoulli conjugacy

Theorem 2.8: Beta prior

If $\theta \sim \mathrm{Beta}(a, b)$, i.e. $p(\theta) = \theta^{a-1}(1-\theta)^{b-1}/B(a, b)$ with $B(a, b) = \Gamma(a)\Gamma(b)/\Gamma(a+b)$, and s successes, f failures are observed, then

$$\theta \mid D \sim \mathrm{Beta}(a+s,\, b+f), \qquad \mathbb{E}[\theta \mid D] = \frac{a+s}{a+b+n}, \qquad \hat{\theta}_{\mathrm{MAP}} = \frac{a+s-1}{a+b+n-2}$$

(the mode formula needs both posterior parameters above one).

Proof. $p(\theta \mid D) \propto \theta^s(1-\theta)^f \cdot \theta^{a-1}(1-\theta)^{b-1} = \theta^{(a+s)-1}(1-\theta)^{(b+f)-1}$, which is the $\mathrm{Beta}(a+s, b+f)$ kernel. For $\mathrm{Beta}(\alpha, \beta)$, $\mathbb{E}\theta = B(\alpha+1, \beta)/B(\alpha, \beta) = \alpha/(\alpha+\beta)$ using $\Gamma(z+1) = z\Gamma(z)$. The mode solves $\frac{\alpha-1}{\theta} = \frac{\beta-1}{1-\theta}$, i.e. $\theta = \frac{\alpha-1}{\alpha+\beta-2}$. ∎

Under the uniform prior $\mathrm{Beta}(1, 1)$ the posterior mean $(s+1)/(n+2)$ (Laplace's rule of succession) is the predictive $P(X_{\mathrm{new}} = 1 \mid D) = \int \theta\, p(\theta \mid D)\, d\theta$.

Example 2.9: the two sellers

Seller 1: 90 positive, 10 negative reviews; seller 2: 2 positive, 0 negative. Uniform priors give

$$\theta_1 \mid D_1 \sim \mathrm{Beta}(91, 11), \qquad \theta_2 \mid D_2 \sim \mathrm{Beta}(3, 1).$$

Predictive probability of a positive next review: $91/102 \approx 0.892$ versus $3/4$ (seller 1), whereas the MLE (0.9 vs. 1) prefers seller 2. Since $\mathrm{Beta}(3, 1)$ has CDF t^3,

$$P(\theta_1 > \theta_2 \mid D) = \mathbb{E}[\theta_1^3] = \frac{91 \cdot 92 \cdot 93}{102 \cdot 103 \cdot 104} \approx 0.713.$$

5 MAP as Regularization

Motivation. A Gaussian prior turns MAP into ℓ_2-penalized likelihood, so regularization has a probabilistic meaning.

5.1 Gaussian mean with a Gaussian prior

Let $X_1, \ldots, X_n \overset{\text{iid}}{\sim} N(\theta, 1)$ and prior $\theta \sim N(0, \tau^2)$.

Theorem 2.10: MAP is ridge, and it shrinks

$$\hat{\theta}_{\text{MAP}} = \arg\min_{\theta} \sum_{i=1}^{n} (X_i - \theta)^2 + \lambda\theta^2, \quad \lambda = \frac{1}{\tau^2}, \qquad \hat{\theta}_{\text{MAP}} = a\bar{X}, \quad a = \frac{n}{n+\lambda} = \frac{n\tau^2}{n\tau^2+1}.$$

Proof. $-\log p(\theta \mid X) = \frac{1}{2}\sum(X_i-\theta)^2 + \frac{\theta^2}{2\tau^2} + \text{const}$; multiplying by 2 gives the ridge objective. Setting the derivative $-2\sum(X_i-\theta) + 2\lambda\theta$ to zero gives $(n+\lambda)\theta = n\bar{X}$. ■

Remark 2.11: Laplace prior gives the lasso

With $p(\theta) \propto e^{-|\theta|/b}$ instead, $-\log$ posterior is $\frac{1}{2}\sum(X_i-\theta)^2 + \frac{1}{b}|\theta|$ and the MAP is the soft threshold $\operatorname{sign}(\bar{X})\max(|\bar{X}| - \frac{1}{nb}, 0)$ (Chapter 3).

6 Information and Entropy

Motivation. Entropy, the average surprise, is the baseline for cross-entropy and KL losses.

Information $h(x) = -\log p(x)$: larger for rare outcomes, additive over independent events.

Definition 2.12: entropy

For a discrete random variable with pmf p,

$$H(X) = -\sum_{x} p(x)\log p(x) = \mathbb{E}[-\log p(X)], \qquad 0\log 0 := 0.$$

Natural logs give nats; base 2 gives bits.

Theorem 2.13: range of entropy

If X takes at most K values, $0 \le H(X) \le \log K$. $H = 0$ iff X is deterministic; $H = \log K$ iff X is uniform.

Proof. $-p\log p \ge 0$ on $[0,1]$, with equality iff $p \in \{0,1\}$. By Jensen for the concave log, $H = \mathbb{E}\log\frac{1}{p(X)} \le \log\mathbb{E}\frac{1}{p(X)} = \log\sum_{x:p(x)>0} 1 \le \log K$, with equality iff $1/p(X)$ is constant on the support and the support has K points. ■

(Differential entropy of a density can be negative.)

Definition 2.14: joint and conditional entropy

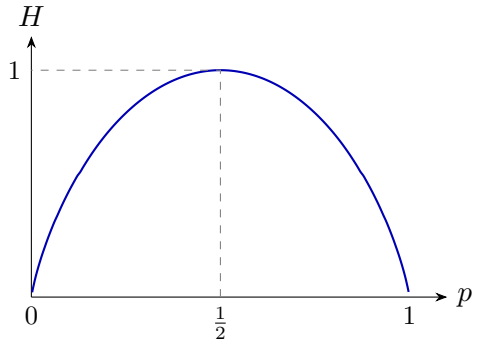
$$H(X,Y) = -\sum_{x,y} p(x,y)\log p(x,y),$$

$$H(Y \mid X) = -\sum_{x,y} p(x,y)\log p(y \mid x) = \sum_{x} p(x) H(Y \mid X = x).$$

Theorem 2.15: chain rule

$H(X,Y) = H(X) + H(Y \mid X)$.

Proof. Take $-\mathbb{E}$ of $\log p(X,Y) = \log p(X) + \log p(Y \mid X)$. ■



Bernoulli(p) entropy in bits, maximal at $p = \frac{1}{2}$.

7 Kullback–Leibler Divergence

Motivation. KL divergence is the expected extra log-loss from using q instead of p; it vanishes only at $q = p$.

Definition 2.16: KL divergence

$$D_{\mathrm{KL}}(p \,\|\, q) = \sum_x p(x) \log \frac{p(x)}{q(x)} = \mathbb{E}_p\Big[\log \frac{p(X)}{q(X)}\Big].$$

Terms with $p(x) = 0$ contribute 0; if $p(x) > 0 = q(x)$ for some x the divergence is $+\infty$.

Theorem 2.17: Gibbs' inequality

$D_{\mathrm{KL}}(p \,\|\, q) \geq 0$, with equality iff $p = q$.

Proof. Sum over $S = \{x : p(x) > 0\}$. By Jensen for the concave log,

$$-D_{\mathrm{KL}}(p \,\|\, q) = \sum_{x \in S} p(x) \log \frac{q(x)}{p(x)} \leq \log \sum_{x \in S} p(x) \frac{q(x)}{p(x)} = \log \sum_{x \in S} q(x) \leq \log 1 = 0.$$

Equality forces $q/p = c$ on S (Jensen) and $\sum_S q = 1$, so $c = 1$ and $q = p$; the converse is clear. ■

Example 2.18: KL is not symmetric

For Bernoulli parameters, $D(r \,\|\, s) = r \log \frac{r}{s} + (1-r) \log \frac{1-r}{1-s}$. With $r = 0.9$, $s = 0.5$: $D(r\|s) \approx 0.368$, $D(s\|r) \approx 0.511$ nats; KL is not a metric.

8 Mutual Information

Motivation. Mutual information measures how much X tells about Y; it is zero exactly under independence.

Definition 2.19: mutual information

$$I(X;Y) = D_{\mathrm{KL}}(p_{XY} \,\|\, p_X p_Y) = \sum_{x,y} p(x,y) \log \frac{p(x,y)}{p(x)p(y)}.$$

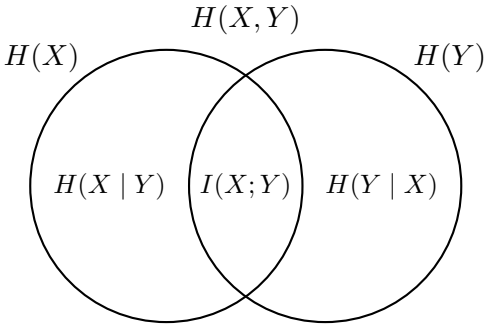
Theorem 2.20: identities

$$I(X;Y) = H(X) - H(X \mid Y) = H(Y) - H(Y \mid X) = H(X) + H(Y) - H(X,Y) \ \geq 0,$$

with $I = 0$ iff $X \perp Y$. Consequently $H(X \mid Y) \leq H(X)$ (conditioning reduces entropy).

Proof. $\log \frac{p(x,y)}{p(x)p(y)} = \log p(x \mid y) - \log p(x)$; taking $\mathbb{E}$ gives $H(X) - H(X \mid Y)$. The last form is the chain rule; nonnegativity and equality are Gibbs. ■

$H(X,Y) = H(X \mid Y) + I(X;Y) + H(Y \mid X)$. A fair bit with $Y = X$ has $I = 1$ bit; independent bits have $I = 0$.



The union of the disks is $H(X,Y)$; the overlap is $I(X;Y)$.

9 Cross-Entropy

Motivation. Cross-entropy is entropy plus KL, so minimizing it over the model minimizes KL.

Definition 2.21: cross-entropy

$H(p,q) = -\sum_x p(x) \log q(x) = \mathbb{E}_p[-\log q(X)]$.

Theorem 2.22: cross-entropy = entropy + KL

$H(p,q) = H(p) + D_{\mathrm{KL}}(p \,\|\, q)$. Hence for fixed p, $\arg\min_q H(p,q) = \arg\min_q D_{\mathrm{KL}}(p \,\|\, q) = p$.

Proof. $-\sum p \log q = -\sum p \log p + \sum p \log(p/q)$. The second statement uses Gibbs. ■

Remark 2.23: classification loss and soft labels

For one-hot $\mathbf{e}_y$, $H(\mathbf{e}_y, q) = D_{\mathrm{KL}}(\mathbf{e}_y \| q) = -\log q_y$, the classification loss. $H(p,q)$ is linear in p, so $H(\frac{1}{n}\sum_j \mathbf{e}_{y_j}, q) = \frac{1}{n}\sum_j (-\log q_{y_j})$: fitting the empirical label distribution by KL equals (up to $H(\hat{p})$) ordinary cross-entropy on each label.

Chapter 3. Probabilistic Regression, Regularization and Kernel Ridge

Linear regression from a probabilistic view, regularization, ridge regression, the kernel trick

1 Gaussian Likelihood Explains Squared Error

Motivation. Under Gaussian noise, maximum likelihood is least squares; each loss corresponds to a noise model.

Theorem 3.1: MLE under Gaussian noise is least squares

If $y_i = \mathbf{x}_i^\top \boldsymbol{\beta} + \varepsilon_i$ with $\varepsilon_i \overset{\text{iid}}{\sim} N(0, \sigma^2)$, then

$$\log p(\mathbf{y} \mid X, \boldsymbol{\beta}, \sigma^2) = -\frac{n}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\|\mathbf{y} - X\boldsymbol{\beta}\|^2.$$

For fixed σ^2, maximizing the likelihood is minimizing the SSE, so $\hat{\boldsymbol{\beta}}_{\text{MLE}}$ solves the normal equations. Maximizing over σ^2 then gives $\hat{\sigma}^2 = \text{SSE}/n$.

Proof. Sum the logs of the independent densities $(2\pi\sigma^2)^{-1/2}\exp\{-(y_i - \mathbf{x}_i^\top\boldsymbol{\beta})^2/(2\sigma^2)\}$. In σ^2: $-\frac{n}{2\sigma^2} + \frac{\text{SSE}}{2\sigma^4} = 0$. ■

2 Ridge Regression

Motivation. Adding $\lambda\|\mathbf{w}\|^2$ makes ill-conditioned least squares well posed and shrinks coefficients.

2.1 Definition and closed form

Definition 3.2: ridge

For $\lambda > 0$, $J(\boldsymbol{\beta}) = \frac{1}{2}\|\mathbf{y} - X\boldsymbol{\beta}\|^2 + \frac{\lambda}{2}\|\boldsymbol{\beta}\|^2$.

Theorem 3.3: ridge solution

$\nabla J = X^\top X\boldsymbol{\beta} - X^\top\mathbf{y} + \lambda\boldsymbol{\beta}$ and the unique minimizer is $\hat{\boldsymbol{\beta}}_\lambda = (X^\top X + \lambda\mathbf{I})^{-1}X^\top\mathbf{y}$, even when X is rank deficient.

Proof. $\mathbf{v}^\top(X^\top X + \lambda\mathbf{I})\mathbf{v} = \|X\mathbf{v}\|^2 + \lambda\|\mathbf{v}\|^2 > 0$ for $\mathbf{v} \neq \mathbf{0}$, so the matrix is positive definite, invertible, and J is strictly convex. ■

2.2 MAP interpretation

With prior $\boldsymbol{\beta} \sim N(\mathbf{0}, \tau^2\mathbf{I})$, $-\log p(\boldsymbol{\beta} \mid \mathbf{y}) = \frac{1}{2\sigma^2}\|\mathbf{y} - X\boldsymbol{\beta}\|^2 + \frac{1}{2\tau^2}\|\boldsymbol{\beta}\|^2 + \text{const}$: ridge with $\lambda = \sigma^2/\tau^2$ (small τ, strong shrinkage).

3 Other Penalties and the Lasso

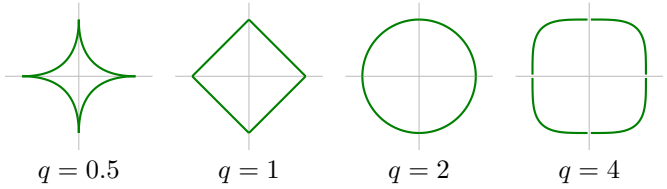
Penalty $\sum_j |\beta_j|^q$: $q = 2$ ridge (pure shrinkage); $q = 1$ lasso (corners on the axes give exact zeros); $0 < q < 1$ nonconvex.

Theorem 3.4: soft thresholding

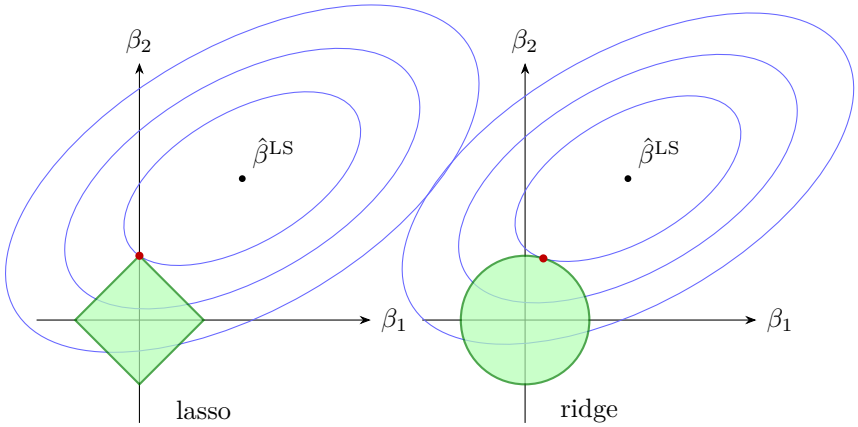
$\arg\min_b \frac{1}{2}(b-z)^2 + \lambda|b| = \operatorname{sign}(z)\max(|z|-\lambda, 0)$, whereas $\arg\min_b \frac{1}{2}(b-z)^2 + \frac{\lambda}{2}b^2 = z/(1+\lambda)$.

Proof. Strict convexity; optimality is $0 \in b - z + \lambda\,\partial|b|$ with $\partial|b| = \{\operatorname{sign} b\}$ $(b \neq 0)$, $[-1, 1]$ $(b = 0)$. $b > 0$: $b = z - \lambda$, iff $z > \lambda$. $b < 0$: $b = z + \lambda$, iff $z < -\lambda$. $b = 0$: iff $|z| \leq \lambda$. Ridge: $b - z + \lambda b = 0$. ■

Lasso shifts coefficients by λ and zeroes small ones; ridge rescales and never zeroes.



Unit balls $\sum_j |\beta_j|^q \leq 1$.



The RSS contour meets the lasso diamond at a corner (a zero) and the ridge disk at a generic point.

4 Kernel Ridge Regression

Motivation. Ridge in feature space needs only inner products, so a kernel gives infinitely many features for an $n \times n$ solve.

4.1 Feature-space and sample-space forms

Feature map $\boldsymbol{\phi}(\mathbf{x}) \in \mathbb{R}^m$, $\Phi = [\boldsymbol{\phi}(\mathbf{x}_1) \cdots \boldsymbol{\phi}(\mathbf{x}_n)] \in \mathbb{R}^{m \times n}$. Ridge $J = \frac{1}{2}\|\mathbf{y} - \Phi^\top \mathbf{w}\|^2 + \frac{\lambda}{2}\|\mathbf{w}\|^2$ gives

$$\nabla J = -\Phi\mathbf{y} + (\lambda \mathbf{I}_m + \Phi\Phi^\top)\mathbf{w}, \qquad \hat{\mathbf{w}} = (\lambda \mathbf{I}_m + \Phi\Phi^\top)^{-1}\Phi\mathbf{y}.$$

Theorem 3.5: push-through identity

$(\lambda \mathbf{I}_m + \Phi\Phi^\top)^{-1}\Phi = \Phi(\lambda \mathbf{I}_n + \Phi^\top\Phi)^{-1}$. Hence with the Gram matrix $K = \Phi^\top\Phi$, $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j) = \boldsymbol{\phi}(\mathbf{x}_i)^\top \boldsymbol{\phi}(\mathbf{x}_j)$,

$$\hat{\mathbf{w}} = \Phi\boldsymbol{\alpha}, \quad \boldsymbol{\alpha} = (K + \lambda \mathbf{I}_n)^{-1}\mathbf{y}, \qquad f(\mathbf{x}) = \boldsymbol{\phi}(\mathbf{x})^\top \hat{\mathbf{w}} = \sum_{i=1}^{n} \alpha_i k(\mathbf{x}_i, \mathbf{x}) = \mathbf{k}_{\mathbf{x}}^\top (K + \lambda \mathbf{I}_n)^{-1}\mathbf{y}.$$

Proof. $\Phi(\lambda \mathbf{I}_n + \Phi^\top\Phi) = \lambda\Phi + \Phi\Phi^\top\Phi = (\lambda \mathbf{I}_m + \Phi\Phi^\top)\Phi$. Multiply by $(\lambda \mathbf{I}_m + \Phi\Phi^\top)^{-1}$ on the left and $(\lambda \mathbf{I}_n + \Phi^\top\Phi)^{-1}$ on the right (both positive definite). ■

Remark 3.6: the kernel trick and cost

Only k is needed. Primal: $m \times m$ system; dual: $n \times n$, better when $m \gg n$ (Gaussian kernel: $m = \infty$). Substituting $\mathbf{w} = \Phi\boldsymbol{\alpha}$ gives $K[(K + \lambda \mathbf{I})\boldsymbol{\alpha} - \mathbf{y}] = \mathbf{0}$; for singular K the displayed $\boldsymbol{\alpha}$ is one solution and all give the same $\hat{\mathbf{w}}$.

4.2 Kernels

- Linear: $k(\mathbf{x}, \mathbf{z}) = \mathbf{x}^\top \mathbf{z}$.
- Quadratic in 2D: $(\mathbf{x}^\top \mathbf{z})^2 = x_1^2 z_1^2 + x_2^2 z_2^2 + 2x_1 x_2 z_1 z_2$, so $\boldsymbol{\phi}(\mathbf{x}) = (x_1^2, x_2^2, \sqrt{2}x_1 x_2)$. In 1D, $(1 + xz)^2$ has $\boldsymbol{\phi}(x) = (1, \sqrt{2}x, x^2)$.
- Gaussian: $k(\mathbf{x}, \mathbf{z}) = \exp(-\|\mathbf{x} - \mathbf{z}\|^2/(2\sigma^2))$; σ is the bandwidth.
- Normalized: $\tilde{k}(\mathbf{x}, \mathbf{z}) = k(\mathbf{x}, \mathbf{z})/\sqrt{k(\mathbf{x}, \mathbf{x})k(\mathbf{z}, \mathbf{z})}$.

Theorem 3.7: Gram matrices are PSD

For any feature-map kernel and any $\mathbf{a} \in \mathbb{R}^n$, $\mathbf{a}^\top K \mathbf{a} = \|\sum_i a_i \boldsymbol{\phi}(\mathbf{x}_i)\|^2 \geq 0$.

A similarity function is a valid kernel only if all its Gram matrices are PSD.

4.3 Worked examples

Example 3.8: linear kernel

$\mathbf{x}_1 = (1, 2)$, $\mathbf{x}_2 = (2, 0)$, $\mathbf{y} = (3, 1)$, $\lambda = 1$: $K = \begin{pmatrix} 5 & 2 \\ 2 & 4 \end{pmatrix}$, $(K + \mathbf{I})^{-1} = \frac{1}{26}\begin{pmatrix} 5 & -2 \\ -2 & 6 \end{pmatrix}$, $\boldsymbol{\alpha} = (1/2, 0)$, $\hat{\mathbf{w}} = \frac{1}{2}\mathbf{x}_1 = (1/2, 1)$. At $\mathbf{x} = (1, 0)$: $\mathbf{k}_{\mathbf{x}} = (1, 2)$ and $f(\mathbf{x}) = 1/2 = \hat{\mathbf{w}}^\top \mathbf{x}$.

Example 3.9: quadratic kernel

$\mathbf{x}_1 = (1, 1)$, $\mathbf{x}_2 = (2, 0)$, $\mathbf{y} = (3, 2)$, $k = (\mathbf{x}^\top \mathbf{z})^2$, $\lambda = 1$. Then $K_{11} = 4$, $K_{12} = 4$, $K_{22} = 16$,

$$(K + \mathbf{I})^{-1} = \frac{1}{69}\begin{pmatrix} 17 & -4 \\ -4 & 5 \end{pmatrix}, \qquad \boldsymbol{\alpha} = \frac{1}{69}\begin{pmatrix} 43 \\ -2 \end{pmatrix}.$$

At $\mathbf{x}_t = (1, 0)$, $\mathbf{k}_{\mathbf{x}_t} = (1, 4)$ and $f(\mathbf{x}_t) = (43 - 8)/69 = 35/69 \approx 0.507$. Check: $\boldsymbol{\phi}(\mathbf{x}_1) = (1, 1, \sqrt{2})$, $\boldsymbol{\phi}(\mathbf{x}_2) = (4, 0, 0)$, $\hat{\mathbf{w}} = \Phi\boldsymbol{\alpha} = (35, 43, 43\sqrt{2})/69$ and $\boldsymbol{\phi}(\mathbf{x}_t) = (1, 0, 0)$ give the same $35/69$.

5 Selected exercises

Motivation. Ridge as augmented least squares, thresholding under orthonormal design, and kernel validity.

Exercise 3.1 (Ridge as augmented least squares; effective degrees of freedom)

Let $X \in \mathbb{R}^{n\times d}$, $X \neq 0$, $\lambda > 0$, and $J(\boldsymbol{\beta}) = \frac{1}{2}\|\mathbf{y} - X\boldsymbol{\beta}\|^2 + \frac{\lambda}{2}\|\boldsymbol{\beta}\|^2$.

(a) Let $\tilde{X} = \begin{pmatrix} X \\ \sqrt{\lambda}\,\mathbf{I}_d \end{pmatrix}$ and $\tilde{\mathbf{y}} = \begin{pmatrix} \mathbf{y} \\ \mathbf{0} \end{pmatrix}$. Show that $J(\boldsymbol{\beta}) = \frac{1}{2}\|\tilde{\mathbf{y}} - \tilde{X}\boldsymbol{\beta}\|^2$, that $\tilde{X}$ always has full column rank, and recover $\hat{\boldsymbol{\beta}}_\lambda$ from the normal equations of Chapter 1.

(b) The fitted values are $X\hat{\boldsymbol{\beta}}_\lambda = H_\lambda \mathbf{y}$ with $H_\lambda = X(X^\top X + \lambda \mathbf{I})^{-1}X^\top$. Show that $\mathrm{df}(\lambda) := \operatorname{tr} H_\lambda = \sum_{i=1}^{r} \frac{s_i^2}{s_i^2+\lambda}$, where $s_1, \dots, s_r$ are the positive singular values of X.

(c) Show that df is strictly decreasing on $(0, \infty)$, tends to $r = \operatorname{rank} X$ as $\lambda \to 0^+$ and to 0 as $\lambda \to \infty$.

Solution. [Supplement]

(a) Blockwise, $\|\tilde{\mathbf{y}} - \tilde{X}\boldsymbol{\beta}\|^2 = \|\mathbf{y} - X\boldsymbol{\beta}\|^2 + \|\sqrt{\lambda}\,\boldsymbol{\beta}\|^2 = 2J(\boldsymbol{\beta})$. $\tilde{X}\mathbf{v} = \mathbf{0}$ forces $\sqrt{\lambda}\,\mathbf{v} = \mathbf{0}$: full column rank. With $\tilde{X}^\top\tilde{X} = X^\top X + \lambda\mathbf{I}$, $\tilde{X}^\top\tilde{\mathbf{y}} = X^\top\mathbf{y}$, Chapter 1 gives the unique minimizer $\hat{\boldsymbol{\beta}}_\lambda$.

(b) Full SVD $X = USV^\top$, U, V orthogonal, $S_{ii} = s_i$ $(i \le r)$, zero elsewhere. Then $(X^\top X + \lambda\mathbf{I})^{-1} = V(S^\top S + \lambda\mathbf{I})^{-1}V^\top$ and

$$H_\lambda = US(S^\top S + \lambda \mathbf{I}_d)^{-1}S^\top U^\top.$$

The middle matrix is diagonal with entries $s_i^2/(s_i^2+\lambda)$ $(i \le r)$ and 0; by cyclicity $\operatorname{tr} H_\lambda$ is their sum.

(c) $r \ge 1$ as $X \neq 0$. Each term has derivative $-s_i^2/(s_i^2+\lambda)^2 < 0$ and tends to 1 $(\lambda \to 0^+)$ and 0 $(\lambda \to \infty)$. For full column rank $\mathrm{df}(0^+) = d = \operatorname{tr} H$ (Exercise 1.1). ■

Exercise 3.2 (Orthonormal design: soft, hard and proportional shrinkage)

Assume $X^\top X = \mathbf{I}_d$, let $\lambda > 0$ and $\mathbf{z} = X^\top\mathbf{y}$ (the least squares estimate).

(a) Show that $\frac{1}{2}\|\mathbf{y} - X\boldsymbol{\beta}\|^2 = \frac{1}{2}\|\mathbf{z} - \boldsymbol{\beta}\|^2 + c$ with c independent of $\boldsymbol{\beta}$.

(b) Deduce that the lasso $\arg\min_{\boldsymbol{\beta}} \frac{1}{2}\|\mathbf{y} - X\boldsymbol{\beta}\|^2 + \lambda\|\boldsymbol{\beta}\|_1$ is $\hat{\beta}_j = \operatorname{sign}(z_j)\max(|z_j| - \lambda, 0)$, and that ridge with penalty $\frac{\lambda}{2}\|\boldsymbol{\beta}\|^2$ gives $\mathbf{z}/(1+\lambda)$.

(c) Show that best-subset selection $\arg\min_{\boldsymbol{\beta}} \frac{1}{2}\|\mathbf{y} - X\boldsymbol{\beta}\|^2 + \frac{\lambda^2}{2}\|\boldsymbol{\beta}\|_0$, where $\|\boldsymbol{\beta}\|_0$ is the number of nonzero entries, is hard thresholding: $\hat{\beta}_j = z_j$ if $|z_j| > \lambda$ and $\hat{\beta}_j = 0$ if $|z_j| < \lambda$.

Solution. [Supplement]

(a) Using $X^\top X = \mathbf{I}$ and $X^\top\mathbf{y} = \mathbf{z}$,

$$\|\mathbf{y} - X\boldsymbol{\beta}\|^2 = \mathbf{y}^\top\mathbf{y} - 2\boldsymbol{\beta}^\top\mathbf{z} + \boldsymbol{\beta}^\top\boldsymbol{\beta} = \|\mathbf{z} - \boldsymbol{\beta}\|^2 + (\mathbf{y}^\top\mathbf{y} - \mathbf{z}^\top\mathbf{z}).$$

(b) By (a) the lasso objective is $c + \sum_j \left[\frac{1}{2}(z_j - \beta_j)^2 + \lambda|\beta_j|\right]$, separable in coordinates; soft thresholding gives $\hat{\beta}_j$, and the same theorem gives ridge $z_j/(1+\lambda)$.

(c) Again $c + \sum_j g_j(\beta_j)$, $g_j(b) = \frac{1}{2}(z_j - b)^2 + \frac{\lambda^2}{2}\mathbf{1}\{b \neq 0\}$: $g_j(0) = z_j^2/2$, and $g_j(b) \ge \lambda^2/2$ for $b \neq 0$ with equality at $b = z_j \neq 0$. If $|z_j| > \lambda$, $\lambda^2/2 < z_j^2/2$ and $b = z_j$ is the unique minimizer; if $|z_j| < \lambda$, $b = 0$. (At $|z_j| = \lambda$ both are optimal.) ■

Exercise 3.3 (Building valid kernels)

Call $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ *valid* if every Gram matrix $K = [k(\mathbf{x}_i, \mathbf{x}_j)]_{i,j=1}^n$ is symmetric positive semidefinite.
(a) Show that if k_1, k_2 are valid and $c \geq 0$, then $k_1 + k_2$ and ck_1 are valid, and so is $k(\mathbf{x}, \mathbf{z}) = g(\mathbf{x})g(\mathbf{z})$ for any $g : \mathcal{X} \to \mathbb{R}$.
(b) Show that if $A, B \in \mathbb{R}^{n \times n}$ are symmetric PSD, so is the entrywise product $A \circ B$; deduce that $k_1 k_2$ is valid.
(c) Deduce that $(1 + \mathbf{x}^\top \mathbf{z})^p$ ($p \in \mathbb{N}$) and the Gaussian kernel $\exp(-\|\mathbf{x} - \mathbf{z}\|^2/(2\sigma^2))$ are valid on $\mathbb{R}^m$.

Solution. [Supplement]

(a) Symmetry is clear. For $\mathbf{a} \in \mathbb{R}^n$: $\mathbf{a}^\top (K_1 + K_2)\mathbf{a} = \mathbf{a}^\top K_1 \mathbf{a} + \mathbf{a}^\top K_2 \mathbf{a} \geq 0$ and $\mathbf{a}^\top (cK_1)\mathbf{a} = c\,\mathbf{a}^\top K_1 \mathbf{a} \geq 0$. For $k = g(\mathbf{x})g(\mathbf{z})$, $\mathbf{a}^\top K \mathbf{a} = \sum_{i,j} a_i a_j g(\mathbf{x}_i) g(\mathbf{x}_j) = \big(\sum_i a_i g(\mathbf{x}_i)\big)^2 \geq 0$.

(b) By the spectral theorem $B = \sum_k \mu_k \mathbf{q}_k \mathbf{q}_k^\top$ with $\mu_k \geq 0$; put $\mathbf{b}_k = \sqrt{\mu_k}\,\mathbf{q}_k$, so $B_{ij} = \sum_k b_{k,i} b_{k,j}$. Then

$$\mathbf{a}^\top (A \circ B)\mathbf{a} = \sum_k \sum_{i,j} (a_i b_{k,i}) A_{ij} (a_j b_{k,j}) = \sum_k (\mathbf{a} \circ \mathbf{b}_k)^\top A (\mathbf{a} \circ \mathbf{b}_k) \geq 0.$$

$A \circ B$ is symmetric, and the Gram matrix of $k_1 k_2$ is $K_1 \circ K_2$.

(c) *Step 1 (polynomial kernel).* The linear kernel ($\mathbf{a}^\top K \mathbf{a} = \|\sum_i a_i \mathbf{x}_i\|^2$) and the constant 1 ($g \equiv 1$) are valid, so $1 + \mathbf{x}^\top \mathbf{z}$ is valid by (a) and its pth power by (b) and induction.

Step 2 (exponential kernel). $e(\mathbf{x}, \mathbf{z}) = \exp(\mathbf{x}^\top \mathbf{z}/\sigma^2)$ is the entrywise limit of the valid $e_N = \sum_{q=0}^N (\mathbf{x}^\top \mathbf{z}/\sigma^2)^q / q!$, so $\mathbf{a}^\top E \mathbf{a} = \lim_N \mathbf{a}^\top E_N \mathbf{a} \geq 0$.

Step 3 (Gaussian kernel). By $\|\mathbf{x} - \mathbf{z}\|^2 = \|\mathbf{x}\|^2 - 2\mathbf{x}^\top \mathbf{z} + \|\mathbf{z}\|^2$,

$$\exp\Big(-\frac{\|\mathbf{x} - \mathbf{z}\|^2}{2\sigma^2}\Big) = g(\mathbf{x})\, e(\mathbf{x}, \mathbf{z})\, g(\mathbf{z}), \qquad g(\mathbf{x}) = \exp\Big(-\frac{\|\mathbf{x}\|^2}{2\sigma^2}\Big),$$

which is valid by (b) applied to e and the kernel $g(\mathbf{x})g(\mathbf{z})$ of (a). ■

Chapter 4. Classification: Logistic Regression, Softmax and SVM

Logistic regression, multiclass classification, support vector machines

1 Odds, Logit and the Logistic Model

Motivation. Linear log-odds give the logistic model: a sigmoid of a linear score.

Odds $o = p/(1-p)$ ($p = 0.8$: odds 4) and

$$\operatorname{logit}(p) = \log \frac{p}{1-p} \in \mathbb{R}, \qquad \sigma(z) = \frac{1}{1+e^{-z}} = \operatorname{logit}^{-1}(z).$$

Definition 4.1: logistic regression

With augmented input $\mathbf{x} = (1, x_1, \dots, x_d)$ and $y \in \{0, 1\}$,

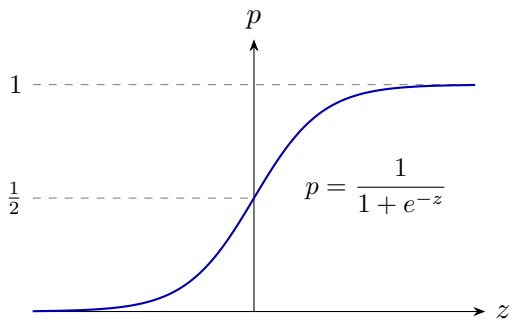
$$\log \frac{P(y=1 \mid \mathbf{x})}{P(y=0 \mid \mathbf{x})} = \mathbf{w}^\top \mathbf{x}, \qquad P(y=1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x}).$$

Thresholding at τ gives the hyperplane $\mathbf{w}^\top \mathbf{x} = \log \frac{\tau}{1-\tau}$ (at $\tau = 0.5$: $\mathbf{w}^\top \mathbf{x} = 0$).

Theorem 4.2: sigmoid identities

$\sigma'(z) = \sigma(z)(1-\sigma(z)) \le \frac{1}{4}$, $1 - \sigma(z) = \sigma(-z)$, $\frac{d}{dz} \log(1+e^z) = \sigma(z)$.

Proof. $\sigma'(z) = e^{-z}/(1+e^{-z})^2 = \sigma(z) \cdot \frac{e^{-z}}{1+e^{-z}}$ and $\frac{e^{-z}}{1+e^{-z}} = \frac{1}{e^z+1} = \sigma(-z) = 1 - \sigma(z)$. The maximum of $s(1-s)$ on $[0,1]$ is $1/4$ at $s = 1/2$. ■



The logistic function, inverse of the logit $z = \log \frac{p}{1-p}$.

2 Likelihood, Gradient, Hessian and Convexity

Motivation. No closed form, but a PSD Hessian: the loss is convex and gradient methods are reliable.

For data $(\mathbf{x}_i, y_i)$ with $p_i = \sigma(\mathbf{w}^\top \mathbf{x}_i)$, $p(y_i \mid \mathbf{x}_i, \mathbf{w}) = p_i^{y_i}(1-p_i)^{1-y_i}$ and

$$J(\mathbf{w}) = -\sum_i \left[y_i \log p_i + (1-y_i) \log(1-p_i) \right] = \sum_i \left[\log(1+e^{z_i}) - y_i z_i \right], \quad z_i = \mathbf{w}^\top \mathbf{x}_i.$$

Theorem 4.3: logistic loss is convex

$\nabla J = \sum_i (p_i - y_i)\mathbf{x}_i = X^\top(\mathbf{p}-\mathbf{y})$ and $\nabla^2 J = \sum_i p_i(1-p_i)\mathbf{x}_i\mathbf{x}_i^\top = X^\top S X$, $S = \mathrm{diag}(p_i(1-p_i))$. Hence J is convex, and strictly convex if X has full column rank.

Proof. $\partial J/\partial z_i = \sigma(z_i) - y_i = p_i - y_i$, $\partial z_i/\partial \mathbf{w} = \mathbf{x}_i$, $\nabla p_i = p_i(1-p_i)\mathbf{x}_i$. Then $\mathbf{v}^\top X^\top S X \mathbf{v} = \sum_i p_i(1-p_i)(\mathbf{x}_i^\top \mathbf{v})^2 \geq 0$, positive when $X\mathbf{v} \neq \mathbf{0}$ since $p_i(1-p_i) > 0$. ■

Example 4.4: one gradient step

$\mathbf{x} = (1, 2)$, $y = 1$, $\mathbf{w} = \mathbf{0}$: $p = 1/2$, $\nabla J = (p-1)\mathbf{x} = (-1/2, -1)$. With $\eta = 0.1$, $\mathbf{w} = (0.05, 0.1)$, score 0.25, $p \approx 0.562$ (up).

3 Labels in $\{\pm 1\}$, Logistic Loss and BCE

Motivation. With labels $\{\pm 1\}$ the loss depends on the margin yf, linking logistic regression to the SVM.

With $Y \in \{\pm 1\}$, $f(\mathbf{x}) = \mathbf{a}^\top \mathbf{x} + b$, $P(Y = \pm 1 \mid \mathbf{x}) = \sigma(\pm f)$, so $-\log P(Y \mid \mathbf{x}) = \ell(Yf)$ with the **logistic loss** $\ell(z) = \log(1 + e^{-z})$; the *margin* $Yf(\mathbf{x})$ is positive iff the sign is correct.

Theorem 4.5: three names for one objective

Let $P(-1) = (1, 0)$, $P(+1) = (0, 1)$, $\mu(r) = (\sigma(-r), \sigma(r))$, $\tilde{Y} = (1 + Y)/2 \in \{0, 1\}$ and $\mathrm{BCE}(s, t) = -[t \log s + (1 - t)\log(1 - s)]$. For every $(\mathbf{a}, b)$ and every i,

$$D_{\mathrm{KL}}\big(P(Y_i) \,\|\, \mu(f(\mathbf{x}_i))\big) = \mathrm{BCE}\left(\sigma(f(\mathbf{x}_i)), \tilde{Y}_i\right) = \ell\big(Y_i f(\mathbf{x}_i)\big).$$

Proof. One-hot KL $= -\log$(model probability of the true class). $Y = +1$: $-\log \sigma(r) = \ell(r)$, and BCE with $t = 1$ is $-\log s$. $Y = -1$: $-\log \sigma(-r) = \ell(-r)$, and BCE with $t = 0$ is $-\log(1 - s) = -\log \sigma(-r)$. ■

Theorem 4.6: ℓ is convex

$\ell'(z) = \sigma(z) - 1$ and $\ell''(z) = \sigma(z)\sigma(-z) > 0$.

3.1 Surrogate risk bounds

For $f = \mathrm{sign}(g)$ ($g = 0$ counted as an error), $\{f(X) \neq Y\} \subseteq \{Yg(X) \leq 0\}$, so for nonnegative nonincreasing ϕ, $\phi(Yg) \geq \phi(0)$ on errors and

$$P(f(X) \neq Y) \leq \frac{\mathbb{E}[\phi(Yg(X))]}{\phi(0)}.$$

Logistic: $\phi(0) = \log 2$, so $R_{\log} < \varepsilon \Rightarrow P(f(X) = Y) > 1 - \varepsilon/\log 2$. Hinge $\max(0, 1 - u)$ and exponential e^{-u} have $\phi(0) = 1$. A convex surrogate thus controls the 0–1 error.

4 Multiclass Classification and Softmax

Motivation. Softmax maps K scores to probabilities, with gradient "prediction minus target".

For Q classes, $W \in \mathbb{R}^{Q\times d}$, scores $s_k = \mathbf{w}_k^\top \mathbf{x}$ and

$$p_k(\mathbf{x}) = \operatorname{softmax}(\mathbf{s})_k = \frac{e^{s_k}}{\sum_r e^{s_r}}, \qquad J(W) = -\sum_{i,k} y_{ik} \log p_k(\mathbf{x}_i).$$

Scores $(2, 1, 0)$ give approximately $(0.665, 0.245, 0.090)$.

Theorem 4.7: softmax calculus

With $\operatorname{LSE}(\mathbf{s}) = \log \sum_r e^{s_r}$ and $\mathbf{p} = \operatorname{softmax}(\mathbf{s})$:

(i) $\operatorname{softmax}(\mathbf{s} + c\mathbf{1}) = \operatorname{softmax}(\mathbf{s})$, $\operatorname{LSE}(\mathbf{s} + c\mathbf{1}) = \operatorname{LSE}(\mathbf{s}) + c$; stable evaluation $\operatorname{LSE}(\mathbf{s}) = M + \log \sum e^{s_r - M}$, $M = \max s_r$.

(ii) $\nabla \operatorname{LSE} = \mathbf{p}$; Jacobian $\partial p_k / \partial s_m = p_k(\delta_{km} - p_m)$, i.e. $\nabla^2 \operatorname{LSE} = \operatorname{diag}(\mathbf{p}) - \mathbf{p}\mathbf{p}^\top \succeq 0$.

(iii) $\nabla_{\mathbf{s}}(-\log p_y) = \mathbf{p} - \mathbf{e}_y$; for the whole data set $\nabla_W J = (P - Y)^\top X$ (rows: classes).

(iv) $\log \operatorname{softmax}(\mathbf{s}) = \mathbf{s} - \operatorname{LSE}(\mathbf{s})\mathbf{1}$ and log-softmax is idempotent: $\log \operatorname{softmax}(\log \operatorname{softmax}(\mathbf{s})) = \log \operatorname{softmax}(\mathbf{s})$.

(v) $\max_k s_k \le \operatorname{LSE}(\mathbf{s}) \le \max_k s_k + \log Q$.

Proof. (i) e^c cancels.

(ii) Quotient rule; $\mathbf{v}^\top(\operatorname{diag}\mathbf{p} - \mathbf{p}\mathbf{p}^\top)\mathbf{v} = \sum p_k v_k^2 - (\sum p_k v_k)^2$ is the variance of v_K with $K \sim \mathbf{p}$.

(iii) $-\log p_y = -s_y + \operatorname{LSE}(\mathbf{s})$.

(iv) With $\boldsymbol{\ell} = \log \operatorname{softmax}(\mathbf{s})$, $e^{\ell_j} = p_j$ so $\operatorname{LSE}(\boldsymbol{\ell}) = \log \sum p_j = 0$.

(v) $e^M \le \sum e^{s_r} \le Q e^M$. ∎

Adding one vector to all rows of W leaves the softmax unchanged: W is identifiable only up to shifts.

5 Support Vector Machines

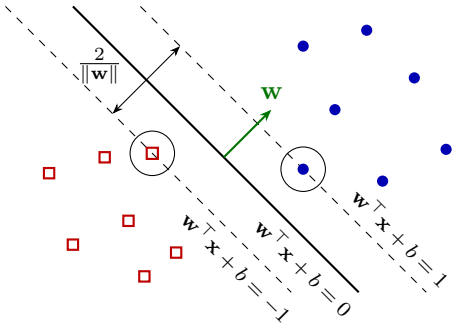
5.1 Geometry and the primal problem

$y_i \in \{\pm 1\}$, $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$. The distance from $\mathbf{x}_0$ to $\{\mathbf{w}^\top \mathbf{x} + b = 0\}$ is $|\mathbf{w}^\top \mathbf{x}_0 + b|/\|\mathbf{w}\|$ ($\mathbf{x}_0 - t\mathbf{w}/\|\mathbf{w}\|$ is on the plane iff $t = (\mathbf{w}^\top \mathbf{x}_0 + b)/\|\mathbf{w}\|$). $(\mathbf{w}, b)$ is scale-free, so normalize the closest functional margin to 1:

Definition 4.8: hard-margin SVM

$$\min_{\mathbf{w},b} \ \tfrac{1}{2}\|\mathbf{w}\|^2 \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \ge 1, \ i = 1, \dots, n.$$

The support planes $\mathbf{w}^\top \mathbf{x} + b = \pm 1$ lie at distance $1/\|\mathbf{w}\|$ (the margin) from the boundary; the slab width is $2/\|\mathbf{w}\|$.



Slab width $2/\|\mathbf{w}\|$; circled points are support vectors.

5.2 Lagrangian, max–min inequality, weak duality

Theorem 4.9: max–min inequality

For any $F : \mathcal{X} \times \mathcal{Y} \to \mathbb{R}$, $\sup_x \inf_y F(x,y) \le \inf_y \sup_x F(x,y)$. Equality can fail: for $F = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ (rows x, columns y) the left side is 0 and the right side 1.

Proof. $\inf_{y'} F(x,y') \le F(x,y) \le \sup_{x'} F(x',y)$; take $\sup_x$ of the left end and $\inf_y$ of the right. ■

With $c_i = 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)$, $\alpha_i \ge 0$, $L(\mathbf{w}, b, \boldsymbol{\alpha}) = \frac{1}{2}\|\mathbf{w}\|^2 + \sum_i \alpha_i c_i$. As $\sup_{\alpha \ge 0} \alpha c$ is 0 ($c \le 0$) or $+\infty$ ($c > 0$), $p^\star = \inf_{\mathbf{w},b} \sup_{\boldsymbol{\alpha}} L$. The dual $d^\star = \sup_{\boldsymbol{\alpha}} \inf_{\mathbf{w},b} L$ satisfies **weak duality** $d^\star \le p^\star$; for the SVM (convex, strictly feasible) $d^\star = p^\star$.

5.3 The dual problem

Theorem 4.10: SVM dual

$\nabla_{\mathbf{w}} L = \mathbf{0} \Rightarrow \mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i$; finiteness of $\inf_b L$ requires $\sum_i \alpha_i y_i = 0$. The dual is

$$\max_{\boldsymbol{\alpha} \ge \mathbf{0}} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j \quad \text{s.t.} \quad \sum_i \alpha_i y_i = 0.$$

Complementary slackness $\alpha_i[1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)] = 0$: $\alpha_i > 0$ only for support vectors, and $b = y_i - \mathbf{w}^\top \mathbf{x}_i$ for any such i.

Proof. b enters L linearly with coefficient $-\sum \alpha_i y_i$; if nonzero, $\inf_b = -\infty$. Substituting $\mathbf{w}$: $\frac{1}{2}\|\mathbf{w}\|^2 - \|\mathbf{w}\|^2 + \sum \alpha_i = \sum \alpha_i - \frac{1}{2}\|\mathbf{w}\|^2$. ■

Example 4.11: two points

$\mathbf{x}_1 = (0,0)$, $y_1 = -1$; $\mathbf{x}_2 = (2,2)$, $y_2 = +1$. $\alpha_1 = \alpha_2 = a$, $\|\mathbf{x}_2\|^2 = 8$: dual $2a - 4a^2$, maximal at $a = 1/4$. Then $\mathbf{w} = (1/2, 1/2)$, $b = -1$, boundary $x^{(1)} + x^{(2)} = 2$, margin $\sqrt{2}$ (slab $2\sqrt{2}$); primal $\frac{1}{2}\|\mathbf{w}\|^2 = 1/4 =$ dual.

Remark 4.12: soft margin and kernels

Slacks: $\min \frac{1}{2}\|\mathbf{w}\|^2 + C\sum \xi_i$ s.t. $y_i f(\mathbf{x}_i) \ge 1 - \xi_i$, $\xi_i \ge 0$ (hinge loss plus ℓ_2); the dual adds $0 \le \alpha_i \le C$. A kernel in place of $\mathbf{x}_i^\top \mathbf{x}_j$ gives score $\sum_i \alpha_i y_i k(\mathbf{x}_i, \mathbf{x}) + b$.

6 Selected exercises

Motivation. Two-class softmax, the population target of the logistic loss, and the SVM margin from the dual.

Exercise 4.1 (Two-class softmax is logistic regression)

Let $Q = 2$, scores $s_k = \mathbf{w}_k^\top \mathbf{x}$, labels $y_i = y_{i1} \in \{0, 1\}$, $y_{i2} = 1 - y_i$, and $J(\mathbf{w}_1, \mathbf{w}_2)$ the softmax cross-entropy.
(a) Show that $p_1(\mathbf{x}) = \sigma((\mathbf{w}_1 - \mathbf{w}_2)^\top \mathbf{x})$ and $J(\mathbf{w}_1, \mathbf{w}_2) = J_{\log}(\mathbf{w}_1 - \mathbf{w}_2)$, where $J_{\log}$ is the logistic loss of Section 2.
(b) Show that $\nabla_{\mathbf{w}_1} J = -\nabla_{\mathbf{w}_2} J$ and that the Hessian of J in $(\mathbf{w}_1, \mathbf{w}_2)$ is singular.
(c) For $\lambda > 0$ let $(\hat{\mathbf{w}}_1, \hat{\mathbf{w}}_2)$ minimize $J + \frac{\lambda}{2}(\|\mathbf{w}_1\|^2 + \|\mathbf{w}_2\|^2)$. Show that $\hat{\mathbf{w}}_1 = -\hat{\mathbf{w}}_2$ and that $\hat{\mathbf{w}} = \hat{\mathbf{w}}_1 - \hat{\mathbf{w}}_2$ minimizes $J_{\log}(\mathbf{w}) + \frac{\lambda}{4}\|\mathbf{w}\|^2$.

Solution. [Supplement]

(a) Divide by e^{s_1}:

$$p_1 = \frac{e^{s_1}}{e^{s_1} + e^{s_2}} = \frac{1}{1 + e^{-(s_1 - s_2)}} = \sigma((\mathbf{w}_1 - \mathbf{w}_2)^\top \mathbf{x}), \qquad p_2 = 1 - p_1.$$

Hence $J = -\sum_i [y_i \log p_1(\mathbf{x}_i) + (1 - y_i) \log(1 - p_1(\mathbf{x}_i))]$, the logistic loss at $\mathbf{w} = \mathbf{w}_1 - \mathbf{w}_2$.

(b) By (a) and the chain rule, $\nabla_{\mathbf{w}_1} J = \nabla J_{\log}(\mathbf{w}_1 - \mathbf{w}_2)$ and $\nabla_{\mathbf{w}_2} J = -\nabla J_{\log}(\mathbf{w}_1 - \mathbf{w}_2)$. $t \mapsto J(\mathbf{w}_1 + t\mathbf{v}, \mathbf{w}_2 + t\mathbf{v})$ is constant, so $\mathbf{u} = (\mathbf{v}, \mathbf{v}) \neq \mathbf{0}$ has $\mathbf{u}^\top \mathcal{H} \mathbf{u} = 0$; J is convex, $\mathcal{H} \succeq 0$, so $\|\mathcal{H}^{1/2}\mathbf{u}\|^2 = 0$ and $\mathcal{H}\mathbf{u} = \mathbf{0}$.

(c) The penalized objective is strictly convex and coercive; its unique minimizer satisfies

$$\nabla_{\mathbf{w}_1} J + \lambda \hat{\mathbf{w}}_1 = \mathbf{0}, \qquad \nabla_{\mathbf{w}_2} J + \lambda \hat{\mathbf{w}}_2 = \mathbf{0}.$$

Adding and using (b), $\lambda(\hat{\mathbf{w}}_1 + \hat{\mathbf{w}}_2) = \mathbf{0}$, so $\hat{\mathbf{w}}_1 = -\hat{\mathbf{w}}_2 = \hat{\mathbf{w}}/2$. On pairs $(\mathbf{w}/2, -\mathbf{w}/2)$ the objective is $J_{\log}(\mathbf{w}) + \frac{\lambda}{2} \cdot 2\|\mathbf{w}/2\|^2 = J_{\log}(\mathbf{w}) + \frac{\lambda}{4}\|\mathbf{w}\|^2$, and the global minimizer lies there, so $\hat{\mathbf{w}}$ minimizes it. ■

Exercise 4.2 (What logistic regression estimates)

Let $(X, Y) \sim P$ with $Y \in \{\pm 1\}$ and $\eta(\mathbf{x}) = P(Y = +1 \mid X = \mathbf{x}) \in (0, 1)$. For measurable g let $R(g) = \mathbb{E}[\ell(Yg(X))]$ with $\ell(z) = \log(1 + e^{-z})$.
(a) For $\eta \in (0, 1)$ show that $C_\eta(t) = \eta\, \ell(t) + (1 - \eta)\ell(-t)$ is strictly convex with unique minimizer $t^\star = \operatorname{logit}(\eta)$.
(b) Deduce that $g^\star(\mathbf{x}) = \operatorname{logit} \eta(\mathbf{x})$ minimizes R over all measurable g, and that $\operatorname{sign} g^\star$ is the Bayes classifier (predict $+1$ iff $\eta > \frac{1}{2}$).
(c) Show that $C_\eta(t^\star) = -\eta \log \eta - (1 - \eta) \log(1 - \eta)$, so $R(g^\star)$ is the expected binary entropy of $\eta(X)$.

Solution. [Supplement]

(a) $\ell'(z) = \sigma(z) - 1 = -\sigma(-z)$, so $\frac{d}{dt}\ell(-t) = \sigma(t)$ and

$$C_\eta'(t) = -\eta\, \sigma(-t) + (1 - \eta)\sigma(t) = -\eta(1 - \sigma(t)) + (1 - \eta)\sigma(t) = \sigma(t) - \eta.$$

$C_\eta'' = \sigma' > 0$: strictly convex, unique stationary point $\sigma(t) = \eta$, $t = \operatorname{logit}(\eta)$.

(b) $R(g) = \mathbb{E}\, C_{\eta(X)}(g(X)) \geq \mathbb{E}\, C_{\eta(X)}(g^\star(X)) = R(g^\star)$ pointwise by (a); $R(g^\star) \leq \log 2$ is finite and $g^\star$ is measurable. $\operatorname{logit} \eta > 0$ iff $\eta > \frac{1}{2}$.

(c) $\sigma(t^\star) = \eta$ gives $\ell(t^\star) = -\log \eta$, $\ell(-t^\star) = -\log(1-\eta)$; substitute. The logistic loss estimates the conditional log-odds, so outputs are probabilities. ■

Exercise 4.3 (The margin from the dual variables)

Let the data be linearly separable with both labels present, let $(\mathbf{w}^\star, b^\star)$ solve the hard-margin primal and $\boldsymbol{\alpha}^\star$ the dual, with the KKT conditions of the dual theorem in force.
(a) Show that $\mathbf{w}^\star$ is unique, $\mathbf{w}^\star \neq \mathbf{0}$, and $b^\star$ is uniquely determined.
(b) Show that $\|\mathbf{w}^\star\|^2 = \sum_i \alpha_i^\star$; hence the margin is $\big(\sum_i \alpha_i^\star\big)^{-1/2}$ and the primal and dual values coincide.
(c) Check (b) on the two-point example.

Solution. [Supplement]

(a) *Step 1 (uniqueness of $\mathbf{w}^\star$).* The feasible set is convex (half-spaces). If $(\mathbf{w}_1, b_1), (\mathbf{w}_2, b_2)$ are optimal with $\mathbf{w}_1 \neq \mathbf{w}_2$, the midpoint is feasible and by the parallelogram identity

$$\tfrac{1}{2}\Big\|\tfrac{\mathbf{w}_1+\mathbf{w}_2}{2}\Big\|^2 = \tfrac{1}{4}\|\mathbf{w}_1\|^2 + \tfrac{1}{4}\|\mathbf{w}_2\|^2 - \tfrac{1}{8}\|\mathbf{w}_1 - \mathbf{w}_2\|^2 < p^\star,$$

a contradiction.

Step 2 ($\mathbf{w}^\star \neq \mathbf{0}$). $\mathbf{w} = \mathbf{0}$ needs $y_i b \geq 1$ for all i, i.e. $b \geq 1$ and $b \leq -1$.

Step 3 ($b^\star$). $\mathbf{w}^\star = \sum_i \alpha_i^\star y_i \mathbf{x}_i \neq \mathbf{0}$, so some $\alpha_i^\star > 0$; complementary slackness and $y_i^2 = 1$ give $b^\star = y_i - \mathbf{w}^{\star\top}\mathbf{x}_i$.

(b) Sum complementary slackness and use $\sum_i \alpha_i^\star y_i = 0$:

$$\sum_i \alpha_i^\star = \sum_i \alpha_i^\star y_i(\mathbf{w}^{\star\top}\mathbf{x}_i + b^\star) = \mathbf{w}^{\star\top}\sum_i \alpha_i^\star y_i \mathbf{x}_i + b^\star \sum_i \alpha_i^\star y_i = \|\mathbf{w}^\star\|^2.$$

Margin $1/\|\mathbf{w}^\star\|$; primal $\tfrac{1}{2}\|\mathbf{w}^\star\|^2$ equals dual $\sum_i \alpha_i^\star - \tfrac{1}{2}\|\mathbf{w}^\star\|^2$.

(c) $\alpha_1 = \alpha_2 = \tfrac{1}{4}$: $\sum_i \alpha_i = \tfrac{1}{2} = \|(1/2, 1/2)\|^2$, margin $\sqrt{2}$. ■

Chapter 5. Neural Networks and Backpropagation

Basic concepts of neural networks, multilayer perceptrons, backpropagation

1 From Hand-Designed Features to Learned Functions

A classifier is $y = f(\mathbf{x}; \boldsymbol{\theta})$. Classical pipelines hand-design features (color histograms, filter banks); deep learning learns representation and classifier end to end. A feature is *invariant* to a transformation if unchanged, *equivariant* if it changes the same way (Chapter 7).

2 Neurons, Layers and Shapes

Definition 5.1: multilayer perceptron

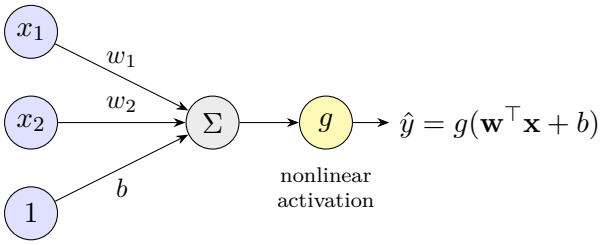
A neuron computes $h = \sigma(\mathbf{w}^\top \mathbf{x} + b)$. A fully connected layer stacks neurons:

$$\mathbf{z}^{(\ell)} = W_\ell \mathbf{a}^{(\ell-1)} + \mathbf{b}_\ell, \qquad \mathbf{a}^{(\ell)} = \sigma(\mathbf{z}^{(\ell)}), \qquad \mathbf{a}^{(0)} = \mathbf{x},$$

with $W_\ell \in \mathbb{R}^{n_\ell \times n_{\ell-1}}$, $\mathbf{b}_\ell \in \mathbb{R}^{n_\ell}$, and σ applied elementwise. An MLP is a composition of such layers.

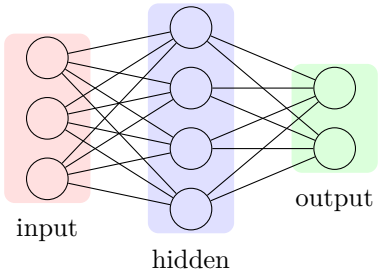
Example 5.2: parameter count

A two-layer classifier $\mathbf{s} = W_2 \,\mathrm{ReLU}(W_1 \mathbf{x})$ on CIFAR-10 images ($32 \times 32 \times 3 = 3072$ inputs), hidden width 100, 10 classes: $W_1 \in \mathbb{R}^{100 \times 3072}$, $W_2 \in \mathbb{R}^{10 \times 100}$, so $307{,}200 + 1000 = 308{,}200$ weights (plus 110 biases).



A neuron: affine combination, then activation g.

$$f(\mathbf{x}) = \mathrm{softmax}\left(W_2 \max(0, W_1 \mathbf{x})\right)$$



A fully connected two-layer network.

3 Why Nonlinearity and Depth

Without activations layers collapse: $W_2(W_1\mathbf{x}+\mathbf{b}_1)+\mathbf{b}_2 = (W_2W_1)\mathbf{x}+(W_2\mathbf{b}_1+\mathbf{b}_2)$ is affine, giving linear boundaries. Hidden layers are learned embeddings, e.g. $\phi(x_1,x_2)=(x_1,x_2,x_1^2+x_2^2)$ separates two concentric circles by a plane; depth composes them (strokes $\to$ loops $\to$ digits), without guaranteeing trainability or generalization (Chapters 6–8).

4 Activation Functions and Outputs

Activation	Definition	Derivative
Sigmoid	$\sigma(z)=(1+e^{-z})^{-1}\in(0,1)$	$\sigma(1-\sigma)\in(0,1/4]$
Tanh	$\tanh z = 2\sigma(2z)-1\in(-1,1)$	$1-\tanh^2 z\in(0,1]$
ReLU	$\max(0,z)$	$\mathbf{1}\{z>0\}$
Leaky ReLU	$\max(az,z)$, $0<a<1$	1 or a
ELU	z $(z\geq 0)$, $a(e^z-1)$ $(z<0)$	1 or ae^z
Maxout	$\max_k(\mathbf{w}_k^\top\mathbf{x}+b_k)$	routes to the max

Outputs: softmax for mutually exclusive classes (loss $-\log p_y$, score gradient $\mathbf{p}-\mathbf{e}_y$), sigmoid for binary probabilities, identity for regression.

Example 5.3: forward pass

$\mathbf{x}=(1,-2)$, $W_1=\begin{pmatrix}1 & 1\\ 2 & -1\end{pmatrix}$, $W_2=\mathbf{I}$, no biases, ReLU hidden layer: $\mathbf{z}_1=(-1,4)$, $\mathbf{a}_1=(0,4)$, scores $(0,4)$, softmax $\approx(0.018,0.982)$.

5 Computation Graphs and the Scalar Chain Rule

Training loop: zero gradients, forward, loss, backward, update. Backpropagation is the chain rule on a computation graph: each node multiplies the *upstream* derivative by its *local* derivative.

Example 5.4: $f=(x+y)z$

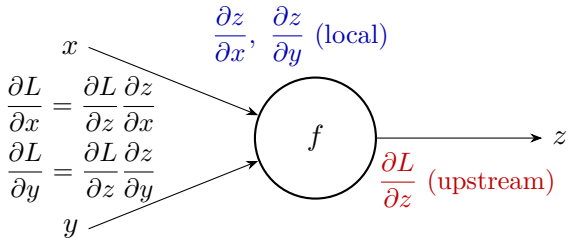
At $(x,y,z)=(-2,5,-4)$: $q=x+y=3$, $f=qz=-12$. Local derivatives $\partial f/\partial q=z=-4$, $\partial f/\partial z=q=3$, $\partial q/\partial x=\partial q/\partial y=1$, hence $\partial f/\partial x=\partial f/\partial y=-4$, $\partial f/\partial z=3$.

Example 5.5: sigmoid neuron

$f=\sigma(w_0x_0+w_1x_1+w_2)$ with $w_0=2, x_0=-1, w_1=-3, x_1=-2, w_2=-3$: $z=1$, $f\approx 0.731$, $\partial f/\partial z=f(1-f)\approx 0.197$. Then $\partial f/\partial w_0=0.197x_0=-0.197$, $\partial f/\partial x_0=0.197w_0=0.393$, $\partial f/\partial w_1=-0.393$, $\partial f/\partial x_1=-0.590$, $\partial f/\partial w_2=0.197$. Splitting σ into $z\to -z\to e^{-z}\to 1+e^{-z}\to 1/(\cdot)$ gives the same numbers.

Remark 5.6: gate patterns

Add: copies upstream to both inputs. Multiply: upstream $\times$ the other input. Copy: derivatives add (shared parameters accumulate). Max: routes to the maximal input.



Downstream gradient = upstream × local.

6 Vector Calculus Rules for Backpropagation

Motivation. Six vector rules differentiate any feedforward network, one backward vector per layer.

Gradient convention: $\nabla_{\mathbf{x}} L$ has the shape of $\mathbf{x}$; the Jacobian of $\mathbf{y} \in \mathbb{R}^m$ in $\mathbf{x} \in \mathbb{R}^n$ is $m \times n$ with entries $\partial y_i / \partial x_j$.

Theorem 5.7: rules R1–R6

R1. Sum rule: the Jacobian of a sum is the sum of Jacobians; terms not depending on the variable contribute 0.

R2. Identity: $\partial \mathbf{x} / \partial \mathbf{x} = \mathbf{I}$.

R3. Chain rule: $\frac{\partial \mathbf{z}}{\partial \mathbf{x}} = \frac{\partial \mathbf{z}}{\partial \mathbf{y}} \frac{\partial \mathbf{y}}{\partial \mathbf{x}}$ (outer Jacobian on the left).

R4. Affine map: $\partial(A\mathbf{x} + \mathbf{b})/\partial \mathbf{x} = A$.

R5. Elementwise map: $\partial \sigma(\mathbf{z})/\partial \mathbf{z} = \operatorname{diag}(\sigma'(\mathbf{z}))$, so $\operatorname{diag}(\sigma'(\mathbf{z}))\mathbf{v} = \sigma'(\mathbf{z}) \odot \mathbf{v}$.

R6. Gradient chain rule (scalar L): $\nabla_{\mathbf{x}} L = \left(\frac{\partial \mathbf{u}}{\partial \mathbf{x}}\right)^{\top} \nabla_{\mathbf{u}} L$.

Proof. R6: by R3 the row vector $\partial L / \partial \mathbf{x} = (\partial L / \partial \mathbf{u})(\partial \mathbf{u} / \partial \mathbf{x})$; transposing reverses the order. ■

Theorem 5.8: affine layer

For $\mathbf{z} = W\mathbf{a} + \mathbf{b}$ and $\boldsymbol{\delta} = \nabla_{\mathbf{z}} L$:

$$\nabla_W L = \boldsymbol{\delta}\, \mathbf{a}^{\top}, \qquad \nabla_{\mathbf{b}} L = \boldsymbol{\delta}, \qquad \nabla_{\mathbf{a}} L = W^{\top} \boldsymbol{\delta}.$$

For an activation layer $\mathbf{h} = \sigma(\mathbf{z})$: $\nabla_{\mathbf{z}} L = \nabla_{\mathbf{h}} L \odot \sigma'(\mathbf{z})$.

Proof. $\nabla_{\mathbf{a}} L$: R6 with R4. $\nabla_{\mathbf{b}} L$: R6 with $\partial \mathbf{z} / \partial \mathbf{b} = \mathbf{I}$ (R1, R2). $\nabla_W L$: row $\mathbf{w}_i^{\top}$ enters only through $z_i = \mathbf{w}_i^{\top} \mathbf{a} + b_i$, so $\nabla_{\mathbf{w}_i} L = \delta_i \mathbf{a}$ (R4); stacking gives $\boldsymbol{\delta}\mathbf{a}^{\top}$, i.e. $\partial L / \partial W_{ij} = \delta_i a_j$. Activation: R6 with R5 (diagonal is symmetric). ■

Remark 5.9: differentials and trace

If $dL = \operatorname{tr}(K\, dW)$ then $\nabla_W L = K^{\top}$. For $\mathbf{z} = W\mathbf{a}$, $dL = \boldsymbol{\delta}^{\top}(dW)\mathbf{a} = \operatorname{tr}(\mathbf{a}\boldsymbol{\delta}^{\top} dW)$, so $\nabla_W L = \boldsymbol{\delta}\mathbf{a}^{\top}$.

6.1 Backpropagation through an MLP

With $\boldsymbol{\delta}_\ell = \nabla_{\mathbf{z}^{(\ell)}} L$ (an $n_\ell \times 1$ column), the recursion is

$$\boldsymbol{\delta}_\ell = (W_{\ell+1}^\top \boldsymbol{\delta}_{\ell+1}) \odot \sigma'(\mathbf{z}^{(\ell)}), \qquad \nabla_{W_\ell} L = \boldsymbol{\delta}_\ell \, \mathbf{a}^{(\ell-1)\top}, \qquad \nabla_{\mathbf{b}_\ell} L = \boldsymbol{\delta}_\ell.$$

(R6 through the affine map, then the activation.) In a batch, outer products are summed or averaged.

Example 5.10: forward example continued backward

Class 1 correct: $\boldsymbol{\delta}_2 = \mathbf{p} - \mathbf{e}_1 \approx (-0.982, 0.982)$, $\nabla_{W_2} L = \boldsymbol{\delta}_2 \mathbf{a}_1^\top = \begin{pmatrix} 0 & -3.928 \\ 0 & 3.928 \end{pmatrix}$. The first pre-activation is negative, so $\boldsymbol{\delta}_1 = (0, 0.982)$, $\nabla_{W_1} L = \boldsymbol{\delta}_1 \mathbf{x}^\top = \begin{pmatrix} 0 & 0 \\ 0.982 & -1.964 \end{pmatrix}$: the inactive unit gets no gradient. ReLU maps upstream $(4, -1, 5, 9)$ at $\mathbf{x} = (1, -2, 3, -1)$ to $(4, 0, 5, 0)$, no Jacobian formed.

7 Worked Results

Motivation. Why cross-entropy beats squared error for a sigmoid neuron, and why residual paths keep gradients alive.

7.1 Single neurons: MSE versus cross-entropy

Theorem 5.11: gradient bounds

For $y = \sigma(wx + b)$ and $C = \frac{1}{2}(y - z)^2$: $\nabla C = y(1-y)(y-z)(x, 1)$ and $\|\nabla C\| < \frac{1}{4}\sqrt{1 + x^2}\,(1 + |z|)$. For cross-entropy $C = -[z \log y + (1 - z) \log(1 - y)]$: $\nabla C = (y - z)(x, 1)$.

Proof. Chain rule with $\sigma' = y(1 - y) \le 1/4$ and $|y - z| \le |y| + |z| < 1 + |z|$ since $0 < y < 1$. For cross-entropy, $\partial C / \partial t = -[z(1 - y) - (1 - z)y] = y - z$: the factor σ' cancels. ■

For a saturated wrong neuron ($z = 1$, $wx + b = -10$) the MSE gradient carries $\sigma' \approx 4.5 \times 10^{-5}$, the cross-entropy one is of order one (GD: $(w_{n+1}, b_{n+1}) = (w_n, b_n) - \eta \nabla C$).

7.2 Residual MLP

Let $y_\ell = \sigma(A_\ell y_{\ell-1} + b_\ell) + y_{\ell-1}$ for $2 \le \ell \le L - 1$, $y_1 = \sigma(A_1 x + b_1)$, $y_L = A_L y_{L-1} + b_L$ (scalar). Put $z_\ell = A_\ell y_{\ell-1} + b_\ell$, $D_\ell = \operatorname{diag}(\sigma'(z_\ell))$ and $g_\ell = \partial y_L / \partial y_\ell \in \mathbb{R}^{1 \times m}$.

Theorem 5.12: residual Jacobians and gradients

$$\frac{\partial y_\ell}{\partial y_{\ell-1}} = I + D_\ell A_\ell \ (2 \le \ell \le L - 1), \qquad \frac{\partial y_L}{\partial y_{L-1}} = A_L, \qquad g_\ell = A_L \prod_{j=L-1}^{\ell+1} (I + D_j A_j),$$

$$\frac{\partial y_L}{\partial b_\ell} = g_\ell D_\ell, \qquad \frac{\partial y_L}{\partial A_\ell} = D_\ell g_\ell^\top y_{\ell-1}^\top \quad \left(\tfrac{\partial y_L}{\partial (A_\ell)_{ij}} = (g_\ell)_i \sigma'((z_\ell)_i)(y_{\ell-1})_j\right),$$

and $\partial y_L / \partial b_L = 1$, $\partial y_L / \partial A_L = y_{L-1}^\top$.

Proof. R1 splits y_ℓ into $\sigma(z_\ell)$ and $y_{\ell-1}$; R2 gives I; R3 with R5, R4 gives $D_\ell A_\ell$. The residual term is free of b_ℓ and $\partial z_\ell/\partial b_\ell = I$, so $\partial y_\ell/\partial b_\ell = D_\ell$; chain with g_ℓ. Row i of A_ℓ acts only through $(z_\ell)_i$, giving the componentwise formula, an outer product. ■

If $A_j = 0$ or $\sigma'(z_j) = 0$, the factor is I, not 0: the identity path carries the gradient, whereas without the residual all lower gradients vanish. $\prod(I + D_j A_j) = I + \sum D_j A_j + \cdots$ shows the skip path (Chapter 7).

8 Selected exercises

Motivation. XOR needs a hidden layer; finite differences check a backward pass.

Exercise 5.1 (XOR needs a hidden layer)

Let $A = \{(0,0),(1,1)\}$ and $B = \{(0,1),(1,0)\}$.
(a) Show that no affine $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$ satisfies $f > 0$ on B and $f < 0$ on A, nor the reverse.
(b) Show that $g(\mathbf{x}) = \mathrm{ReLU}(x_1 + x_2) - 2\,\mathrm{ReLU}(x_1 + x_2 - 1)$ equals 0 on A and 1 on B.
(c) Show that $\{\mathbf{x} \in \mathbb{R}^2 : g(\mathbf{x}) > \frac{1}{2}\}$ is the open strip $\{\frac{1}{2} < x_1 + x_2 < \frac{3}{2}\}$, which is not a half-plane.

Solution. [Supplement]

(a) B: $w_2 + b > 0$, $w_1 + b > 0$, so $w_1 + w_2 + 2b > 0$. A: $b < 0$, $w_1 + w_2 + b < 0$, so $w_1 + w_2 + 2b < 0$: contradiction. For the reverse apply this to $-f$.

(b) $g = h(t)$, $t = x_1 + x_2$, $h(t) = \mathrm{ReLU}(t) - 2\,\mathrm{ReLU}(t-1)$: on A, $h(0) = 0$, $h(2) = 0$; on B, $h(1) = 1$.

(c) $h(t) = 0$ $(t \le 0)$, t $(0 \le t \le 1)$, $2 - t$ $(t \ge 1)$, so $h > \frac{1}{2}$ iff $\frac{1}{2} < t < \frac{3}{2}$. The complement of a half-plane is convex, but the strip's complement contains $(0,0), (1,1)$ and not $(\frac{1}{2}, \frac{1}{2})$. ■

Exercise 5.2 (Checking a backward pass by finite differences)

Let $\varphi : \mathbb{R} \to \mathbb{R}$ be three times continuously differentiable on $[x - h, x + h]$, $h > 0$, and $M_k = \max_{[x-h,x+h]} |\varphi^{(k)}|$.
(a) Show that $\left|\frac{\varphi(x+h) - \varphi(x)}{h} - \varphi'(x)\right| \le \frac{h}{2} M_2$.
(b) Show that $\frac{\varphi(x+h) - \varphi(x-h)}{2h} - \varphi'(x) = \frac{h^2}{6}\varphi'''(\xi)$ for some $\xi \in [x - h, x + h]$, so the error is at most $\frac{h^2}{6} M_3$.
(c) For $\varphi = \exp$ at $x = 0$ and $h = 10^{-3}$, compute both errors to leading order. Explain how this checks $\partial L/\partial \theta_j$ computed by backpropagation.

Solution. [Supplement]

(a) Taylor: $\varphi(x + h) = \varphi(x) + h\varphi'(x) + \frac{h^2}{2}\varphi''(\xi)$, $\xi \in (x, x + h)$, so the error is $\frac{h}{2}\varphi''(\xi)$.

(b) *Step 1 (two expansions).*

$$\varphi(x \pm h) = \varphi(x) \pm h\varphi'(x) + \frac{h^2}{2}\varphi''(x) \pm \frac{h^3}{6}\varphi'''(\xi_\pm), \qquad \xi_+ \in (x, x + h),\ \xi_- \in (x - h, x).$$

Step 2 (subtract). $\varphi(x + h) - \varphi(x - h) = 2h\varphi'(x) + \frac{h^3}{6}(\varphi'''(\xi_+) + \varphi'''(\xi_-))$, so the error is $\frac{h^2}{6} \cdot \frac{1}{2}(\varphi'''(\xi_+) + \varphi'''(\xi_-))$.

Step 3 (one point). By the intermediate value theorem the average equals $\varphi'''(\xi)$ for some ξ between ξ_- and ξ_+.

(c) $\varphi'(0) = 1$. Forward: $(e^h - 1)/h = 1 + \frac{h}{2} + \frac{h^2}{6} + \cdots$, error $\approx 5.0 \times 10^{-4}$; central: $\sinh(h)/h = 1+\frac{h^2}{6}+\cdots$, error $\approx 1.7\times10^{-7}$. With $\varphi(t) = L(\boldsymbol{\theta}+t\mathbf{e}_j)$, two forward passes at $\boldsymbol{\theta}\pm h\mathbf{e}_j$ estimate $\partial L/\partial\theta_j$ to $O(h^2)$, compared in relative error with backpropagation; too small h lets rounding $\approx \epsilon_{\text{mach}}/h$ dominate. ■

Chapter 6. Training and Optimization of Deep Networks

Mini-batch SGD, activations, weight initialization, learning rates, batch normalization; Taylor expansion and the descent lemma, SGD, momentum, Nesterov, AdaGrad, RMSProp, Adam

1 Full, Stochastic and Mini-Batch Gradients

Motivation. Random mini-batch gradients are cheap and unbiased.

For $f(\boldsymbol{\theta}) = \frac{1}{N}\sum_{i=1}^{N} f_i(\boldsymbol{\theta})$ and uniform I, $\mathbf{g} = \nabla f_I$ is **unbiased**: $\mathbb{E}[\mathbf{g} \mid \boldsymbol{\theta}] = \nabla f$. A mini-batch averages b of them.

Theorem 6.1: mini-batch variance

If the b indices are drawn independently with replacement and a single draw has conditional covariance Σ, the mini-batch gradient has covariance Σ/b. Without replacement it is $\frac{N-b}{b(N-1)}\Sigma$ (with Σ normalized by N).

Proof. With replacement: $\frac{1}{b^2} \cdot b\Sigma$. Without: pairwise covariances $-\Sigma/(N-1)$ give the finite-population correction. ■

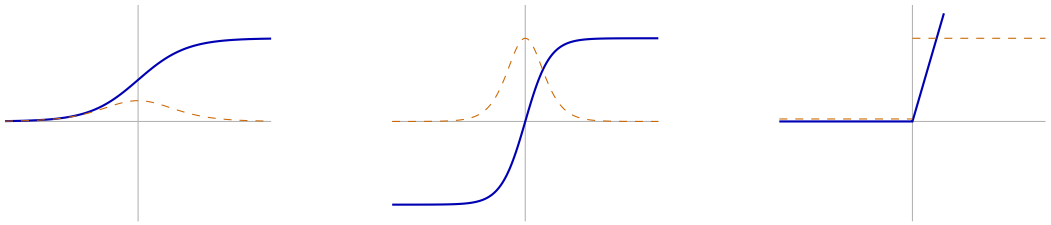
Example 6.2: unbiased but noisy

$f_i(w) = \frac{1}{2}(w - y_i)^2$ with $\mathbf{y} = (1, 3, 5)$ at $w = 0$: per-sample gradients $-1, -3, -5$, full -3; two-sample batches (no replacement) give $-2, -3, -4$, mean -3.

Iteration: one update; *epoch*: one pass (1000 samples, batch 100: 10 iterations).

2 Activation Functions Revisited

Sigmoid. Saturates ($\sigma' \le 1/4, \to 0$). Positive outputs $\mathbf{z}$ make $\partial L/\partial W_{ij} = \delta_i z_j$ share the sign of δ_i: all weights into a neuron move together (zig-zag). **Tanh.** Zero-centered, $\tanh' \in (0, 1]$, still saturates. **ReLU.** Cheap, slope 1 for $z > 0$; negative inputs get zero gradient (dead ReLU), outputs uncentered. Leaky ReLU, ELU, maxout keep a nonzero slope.



Activations (solid) and derivatives (dashed).

3 Weight Initialization

3.1 Why the scale matters

`W = s * np.random.randn(Din, Dout)`: iid $N(0, s^2)$. For $z_j = \sum_{i=1}^{D_{\text{in}}} W_{ji} x_i$, zero-mean independent weights of variance σ_W^2, inputs independent with $\mathbb{E}x_i^2 = q$,

$$\mathbb{E}z_j = 0, \qquad \operatorname{Var} z_j = D_{\text{in}} \sigma_W^2 q$$

($\mathbb{E}[W_{ji}W_{jk}] = 0$, $i \neq k$). Six tanh layers of width 4096:

- $s = 0.01$: $D_{\text{in}} s^2 = 0.41$; activations shrink to 0, weight gradients become tiny.
- $s = 0.05$: $D_{\text{in}} s^2 = 10.24$; pre-activation sd ≈ 3, tanh saturates at ± 1, and $\partial L/\partial \mathbf{z} = \sigma'(\cdot) W^\top \partial L/\partial \mathbf{h} \approx 0$: no learning at large loss (not optimality).

3.2 Zero and symmetric initialization

Theorem 6.3: zero initialization freezes a tanh MLP

For $f(\mathbf{x}) = A_L y_{L-1} + b_L$, $y_\ell = \tanh(A_\ell y_{\ell-1} + b_\ell)$, all parameters initialized to 0 and GD on $\frac{1}{2}\sum_i (f(X_i) - Y_i)^2$: $A_1, \ldots, A_L, b_1, \ldots, b_{L-1}$ stay 0 forever, and $b_L \to \bar{Y}$ if $0 < \eta < 2/N$.

Proof. Induct on "all parameters except b_L are zero". Forward: $\tanh(0) = 0$, hidden outputs 0, $f = b_L$. Backward: $\partial/\partial A_L = \sum_i \delta_L^{(i)} y_{L-1}^\top = 0$; $\boldsymbol{\delta}_{L-1} = (A_L^\top \delta_L) \odot \tanh'(0) = 0$, inductively all $\boldsymbol{\delta}_\ell = 0$, so $\partial A_\ell = \boldsymbol{\delta}_\ell y_{\ell-1}^\top = 0$, $\partial b_\ell = 0$. For b_L: $J = \frac{1}{2}\sum (b_L - Y_i)^2$, $b^{(t+1)} - \bar{Y} = (1 - \eta N)(b^{(t)} - \bar{Y})$. ■

Theorem 6.4: duplicate neurons stay duplicates

For $f(x) = \sum_{j=1}^p u_j \sigma(a_j x + b_j)$ trained by Adam (or GD, momentum, RMSProp), if $(a_{p-1}, b_{p-1}, u_{p-1}) = (a_p, b_p, u_p)$ at initialization, they remain equal for all iterations.

Proof. $\partial f/\partial u_j = \sigma(a_j x + b_j)$, $\partial f/\partial a_j = u_j \sigma'(a_j x + b_j) x$, $\partial f/\partial b_j = u_j \sigma'(a_j x + b_j)$; the shared factor $\partial \ell/\partial f$ multiplies these, so equal parameters get equal gradients. Induct on "paired coordinates of $\boldsymbol{\theta}^k$, m_1^k, m_2^k agree" (true at $k = 0$, $m_1^0 = m_2^0 = 0$): every Adam operation is coordinatewise. (The optimizer state must be in the hypothesis.) ■

Random initialization is needed to break symmetry.

3.3 Xavier and He initialization

Near-linear activations preserve the second moment iff $D_{\text{in}} \sigma_W^2 = 1$, $\sigma_W = 1/\sqrt{D_{\text{in}}}$ ("Xavier"; Glorot's $2/(D_{\text{in}} + D_{\text{out}})$ also balances the backward pass).

Theorem 6.5: He initialization

Let $y^+ = A\mathbf{x}$, $\mathbf{x} = \text{ReLU}(\mathbf{y})$, $A_{ij} \overset{\text{iid}}{\sim} N(0, \sigma_A^2)$, where $\mathbf{y}$ is independent of A, its coordinates have mean 0, variance 1, are uncorrelated, and $\mathbf{y} \overset{d}{=} -\mathbf{y}$. Then

(i) $\mathbb{E}x_j^2 = \frac{1}{2}$; (ii) $\mathbb{E}y^+ = 0$; (iii) $\mathbb{E}(y_i^+)^2 = \frac{n_{\text{in}} \sigma_A^2}{2}$;

(iv) $y_1^+, \ldots, y_{n_{\text{out}}}^+$ are uncorrelated; (v) $y^+ \overset{d}{=} -y^+$.

Thus $\sigma_A^2 = 2/n_{\text{in}}$ preserves the second moment through ReLU layers.

Proof. (i) $\text{ReLU}(y)^2 = y^2\mathbf{1}\{y > 0\}$; by symmetry this is half of $\mathbb{E}y^2 = 1$.

(ii) Independence and $\mathbb{E}A_{ij} = 0$.

(iii) $\mathbb{E}(y_i^+)^2 = \sum_{j,k} \mathbb{E}[A_{ij}A_{ik}]\mathbb{E}[x_j x_k] = \sum_j \sigma_A^2 \mathbb{E}x_j^2$.

(iv) For $i \neq k$ all $\mathbb{E}[A_{ij}A_{kl}] = 0$.

(v) $(A, \mathbf{y}) \overset{d}{=} (-A, \mathbf{y})$ since A is independent of $\mathbf{y}$ and symmetric; apply $F(A, \mathbf{y}) = A\,\text{ReLU}(\mathbf{y})$. ■

Remark 6.6: refinements

Leaky ReLU with slope α: $\mathbb{E}\phi(y)^2 = \frac{1+\alpha^2}{2}$ and $\sigma_A^2 = \frac{2}{(1+\alpha^2)n_{\text{in}}}$. For $z \sim N(0, v)$, $\mathbb{E}\,\text{ReLU}(z) = \sqrt{v/(2\pi)}$, $\mathbb{E}\,\text{ReLU}(z)^2 = v/2$: He preserves the second moment, not the variance. Backward, $A^\top \partial\ell/\partial\mathbf{y}^+$ has variance $n_{\text{out}}\sigma_A^2$ (n_{out} instead of n_{in}).

Theorem 6.7: sigmoid layer

For $\mathbf{y} = \sigma(A\mathbf{x})$ with $\sigma(t) \approx \frac{1}{2} + \frac{t}{4}$, $\sigma' \approx \frac{1}{4}$, $A_{ij} \overset{\text{iid}}{\sim} N(0, \sigma_A^2)$ independent of $\mathbf{x}$, and x_j uncorrelated with mean $\frac{1}{2}$ and variance 1: $\mathbb{E}y_i = \frac{1}{2}$, $\text{Var}\, y_i = \frac{5n_{\text{in}}\sigma_A^2}{64}$. If the upstream gradients $\partial\ell/\partial y_i$ have mean 0, variance 1, are uncorrelated and independent of A, then $\partial\ell/\partial x_j \approx \frac{1}{4}\sum_i A_{ij}\partial\ell/\partial y_i$ has mean 0 and variance $\frac{n_{\text{out}}\sigma_A^2}{16}$.

Proof. $\text{Var}(A_{ij}x_j) = \sigma_A^2(1 + \frac{1}{4})$, uncorrelated terms, times $\frac{1}{16}$. Backward: the same with $\mathbb{E}(\partial\ell/\partial y_i)^2 = 1$. ■

Theorem 6.8: convolution layer

For a 3×3, padding-1 convolution $Y_{cij} = \sum_{p,q,r} w_{cpqr}\tilde{X}_{p,i+q-2,j+r-2} + b_c$ with zero-mean unit-variance uncorrelated inputs, $w \overset{\text{iid}}{\sim} N(0, \sigma_w^2)$, $b \overset{\text{iid}}{\sim} N(0, \sigma_b^2)$ independent: $\mathbb{E}Y_{cij} = 0$ everywhere, and $\mathbb{E}Y_{cij}^2 = \sigma_w^2 \cdot \#\{\text{in-range inputs}\} + \sigma_b^2$, i.e. $9C_{\text{in}}\sigma_w^2 + \sigma_b^2$ in the interior, $6C_{\text{in}}\sigma_w^2 + \sigma_b^2$ on edges and $4C_{\text{in}}\sigma_w^2 + \sigma_b^2$ at corners.

Proof. Expanding the square, only diagonal weight terms $\sigma_w^2\mathbb{E}\tilde{X}_a^2$ and $\mathbb{E}b_c^2 = \sigma_b^2$ survive; padded entries are zero. ■

3.4 Variance of a sigmoid output

Taylor $f(X) = f(\mu) + f'(\mu)(X-\mu) + \frac{1}{2}f''(\xi)(X-\mu)^2$ gives $\text{Var}\, f(X) = f'(\mu)^2\,\text{Var}\, X + \text{Error}$ (terms with f''). For $X \sim N(0,1)$, $Y = \sigma(wX + b)$: $\text{Var}\, Y \approx w^2\sigma'(b)^2$, small for small w or saturated b. Exactly, L-Lipschitz f gives $\text{Var}\, f(X) \leq \mathbb{E}(f(X) - f(\mu))^2 \leq L^2\,\text{Var}\, X$, so $\text{Var}\, Y \leq w^2/16$.

4 Learning-Rate Schedules

Too large diverges, too small is slow; decaying the rate refines the solution and reduces noise.

- Step: multiply by 0.1 at fixed epochs (ResNet: 30, 60, 90), giving $\eta_0, 0.1\eta_0, 0.01\eta_0, 0.001\eta_0$.
- Cosine: $\eta_t = \frac{\eta_0}{2}[1 + \cos(\pi t/T)]$; linear: $\eta_t = \eta_0(1 - t/T)$; inverse square root: $\eta_t = \eta_0/\sqrt{t}$.

- Linear warmup: increase from 0 to $\eta_{\max}$ over the first iterations (e.g. 5000), then decay.

5 Batch Normalization

5.1 Training-time computation

For a mini-batch $X \in \mathbb{R}^{B\times D}$ (rows: samples, columns: features),

$$\mu_j = \frac{1}{B}\sum_i X_{ij}, \quad v_j = \frac{1}{B}\sum_i (X_{ij}-\mu_j)^2, \quad \hat{X}_{ij} = \frac{X_{ij}-\mu_j}{\sqrt{v_j+\epsilon}}, \quad Y_{ij} = \gamma_j \hat{X}_{ij} + \beta_j.$$

Learned γ, β set scale and center ($\gamma_j = \sqrt{v_j+\epsilon}$, $\beta_j = \mu_j$ recovers the input).

Example 6.9: two features

$X = \begin{pmatrix} 1 & 10 \\ 3 & 14 \end{pmatrix}$: $\mu = (2, 12)$, $v = (1, 4)$, $\hat{X} = \begin{pmatrix} -1 & -1 \\ 1 & 1 \end{pmatrix}$ (ϵ ignored). With $\gamma = (2, 3)$, $\beta = (-1, 5)$, $Y = \begin{pmatrix} -3 & 2 \\ 1 & 8 \end{pmatrix}$.

Theorem 6.10: invariances

With $\epsilon = 0$:
(i) each column of $\hat{X}$ has mean 0 and mean square 1;
(ii) for $z = \mathbf{w}^\top \mathbf{x} + b$ the BN output does not depend on b and is invariant under $\mathbf{w} \mapsto c\mathbf{w}$, $c > 0$;
(iii) consequently $\langle \nabla_{\mathbf{w}} L, \mathbf{w} \rangle = 0$.

Proof. (ii) $z - \mu = \mathbf{w}^\top(\mathbf{x} - \bar{\mathbf{x}})$ removes b; scaling $\mathbf{w}$ scales both $z - \mu$ and $\sqrt{v}$ by c.
(iii) $h(c) = L(c\mathbf{w})$ is constant, so $0 = h'(1) = \nabla L(\mathbf{w})^\top \mathbf{w}$. ■

5.2 Test time

At inference the output must not depend on other samples, so training keeps running averages

$$\mu^{\text{run}} \leftarrow m\,\mu^{\text{run}} + (1-m)\mu_{\text{batch}}, \qquad (\sigma^2)^{\text{run}} \leftarrow m\,(\sigma^2)^{\text{run}} + (1-m)\sigma^2_{\text{batch}}$$

(EMA; PyTorch's `momentum` is $1-m$), used at test time:

$$\hat{x}_{ij} = \frac{x_{ij} - \mu_j^{\text{run}}}{\sqrt{(\sigma_j^2)^{\text{run}} + \epsilon}}, \qquad y_{ij} = \gamma_j \hat{x}_{ij} + \beta_j.$$

(Batch statistics on one sample give $\hat{x} = 0$.) Code: `model.train()` vs. `model.eval()`.

Theorem 6.11: fusion at inference

At test time BN is the affine map $z \mapsto az + c$ with $a_j = \gamma_j / \sqrt{v_j^{\text{run}} + \epsilon}$, $c_j = \beta_j - a_j \mu_j^{\text{run}}$, so $\text{BN}(W\mathbf{x} + \mathbf{b}) = \text{diag}(\mathbf{a})W\mathbf{x} + (\text{diag}(\mathbf{a})\mathbf{b} + \mathbf{c})$ can be folded into the preceding layer.

Remark 6.12: why BN helps

Easier optimization, larger learning rates, less initialization sensitivity, mild regularization; "internal covariate shift" is disputed (smoother landscape is an alternative). Training-mode BN couples samples backward: with $g_i = \partial L/\partial y_i$ and $r = \sqrt{v+\epsilon}$, $\partial L/\partial x_i = \frac{\gamma}{r}(g_i - \bar{g} - \hat{x}_i\,\overline{g\hat{x}})$.

6 Gradient Descent and Taylor's Theorem

Motivation. Taylor's theorem says a small enough step against the gradient decreases the loss.

GD on $f:\mathbb{R}^d\to\mathbb{R}$: $\mathbf{x}_{t+1} = \mathbf{x}_t - \eta\nabla f(\mathbf{x}_t)$; $-\nabla f$ is steepest since $\nabla f^\top \mathbf{v} \geq -\|\nabla f\|$ for unit $\mathbf{v}$.

Theorem 6.13: Taylor along a segment

With $\mathbf{d} = \mathbf{y} - \mathbf{x}$ and $h(t) = f(\mathbf{x}+t\mathbf{d})$,

$$f(\mathbf{y}) - f(\mathbf{x}) = \int_0^1 \nabla f(\mathbf{x}+t\mathbf{d})^\top \mathbf{d}\,dt, \qquad f(\mathbf{y}) = f(\mathbf{x}) + \nabla f(\mathbf{x})^\top \mathbf{d} + \int_0^1 (1-t)\,\mathbf{d}^\top \nabla^2 f(\mathbf{x}+t\mathbf{d})\mathbf{d}\,dt.$$

Proof. $h(1) - h(0) = \int_0^1 h'$; and $h(1) = h(0) + h'(0) + \int_0^1 \int_0^t h''(s)\,ds\,dt$, where swapping the order gives $\int_0^1 (1-s)h''(s)\,ds$. ■

7 Smoothness and the Descent Lemma

Motivation. An L-Lipschitz gradient puts the loss below a quadratic, giving the safe step $1/L$.

Definition 6.14: L-smoothness

f is L-smooth if $\|\nabla f(\mathbf{x}) - \nabla f(\mathbf{y})\| \leq L\|\mathbf{x}-\mathbf{y}\|$ for all $\mathbf{x},\mathbf{y}$. For $f \in C^2$ this is equivalent to $-L\mathbf{I} \preceq \nabla^2 f \preceq L\mathbf{I}$ (all Hessian eigenvalues in $[-L, L]$).

Theorem 6.15: descent lemma

If f is L-smooth, $f(\mathbf{y}) \leq f(\mathbf{x}) + \nabla f(\mathbf{x})^\top(\mathbf{y}-\mathbf{x}) + \frac{L}{2}\|\mathbf{y}-\mathbf{x}\|^2$.

Proof. By the first Taylor formula, Cauchy–Schwarz and smoothness,

$$f(\mathbf{y}) - f(\mathbf{x}) - \nabla f(\mathbf{x})^\top \mathbf{d} = \int_0^1 (\nabla f(\mathbf{x}+t\mathbf{d}) - \nabla f(\mathbf{x}))^\top \mathbf{d}\,dt \leq \int_0^1 Lt\|\mathbf{d}\|^2 dt = \frac{L}{2}\|\mathbf{d}\|^2.$$

■

Theorem 6.16: GD on smooth functions

GD satisfies $f(\mathbf{x}_{t+1}) \leq f(\mathbf{x}_t) - \eta(1 - \frac{L\eta}{2})\|\nabla f(\mathbf{x}_t)\|^2$, so it decreases f when $0 < \eta < 2/L$. With $\eta = 1/L$ and $f^\star = \inf f > -\infty$,

$$\min_{0\leq t<T} \|\nabla f(\mathbf{x}_t)\|^2 \leq \frac{1}{T}\sum_{t<T} \|\nabla f(\mathbf{x}_t)\|^2 \leq \frac{2L\,(f(\mathbf{x}_0) - f^\star)}{T}.$$

Proof. Put $\mathbf{y} = \mathbf{x}_t - \eta\nabla f(\mathbf{x}_t)$ in the descent lemma; at $\eta = 1/L$ the coefficient is $1/(2L)$, and summing telescopes to $f(\mathbf{x}_0) - f(\mathbf{x}_T) \le f(\mathbf{x}_0) - f^\star$. ■

Only stationarity: for nonconvex f a small gradient may be a saddle. For $f = \frac{1}{2}\mathbf{x}^\top H\mathbf{x}$ direction i contracts by $1 - \eta\lambda_i$: stability needs $\eta < 2/\lambda_{\max}$, and small λ_i are slow.

8 Stochastic Gradient Descent

Motivation. Gradient noise sets a floor proportional to the step size.

SGD: $\mathbf{x}_{t+1} = \mathbf{x}_t - \eta\mathbf{g}_t$, $\mathbb{E}_t\mathbf{g}_t = \nabla f(\mathbf{x}_t)$ ($\mathbb{E}_t$ given the history up to $\mathbf{x}_t$).

Theorem 6.17: second moment is not variance

$\mathbb{E}_t\|\mathbf{g}_t\|^2 = \|\nabla f(\mathbf{x}_t)\|^2 + \mathbb{E}_t\|\mathbf{g}_t - \nabla f(\mathbf{x}_t)\|^2$.

Proof. Expand $\mathbf{g}_t = \nabla f + (\mathbf{g}_t - \nabla f)$; the cross term has conditional mean zero. ■

Theorem 6.18: $T^{-1/2}$ rate for SGD

If f is L-smooth, $\mathbb{E}_t\|\mathbf{g}_t\|^2 \le G$ and $\Delta = f(\mathbf{x}_0) - f^\star$, then for a constant step

$$\frac{1}{T}\sum_{t<T}\mathbb{E}\|\nabla f(\mathbf{x}_t)\|^2 \le \frac{\Delta}{\eta T} + \frac{L\eta G}{2}, \qquad \text{and with } \eta = \sqrt{\tfrac{2\Delta}{LGT}}: \quad \le \sqrt{\frac{2L\Delta G}{T}}.$$

Proof. Descent lemma: $f(\mathbf{x}_{t+1}) \le f(\mathbf{x}_t) - \eta\nabla f(\mathbf{x}_t)^\top\mathbf{g}_t + \frac{L\eta^2}{2}\|\mathbf{g}_t\|^2$. So $\mathbb{E}_t f(\mathbf{x}_{t+1}) \le f(\mathbf{x}_t) - \eta\|\nabla f(\mathbf{x}_t)\|^2 + \frac{L\eta^2}{2}G$; take total expectation, telescope over $t < T$, use $\mathbb{E}f(\mathbf{x}_T) \ge f^\star$, divide by ηT. $\frac{a}{\eta} + b\eta$ is minimal at $\eta = \sqrt{a/b}$, value $2\sqrt{ab}$. ■

Noise forces $\eta \sim T^{-1/2}$ (GD: T^{-1}); batch size B divides the noise variance by B.

9 Momentum

Motivation. Averaging past gradients cancels oscillation and accumulates consistent directions.

9.1 Heavy-ball momentum

Definition 6.19: momentum

$\mathbf{v}_{t+1} = \beta\mathbf{v}_t - \eta\nabla f(\mathbf{x}_t)$, $\mathbf{x}_{t+1} = \mathbf{x}_t + \mathbf{v}_{t+1}$, $\mathbf{v}_0 = \mathbf{0}$, $0 \le \beta < 1$.

$\mathbf{v}_{t+1} = -\eta\sum_{k=0}^{t}\beta^{t-k}\nabla f(\mathbf{x}_k)$; for constant $\mathbf{g}$, $\|\mathbf{v}_t\| \to \eta\|\mathbf{g}\|/(1-\beta)$ ($10\times$ at $\beta = 0.9$). Gradient is force, $1-\beta$ friction: with $\mathbf{m}_{t+1} = \mathbf{g}_t + \beta\mathbf{m}_t$, $\mathbf{m}_{t+1} - \mathbf{m}_t = \mathbf{g}_t - (1-\beta)\mathbf{m}_t$.

Theorem 6.20: momentum on a quadratic

For $f(x) = \frac{h}{2}x^2$, $x_{t+1} = (1 + \beta - \eta h)x_t - \beta x_{t-1}$; the iteration converges iff both roots of $r^2 - (1 + \beta - \eta h)r + \beta = 0$ lie in the open unit disk, which for $0 \le \beta < 1$ holds iff $0 < \eta h < 2(1 + \beta)$. Complex roots (modulus $\sqrt{\beta}$) mean damped oscillation.

Proof. Substitute $\mathbf{v}_{t+1} = x_{t+1} - x_t$, $\mathbf{v}_t = x_t - x_{t-1}$. Jury/Schur for $r^2 + a_1 r + a_0$: $|a_0| < 1$, $1 + a_1 + a_0 > 0$, $1 - a_1 + a_0 > 0$, i.e. $\beta < 1$, $\eta h > 0$, $\eta h < 2(1 + \beta)$. ■

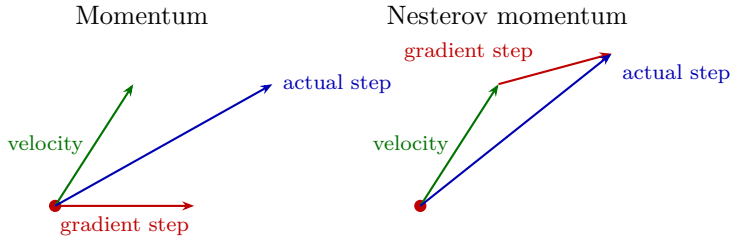
9.2 Nesterov momentum

Gradient at the look-ahead point:

$$\mathbf{y}_t = \mathbf{x}_t + \beta \mathbf{v}_t, \qquad \mathbf{v}_{t+1} = \beta \mathbf{v}_t - \eta \nabla f(\mathbf{y}_t), \qquad \mathbf{x}_{t+1} = \mathbf{x}_t + \mathbf{v}_{t+1}.$$

Example 6.21: a numerical illustration

$f = x^2/2$, $x_0 = 1$, $\eta = 0.1$, $\beta = 0.9$. Both methods give $v_1 = -0.1$, $x_1 = 0.9$. Momentum: $v_2 = 0.9(-0.1) - 0.1(0.9) = -0.18$, $x_2 = 0.72$. Nesterov: $y_1 = 0.81$, $v_2 = -0.09 - 0.081 = -0.171$, $x_2 = 0.729$: it brakes earlier (not always closer to the optimum).



Momentum adds the gradient at the current point; Nesterov takes it after moving along the velocity.

10 Adaptive Methods

Motivation. Per-coordinate steps scaled by a running gradient magnitude.

All operations are coordinatewise.

Definition 6.22: AdaGrad, RMSProp, Adam

AdaGrad: $\mathbf{r}_t = \mathbf{r}_{t-1} + \mathbf{g}_t \odot \mathbf{g}_t$, $\mathbf{x}_{t+1} = \mathbf{x}_t - \eta\, \mathbf{g}_t \oslash (\sqrt{\mathbf{r}_t} + \epsilon)$.
RMSProp: $\mathbf{r}_t = \rho \mathbf{r}_{t-1} + (1 - \rho)\mathbf{g}_t \odot \mathbf{g}_t$, same update.
Adam: $\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1)\mathbf{g}_t$, $\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2)\mathbf{g}_t \odot \mathbf{g}_t$, $\hat{\mathbf{m}}_t = \mathbf{m}_t/(1 - \beta_1^t)$, $\hat{\mathbf{v}}_t = \mathbf{v}_t/(1 - \beta_2^t)$, $\mathbf{x}_t = \mathbf{x}_{t-1} - \eta\, \hat{\mathbf{m}}_t \oslash (\sqrt{\hat{\mathbf{v}}_t} + \epsilon)$. Typical $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\eta = 10^{-3}$.

Remark 6.23: what each piece does

AdaGrad's rate $\eta/\sqrt{r_t}$ shrinks ($\eta/\sqrt{t}$ for constant g). RMSProp forgets: $g = 2$, $\rho = 0.9$, $\eta = 0.1$ give $r_1 = 0.4$, $r_2 = 0.76$, steps $0.316, 0.229 \to 0.1$. Adam = momentum numerator + RMSProp denominator + bias correction (with Nesterov: NAdam).

10.1 Exponential moving averages and bias correction

Theorem 6.24: EMA of iid gradients

If $g_0, g_1, \dots$ are iid with mean μ and variance s^2, and $m_{k+1} = \beta m_k + (1-\beta) g_k$, $m_0 = 0$, then

$$\mathbb{E} m_k = (1-\beta^k)\mu, \qquad \operatorname{Var} m_k = s^2 \frac{1-\beta}{1+\beta}(1-\beta^{2k}),$$

with limits μ and $s^2 \frac{1-\beta}{1+\beta}$. The corrected $\hat{m}_k = m_k/(1-\beta^k)$ is unbiased with variance $s^2 \frac{(1-\beta)(1+\beta^k)}{(1+\beta)(1-\beta^k)}$ (equal to s^2 at $k=1$).

Proof. m_k is independent of g_k. Mean by induction: $\beta(1-\beta^k)\mu + (1-\beta)\mu = (1-\beta^{k+1})\mu$. Variance: $\operatorname{Var} m_{k+1} = \beta^2 \operatorname{Var} m_k + (1-\beta)^2 s^2$; writing $(1-\beta)^2 = \frac{1-\beta}{1+\beta}(1-\beta^2)$, the induction step is $\beta^2(1-\beta^{2k}) + (1-\beta^2) = 1-\beta^{2k+2}$. Correction: divide by $(1-\beta^k)^2$ and use $1-\beta^{2k} = (1-\beta^k)(1+\beta^k)$. ■

Uncorrected, Adam's first step is $\frac{1-\beta_1}{\sqrt{1-\beta_2}} \operatorname{sign}(g_1)\eta \approx 3.16\,\eta$; corrected, $\hat{m}_1 = g_1$, $\hat{v}_1 = g_1^2$, step $\eta \operatorname{sign}(g_1)$: no bias, larger early variance.

10.2 Special cases and invariances

Theorem 6.25: Adam contains RMSProp and signSGD

For Adam without bias correction: $\beta_1 = 0$ gives RMSProp with $\rho = \beta_2$; $\beta_1 = \beta_2 = 0$, $\epsilon = 0$ give signSGD $\theta_{k+1} = \theta_k - \eta \operatorname{sign}(g_k)$ (if no coordinate of g_k is 0). With $\epsilon = 0$, Adam's iterates are unchanged when the loss is multiplied by $c > 0$, while SGD's step scales by c.

Proof. $\beta_1 = 0$ makes $m_{k+1} = g_k$. With $\beta_2 = 0$ also, $\sqrt{v_{k+1}} = |g_k|$. Scaling $g \mapsto cg$ scales m by c and $\sqrt{v}$ by c. ■

Chapter 7. Convolutional Neural Networks and Architectures

Convolution, stride, padding, pooling, equivariance; AlexNet, LRN, ZFNet, VGG, GoogLeNet/Inception, ResNet, saliency maps

1 Why a New Layer

A fully connected layer on flattened $X \in \mathbb{R}^{H\times W\times C}$ has $M(HWC+1)$ parameters (3,073,000 for $32\times 32\times 3$, $M=1000$) and ignores neighborhoods. Convolution assumes:

- **Locality**: edges and textures live in small patches (local connectivity).
- **Stationarity**: a pattern may occur anywhere (weight sharing).

Definition 7.1: equivariance and invariance

Let $T_{\boldsymbol{\tau}}$ shift an image, $(T_{\boldsymbol{\tau}}X)_{i,j,c} = X_{i-\tau_1,j-\tau_2,c}$. F is shift *equivariant* if $F(T_{\boldsymbol{\tau}}X) = T_{\boldsymbol{\tau}}F(X)$ and G is shift *invariant* if $G(T_{\boldsymbol{\tau}}X) = G(X)$. Generally, for a group acting by $\rho_{\text{in}}(g)$, $\rho_{\text{out}}(g)$, f is equivariant if $f(\rho_{\text{in}}(g)x) = \rho_{\text{out}}(g)f(x)$; invariance is the case $\rho_{\text{out}} = \text{id}$.

2 Convolution as a Local Dot Product

2.1 A fixed filter: moving average

With the 3×3 averaging kernel (1/9 everywhere), $Y_{i,j} = \frac{1}{9}\sum_{u,v=0}^{2} X_{i+u,j+v}$: on a 0/90 image a window with r bright pixels gives $10r$; isolated pixels are suppressed, edges blurred. A CNN *learns* the kernel (any signs, any sum).

2.2 Single and multiple channels

Definition 7.2: convolutional layer

With D filters $W \in \mathbb{R}^{D\times K\times K\times C}$, biases $\mathbf{b}\in\mathbb{R}^D$, stride S, padding P,

$$Z_{i,j,d} = b_d + \sum_{u=0}^{K-1}\sum_{v=0}^{K-1}\sum_{c=1}^{C} W_{d,u,v,c}\,\tilde{X}_{Si+u,\,Sj+v,\,c},$$

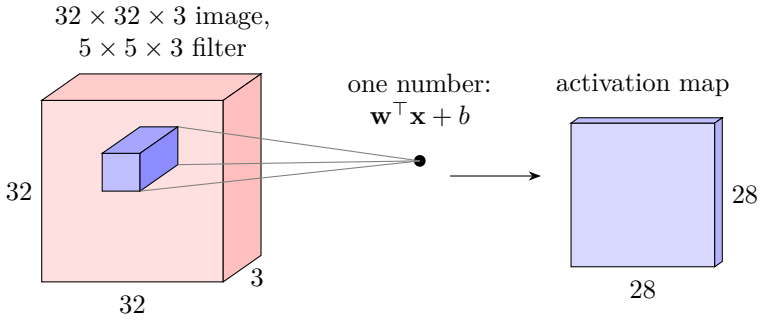
$\tilde{X}$ padded. Each output is a patch–filter dot product over *all* input channels plus bias; $Z_{\cdot,\cdot,d}$ is the feature map of filter d. (Cross-correlation, no flip; irrelevant for learned kernels.)

Example 7.3: one channel, 2×2 filter

$X = \begin{pmatrix} 1 & 2 & 0\\ 0 & 1 & 3\\ 2 & 1 & 0\end{pmatrix}$, $W = \begin{pmatrix} 1 & 0\\ -1 & 2\end{pmatrix}$, $b=1$, $S=1$, $P=0$: $Z = \begin{pmatrix} 4 & 8\\ 1 & 1\end{pmatrix}$, same weights at all four positions.

Example 7.4: a 5×5 example

With a 5×5 image and $W = \begin{pmatrix} 1 & 2 & 3 \\ -4 & 7 & 4 \\ 2 & -5 & 1 \end{pmatrix}$ ($b = 0$): unpadded $Z = \begin{pmatrix} 51 & 66 & 20 \\ 31 & 49 & 101 \\ 15 & 53 & -2 \end{pmatrix}$; with $P = 1$ the 5×5 output has this as its central block.



A full-depth filter slid over all positions gives one 28×28 map.

2.3 Parameter counts

Six $5 \times 5 \times 3$ filters: $32 \times 32 \times 3 \to 28 \times 28 \times 6$ with $6(75 + 1) = 456$ parameters. Same shapes:

Layer	Parameters
Fully connected	$4704(3072 + 1) = 14{,}455{,}392$
Locally connected, unshared	$28^2 \cdot 6(75 + 1) = 357{,}504$
Convolution (shared)	$6(75 + 1) = 456$

As a matrix, convolution is sparse and banded with repeated entries (filter (a, b) on $(x_1, \ldots, x_4)$: two free numbers).

3 Stride, Padding and Output Size

Motivation. One formula gives every layer's output shape.

Theorem 7.5: output size

For input length N, kernel K, stride S, padding P (symmetric), the number of output positions is

$$N' = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1, \qquad C' = D.$$

Proof. A window starting at tS needs $tS + K - 1 \le N + 2P - 1$, i.e. $t = 0, \ldots, \lfloor (N + 2P - K)/S \rfloor$. ■

$N = 7$, $K = 3$: $S = 1, 2, 3 \Rightarrow 5, 3, 2$ (for $S = 3$ a border is unused). $S = 1$, odd K: $P = (K - 1)/2$ preserves size. Zero padding is a boundary assumption.

Example 7.6: further examples

Input $52 \times 52 \times 128$, $D = 256$.
(1) $K = 3, S = 1, P = 1$: output $52 \times 52 \times 256$, parameters $256(9 \cdot 128 + 1) = 295{,}168$; each filter is $3 \times 3 \times 128$.
(2) $K = 4, S = 2, P = 1$: $\lfloor (52 + 2 - 4)/2 \rfloor + 1 = 26$, output $26 \times 26 \times 256$, parameters $256(16 \cdot 128 + 1) = 524{,}544$.

Stacking $32 \times 32 \times 3 \to 28 \times 28 \times 6 \to 24 \times 24 \times 10$ (ten $5 \times 5 \times 6$ filters, 1510 parameters); two stride-1 5×5 layers see 9×9. With strides, $r_\ell = r_{\ell-1} + (K_\ell - 1) j_{\ell-1}$ and $j_\ell = S_\ell j_{\ell-1}$, starting from $r_0 = j_0 = 1$.

4 Learning Filters

Unpadded, stride 1, $G_{i,j,d} = \partial L / \partial Z_{i,j,d}$:

$$\frac{\partial L}{\partial W_{d,u,v,c}} = \sum_{i,j} G_{i,j,d} X_{i+u,j+v,c}, \qquad \frac{\partial L}{\partial b_d} = \sum_{i,j} G_{i,j,d} :$$

every position updates the shared weight.

5 Shift Equivariance

Motivation. Convolution shifts features with the image.

Theorem 7.7: convolution commutes with shifts

On an infinite grid (or with circular indices), stride-1 convolution satisfies $C_W(T_{\boldsymbol{\tau}} X) = T_{\boldsymbol{\tau}} C_W(X)$, and pointwise nonlinearities also commute with shifts. Global average pooling of a circularly shifted map is unchanged, so GAP $\circ$ (convolutions, ReLUs) is shift invariant.

Proof. $[C_W(T_{\boldsymbol{\tau}} X)]_{i,j,d} = b_d + \sum_{u,v,c} W_{d,u,v,c} X_{i+u-\tau_1, j+v-\tau_2, c} = [C_W X]_{i-\tau_1, j-\tau_2, d}$. A circular shift permutes positions; averages are permutation invariant. ■

Remark 7.8: when exactness fails

Zero padding breaks equality near the border. Stride S (or pooling) commutes only with shifts by multiples of S: for width-2, stride-2 max pooling M, $M(T_2 x) = T_1 M(x)$ but $M(T_1 x)$ need not be a shift of $M(x)$ (e.g. $x = (0, 1, 2, 0)$, $M(x) = (1, 2)$, $M(T_1 x) = (0, 2)$).

6 1×1 Convolution, Pooling and Global Average Pooling

1×1 convolution. $\mathbf{y}_{i,j} = A\mathbf{x}_{i,j} + \mathbf{b}$, $A \in \mathbb{R}^{D \times C}$ shared: mixes channels, not neighbors. $56 \times 56 \times 64 \to 56 \times 56 \times 32$ costs $32(64 + 1) = 2080$ parameters.

Pooling. Channelwise, no parameters: $Y^{\max}_{i,j,c} = \max_{u,v} X_{iS+u, jS+v, c}$ or the average. For $X = \begin{pmatrix} 5 & 10 & 6 & 7 \\ 4 & 0 & 4 & 3 \\ 2 & 4 & 3 & 4 \\ 0 & 8 & 1 & 6 \end{pmatrix}$ and 2×2, stride 2: average $\begin{pmatrix} 4.75 & 5 \\ 3.5 & 3.5 \end{pmatrix}$, max $\begin{pmatrix} 10 & 7 \\ 8 & 6 \end{pmatrix}$. Pooling lowers resolution,

with limited robustness to small shifts. Backward: max routes to the maximizer, average spreads g/K^2.

Global average pooling. $g_c = \frac{1}{HW}\sum_{i,j} X_{i,j,c}$, $H \times W \times C \to C$: the classifier needs $Q(C+1)$, not $Q(HWC+1)$, parameters; spatial arrangement is discarded.

7 Algebraic Facts about CNN Layers

Motivation. Identities saying when CNN layers can be reordered or merged.

Theorem 7.9: positive homogeneity

A network built only from bias-free convolutions and linear layers, ReLU, max pooling, reshaping and dropout satisfies $f(\lambda X) = \lambda f(X)$ for all $\lambda \geq 0$.

Proof. Each piece is positively homogeneous: linear maps; $\max(0, \lambda t) = \lambda \max(0, t)$, $\max_i \lambda x_i = \lambda \max_i x_i$ ($\lambda \geq 0$ preserves order and signs); dropout is a fixed mask. Compositions: $g_1(g_2(\lambda X)) = g_1(\lambda g_2(X)) = \lambda g_1(g_2(X))$. Biases break it (ReLU$(x+1)$ at $x=0$). ■

Theorem 7.10: conv–MP–ReLU orders coincide

conv→MaxPool→ReLU, conv→ReLU→MaxPool and conv→ReLU→MaxPool→ReLU are the same function.

Proof. Nondecreasing ϕ gives $\max_i \phi(x_i) = \phi(\max_i x_i)$ (equality at the maximizer). ReLU is nondecreasing and elementwise, so ReLU $\circ$MP = MP $\circ$ ReLU, and ReLU $\circ$ ReLU = ReLU. (Average pooling: only avg ReLU $\geq$ ReLU avg, by convexity.) ■

Theorem 7.11: convolutions that implement given maps

With PyTorch's convention $Y_{o,i,j} = b_o + \sum_c \sum_{k,l=1}^{3} w_{o,c,k,l} \tilde{X}_{c,i+k-2,j+l-2}$ (3×3, padding 1), reading the value at offset $(\Delta i, \Delta j)$ requires a 1 at $(k,l) = (2+\Delta i, 2+\Delta j)$.

- Identity $C \to C$: $w_{c,c',k,l} = \delta_{cc'}\delta_{k2}\delta_{l2}$, $\mathbf{b} = \mathbf{0}$.
- Colorwise finite differences $Y_{2c-1} = X_{c,i+1,j} - X_{c,i,j}$, $Y_{2c} = X_{c,i,j+1} - X_{c,i,j}$: $w_{2c-1,c} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 1 & 0 \end{pmatrix}$, $w_{2c,c} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & -1 & 1 \\ 0 & 0 & 0 \end{pmatrix}$, all other slices zero; zero padding supplies the out-of-range zeros.

8 Vocabulary and Costs

$X \mapsto h(X) \mapsto \mathbf{s}(X) \in \mathbb{R}^Q \mapsto \mathbf{p}(X)$. A convolution with D filters $K \times K \times C$, output $H' \times W' \times D$:

$$N_{\text{param}} = D(K^2C+1), \qquad N_{\text{act}} = H'W'D, \qquad N_{\text{MAC}} \approx H'W'DK^2C.$$

FC $\mathbb{R}^n \to \mathbb{R}^m$: $m(n+1)$. Top-1 error: top score wrong; top-5: true label not among the top five.

9 AlexNet

Five convolutional and three FC layers (eight weight layers):

Stage	Settings	Output
Input	RGB	$227 \times 227 \times 3$
Conv1	96, 11×11, $S = 4$, $P = 0$	$55 \times 55 \times 96$ $((227 - 11)/4 + 1)$
Pool1, Norm1	3×3, $S = 2$; LRN	$27 \times 27 \times 96$
Conv2	256, 5×5, $P = 2$	$27 \times 27 \times 256$
Pool2, Norm2	3×3, $S = 2$; LRN	$13 \times 13 \times 256$
Conv3, Conv4	384, 3×3, $P = 1$	$13 \times 13 \times 384$
Conv5	256, 3×3, $P = 1$	$13 \times 13 \times 256$
Pool3	3×3, $S = 2$	$6 \times 6 \times 256 = 9216$
FC6, FC7, FC8	$9216 \to 4096 \to 4096 \to 1000$	1000 scores

The FC layers dominate: $4096 \cdot 9217 = 37{,}752{,}832$, $4096 \cdot 4097 = 16{,}781{,}312$, $1000 \cdot 4097 = 4{,}097{,}000$, together 58,631,144, versus $96(11^2 \cdot 3 + 1) = 34{,}944$ for Conv1. Design:

- **ReLU**: no positive-side saturation.
- **Large first stride**: $227 \to 55$ saves memory and compute.
- **Local response normalization**: $a_{i,j,c} \mapsto a_{i,j,c} / \left(k + \alpha \sum_{r \in N(c)} a_{i,j,r}^2\right)^{\beta}$ across neighboring channels.
- **Dropout** 0.5 in the FC layers.

10 ZFNet and VGG

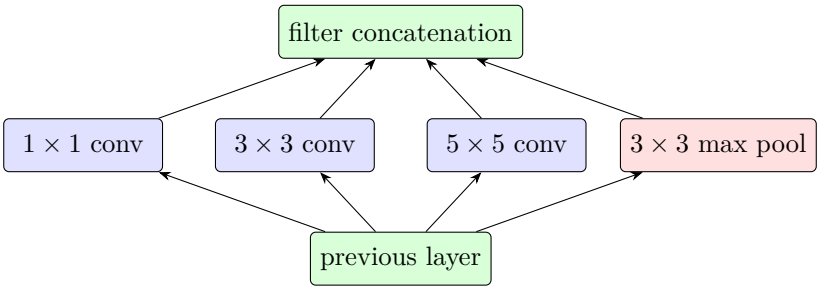
ZFNet retunes AlexNet's kernels and strides. VGG stacks 3×3 convolutions: two give a 5×5 field with $18C^2$ weights (vs. $25C^2$), three 7×7 with $27C^2$ (vs. $49C^2$), plus extra ReLUs. Receptive fields: $r_\ell = r_{\ell-1} + (K_\ell - 1) j_{\ell-1}$, $j_\ell = S_\ell j_{\ell-1}$.

11 GoogLeNet and the Inception Module

Motivation. Several filter sizes in parallel, concatenated, with 1×1 reductions to control cost.

11.1 Naive module

Input $28 \times 28 \times 256$, size-preserving branches: 1×1 (128), 3×3 (192), 5×5 (96), 3×3 max pool (stride 1). Channel concatenation gives $128 + 192 + 96 + 256 = 672$ channels. Weights: $256 \cdot 128 + 9 \cdot 256 \cdot 192 + 25 \cdot 256 \cdot 96 = 1{,}089{,}536$.



Naive Inception: parallel branches concatenated along depth.

11.2 Bottleneck module

1×1 reductions to 64 channels before the 3×3, 5×5 convolutions and after pooling: outputs $128 + 192 + 96 + 64 = 480$ channels and

$$32{,}768 + (16{,}384 + 110{,}592) + (16{,}384 + 153{,}600) + 16{,}384 = 346{,}112 \text{ weights}$$

(about 330k), a 68% reduction.

Theorem 7.12: when a bottleneck saves weights

A direct $K \times K$ map $C \to D$ costs K^2CD; reducing to R channels first costs $CR + K^2RD$. This is cheaper iff $R < \dfrac{K^2CD}{C + K^2D}$ (for $K = 5, C = 256, D = 96$ the threshold is ≈ 231).

GoogLeNet stacks such modules (22 weight layers) and ends with global average pooling instead of a large FC stack.

11.3 Split–transform–merge

A $256 \to 128 \to 128 \to 256$ block (1×1, 3×3 padding 1, 1×1, biases) has $32{,}896+147{,}584+33{,}024 = 213{,}504$ parameters; thirty-two summed $256 \to 4 \to 4 \to 256$ paths have $32(1028 + 148 + 1280) = 78{,}592$ (the 3×3 cost drops by 32). Implementation: three `nn.ModuleLists` of 32 convolutions and a sum, or `groups=32`.

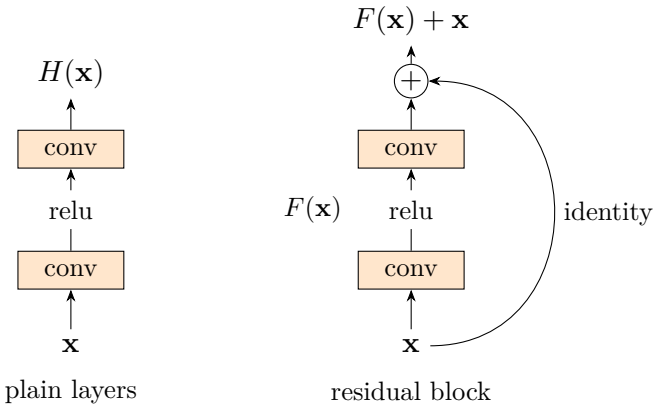
12 Why Depth Can Fail, and Residual Networks

Deeper plain networks can have *higher training error*: an optimization (degradation) problem, not overfitting, since with identity layers the optimal loss is no worse. For $\mathbf{x}_{\ell+1} = f_\ell(\mathbf{x}_\ell)$, $\nabla_{\mathbf{x}_0} L = J_0^\top \cdots J_{L-1}^\top \nabla_{\mathbf{x}_L} L$; Jacobian norms $\le \rho < 1$ shrink it like ρ^L ($0.9^{50} \approx 0.005$).

Theorem 7.13: residual block

$\mathbf{y} = \mathbf{x} + F(\mathbf{x}; \boldsymbol{\theta})$ gives $\dfrac{\partial \mathbf{y}}{\partial \mathbf{x}} = \mathbf{I} + J_F(\mathbf{x})$ and $\nabla_{\mathbf{x}} L = \mathbf{g} + J_F^\top \mathbf{g}$ with $\mathbf{g} = \nabla_{\mathbf{y}} L$.

The identity path carries the gradient when J_F is small, and $F = 0$ is the identity: the block learns $F = H - \mathrm{id}$. Not a guarantee ($J_F = -\mathbf{I}$ cancels; a ReLU after the sum alters the identity). Shape changes use $\mathbf{y} = P(\mathbf{x}) + F(\mathbf{x})$ (1×1 or strided convolution); addition keeps the channel count. Bottleneck $256 \to 64 \to 64 \to 256$: $2 \cdot 256 \cdot 64 + 9 \cdot 64^2 = 69{,}632$ weights vs. $18 \cdot 256^2 = 1{,}179{,}648$ for two full 3×3 layers. On CIFAR-10, residual nets remove the degradation; residual-MLP backpropagation: Chapter 5.



A residual block learns $F(\mathbf{x}) = H(\mathbf{x}) - \mathbf{x}$; gradients pass through the shortcut.

13 Saliency Maps

Differentiate the class score in the input (not the weights):

$$G_{i,j,k} = \frac{\partial s_c(X)}{\partial X_{i,j,k}}, \qquad M_{i,j} = \max_k |G_{i,j,k}|.$$

By Taylor, $s_c(X + \Delta X) \approx s_c(X) + \langle \nabla_X s_c(X), \Delta X \rangle$: large $|G|$ means high sensitivity. For $s(\mathbf{x}) = 2x_1 - 3x_2 + 0.1x_3$, x_2 ranks first. Saliency is local sensitivity, not causal explanation.

Chapter 8. Universal Approximation, Barron's Theorem and Generalization

Lectures 10/21, 10/26: measures, normed spaces, Cybenko's theorem; Barron's theorem and proof; Rademacher complexity

1 Three Sources of Error

Target f^*, architecture class $\mathcal{F}$, best element $f_{\text{app}} \in \mathcal{F}$, empirical risk minimizer f_{gen}, optimizer output f_{opt}:

$$\underbrace{f^* \to f_{\text{app}}}_{\text{approximation}} + \underbrace{f_{\text{app}} \to f_{\text{gen}}}_{\text{generalization (estimation)}} + \underbrace{f_{\text{gen}} \to f_{\text{opt}}}_{\text{optimization}}.$$

Here: approximation error of one-hidden-layer networks (universality; Barron rate).

2 Preliminaries from Analysis

2.1 Measures

Definition 8.1: measures on a compact set $K \subset \mathbb{R}^n$

The **Borel sets** of K form the smallest collection containing all open subsets of K and closed under complements in K and countable unions. A (nonnegative) **measure** μ assigns $\mu(A) \in [0, \infty]$ to Borel sets with

$$\mu(\emptyset) = 0, \qquad \mu\Big(\bigcup_{j=1}^{\infty} A_j\Big) = \sum_{j=1}^{\infty} \mu(A_j) \quad \text{for pairwise disjoint } A_j.$$

A **signed measure** is the same with real (possibly negative) values, e.g. a difference of two finite measures. $\mu = 0$ means $\mu(A) = 0$ for every Borel set A. $M(I_n)$ denotes the finite signed regular Borel measures on the unit cube $I_n = [0,1]^n$.

2.2 Normed spaces and density

Definition 8.2: linear and normed spaces

A real **linear space** is closed under addition and real scaling with the vector-space rules, e.g. $C(K) = \{f : K \to \mathbb{R} \text{ continuous}\}$. A **normed linear space** is a linear space X with $\|\cdot\| : X \to [0, \infty)$ such that

$$\|u\| = 0 \iff u = 0, \qquad \|cu\| = |c|\|u\|, \qquad \|u + v\| \le \|u\| + \|v\|.$$

On $C(K)$ we use the sup norm $\|f\|_\infty = \max_{x \in K} |f(x)|$. A subset $D \subseteq X$ is **dense** in X if for every $f \in X$ and $\varepsilon > 0$ there is $g \in D$ with $\|f - g\| < \varepsilon$.

Density in $(C(I_n), \|\cdot\|_\infty)$ is uniform approximation.

3 Cybenko's Universal Approximation Theorem

Motivation. One sigmoidal hidden layer approximates any continuous function uniformly (an existence proof via Hahn–Banach and Riesz).

Definition 8.3: sigmoidal and discriminatory

$\sigma : \mathbb{R} \to [0,1]$ is **sigmoidal** if $\lim_{t\to-\infty}\sigma(t) = 0$ and $\lim_{t\to+\infty}\sigma(t) = 1$. For $\mu \in M(I_n)$, σ is **discriminatory** if

$$\int_{I_n} \sigma(\mathbf{w}^\top \mathbf{x} + \theta)\, d\mu(\mathbf{x}) = 0 \quad \text{for all } \mathbf{w} \in \mathbb{R}^n,\ \theta \in \mathbb{R} \qquad \Longrightarrow \qquad \mu = 0.$$

Theorem 8.4: Cybenko 1989 (UAT)

Let σ be any continuous sigmoidal function. Then finite sums

$$G(\mathbf{x}) = \sum_{j=1}^{N} \alpha_j\, \sigma(\mathbf{w}_j^\top \mathbf{x} + \theta_j), \qquad \mathbf{w}_j \in \mathbb{R}^n,\ \alpha_j, \theta_j \in \mathbb{R},\ N \in \mathbb{N},$$

are dense in $C(I_n)$: for every continuous f and $\varepsilon > 0$ some G satisfies $\sup_{\mathbf{x}\in I_n} |f(\mathbf{x}) - G(\mathbf{x})| < \varepsilon$.

Ingredients: Theorems 8.5 (Hahn–Banach), 8.6 (Riesz), 8.9 (discriminatory).

3.1 The functional-analytic lemmas

Theorem 8.5: a consequence of Hahn–Banach

Let W be a linear, non-dense subspace of a normed linear space X. Then there is a bounded linear functional L on X with $L \neq 0$ on X and $L|_W = 0$.

Proof. Pick $x_0 \notin \overline{W} \neq X$; $d = \inf_{w\in\overline{W}} \|x_0 - w\| > 0$. On $\overline{W} \oplus \mathbb{R}x_0$ let $\ell(w + tx_0) = t$: linear, $\ell|_W = 0$, $\ell(x_0) = 1$, and $|t|d \le |t|\|x_0 + w/t\| = \|w + tx_0\|$ $(t \neq 0)$, so $\|\ell\| \le 1/d$. Extend by Hahn–Banach. ∎

Theorem 8.6: Riesz representation theorem

Let $K \subset \mathbb{R}^n$ be compact. For every bounded linear functional F on $C(K)$ there is a unique finite signed (regular Borel) measure μ on K with

$$F(f) = \int_K f(\mathbf{x})\, d\mu(\mathbf{x}) \qquad \text{for all } f \in C(K).$$

(For positive functionals the measure is nonnegative.)

Theorem 8.7: an annihilating measure

Let W be a linear, non-dense subspace of $C(I_n)$. Then there is $\mu \in M(I_n)$, $\mu \neq 0$, with $\int_{I_n} h\, d\mu = 0$ for all $h \in W$.

Proof [Supplement]. Theorem 8.5 gives bounded $L \neq 0$, $L|_W = 0$; Riesz gives $L(f) = \int f\, d\mu$ with $\mu \neq 0$, and $\int h\, d\mu = L(h) = 0$ on W. ∎

3.2 Density for discriminatory functions

Theorem 8.8: density for discriminatory activations

If σ is continuous and discriminatory, the sums G above are dense in $C(I_n)$.

Proof. $W = \{G\}$ is a linear subspace of $C(I_n)$. If not dense, Theorem 8.7 gives $\mu \neq 0$ annihilating W, in particular

$$\int_{I_n} \sigma(\mathbf{w}^\top \mathbf{x} + \theta)\, d\mu(\mathbf{x}) = 0 \qquad \text{for all } \mathbf{w} \in \mathbb{R}^n,\ \theta \in \mathbb{R}.$$

so $\mu = 0$ (discriminatory): contradiction. ■

3.3 Continuous sigmoidals are discriminatory

Theorem 8.9: continuous sigmoidals are discriminatory

Every continuous sigmoidal function is discriminatory for all $\mu \in M(I_n)$.

Proof [Supplement]. Assume $\int \sigma(\mathbf{w}^\top \mathbf{x} + \theta)\, d\mu = 0$ for all $\mathbf{w}, \theta$. Fix $\mathbf{w}, \theta, \varphi$ and let $\sigma_\lambda(\mathbf{x}) = \sigma(\lambda(\mathbf{w}^\top \mathbf{x} + \theta) + \varphi)$. As $\lambda \to \infty$, $\sigma_\lambda \to 1$ on $\Pi = \{\mathbf{w}^\top \mathbf{x} + \theta > 0\}$, $\to 0$ on $\{\mathbf{w}^\top \mathbf{x} + \theta < 0\}$, $= \sigma(\varphi)$ on $H = \{\mathbf{w}^\top \mathbf{x} + \theta = 0\}$; by dominated convergence ($|\sigma_\lambda| \leq 1$)

$$0 = \int \sigma_\lambda \, d\mu \ \longrightarrow\ \mu(\Pi) + \sigma(\varphi)\mu(H) \qquad \text{for every } \varphi.$$

$\varphi \to \mp\infty$ gives $\mu(\Pi) = \mu(H) = 0$ for all half-spaces and hyperplanes. Fix $\mathbf{w}$; $F(h) = \int h(\mathbf{w}^\top \mathbf{x})\, d\mu(\mathbf{x})$ is bounded linear on bounded measurable h, with $F(\mathbf{1}_{[\theta,\infty)}) = \mu(\Pi) + \mu(H) = 0$ and $F(\mathbf{1}_{(\theta,\infty)}) = 0$; so F vanishes on interval indicators, simple functions, and by uniform approximation all bounded measurable h. With $h = \cos, \sin$: $\int e^{i\mathbf{w}^\top \mathbf{x}}\, d\mu = 0$ for all $\mathbf{w}$, so $\mu = 0$. ■

Proof of Cybenko's theorem. Theorem 8.9 shows σ is discriminatory; Theorem 8.8 gives density. ■

Remark 8.10: what the theorem says and does not say

Any compact K works after rescaling. Pure existence: no bound on N, no algorithm, no generalization. Extends to every continuous non-polynomial activation incl. ReLU (Leshno et al. 1993). 1D intuition: steep sigmoid $\approx$ step, two steps make a bump, bumps sum to any continuous function.

4 Barron's Theorem: a Dimension-Free Rate

4.1 Fourier preliminaries

For $f \in L^1(\mathbb{R}^d)$ the Fourier transform is $\hat{f}(\boldsymbol{\xi}) = \int_{\mathbb{R}^d} f(\mathbf{x}) e^{-2\pi i \boldsymbol{\xi}^\top \mathbf{x}}\, d\mathbf{x}$. The Sobolev space $H^s(\mathbb{R}^d)$ consists of f with $\int |\hat{f}(\boldsymbol{\xi})|^2 (1 + |\boldsymbol{\xi}|^2)^s\, d\boldsymbol{\xi} < \infty$.

Definition 8.11: Barron class

For $C > 0$,

$$\Gamma_C = \Big\{ f \in L^1(\mathbb{R}^d) : \ \|\hat{f}\|_{L^1(\mathbb{R}^d)} < \infty, \ \int_{\mathbb{R}^d} |2\pi\boldsymbol{\xi}| \, |\hat{f}(\boldsymbol{\xi})| \, d\boldsymbol{\xi} < C \Big\}.$$

On a domain Ω one also uses the spectral Barron norm $\|f\|_{\mathcal{B}^s(\Omega)} = \inf_{f_e|_\Omega = f} \int_{\mathbb{R}^d} (1 + |\boldsymbol{\xi}|)^s |\hat{f}_e(\boldsymbol{\xi})| \, d\boldsymbol{\xi}$, the infimum over all extensions f_e of f beyond Ω.

Theorem 8.12 (Theorem 1, Barron 1993)

Let σ be sigmoidal, $f \in \Gamma_C$ and $B_1^d = \{\mathbf{x} \in \mathbb{R}^d : \|\mathbf{x}\| \le 1\}$. Then for every $c > 4C^2$ and every $N \in \mathbb{N}$ there is a network f_N with N neurons such that

$$\frac{1}{|B_1^d|} \int_{B_1^d} |f(\mathbf{x}) - f_N(\mathbf{x})|^2 \, d\mathbf{x} < \frac{c}{N}.$$

The rate $O(1/N)$ is free of d; the price is the spectral condition $f \in \Gamma_C$.

Remark 8.13: curse of dimensionality

A fixed linear family needs $\sim \varepsilon^{-d/s}$ terms for s-smooth functions; Barron needs $\approx c/\varepsilon$ neurons for every d (d enters only via C).

5 Proof of Barron's Theorem

Lemma 1 (Maurey): closed-convex-hull elements are approximated at rate $1/N$ by N-term averages. Lemma 2: Barron functions lie in the hull of half-space indicators. Finally indicators become sigmoids.

5.1 Convex hulls and Maurey's lemma

Definition 8.14: convex hull

For a subset G of a vector space,

$$\mathrm{co}(G) = \Big\{ \sum_{j=1}^{n} \alpha_j x_j : \ n \in \mathbb{N}, \ x_j \in G, \ \alpha_j \ge 0, \ \sum_{j=1}^{n} \alpha_j = 1 \Big\},$$

and $\overline{\mathrm{co}}(G)$ denotes its closure in a given norm.

Theorem 8.15 (Lemma 1, Maurey)

Let H be a Hilbert space and $G \subset H$ with $\|g\|_H \le B$ for all $g \in G$, for some $B > 0$. Let $f \in \overline{\mathrm{co}}(G)$. Then for every $N \in \mathbb{N}$ and every $c > B^2$ there exist $g_1, \dots, g_N \in G$ with

$$\Big\| f - \frac{1}{N} \sum_{i=1}^{N} g_i \Big\|_H^2 \le \frac{c}{N}.$$

Proof. Fix N, $\varepsilon > 0$, and $\alpha_j \in [0,1]$ summing to 1, $h_j \in G$ with

$$f^* = \sum_{j=1}^{m} \alpha_j h_j \quad \text{satisfies} \quad \|f - f^*\|_H < \varepsilon.$$

Claim. There are $g_1, \dots, g_N \in \{h_1, \dots, h_m\}$ with $\left\|f^* - \frac{1}{N}\sum_i g_i\right\|_H^2 \le B^2/N$.
Let $X_1, \dots, X_N$ be iid with $P(X_i = h_j) = \alpha_j$, so $\mathbb{E}X_i = f^*$. Expanding,

$$\mathbb{E}\Big\|f^* - \frac{1}{N}\sum_{j=1}^{N} X_j\Big\|^2 = \frac{1}{N^2}\,\mathbb{E}\Big[\sum_{j=1}^{N}\|f^* - X_j\|^2 + \sum_{i\neq j}\langle f^* - X_i, f^* - X_j\rangle\Big].$$

For $i \neq j$, $\mathbb{E}\langle X_i, X_j\rangle = \langle f^*, f^*\rangle$ by independence, so

$$\mathbb{E}\langle f^* - X_i, f^* - X_j\rangle = \langle f^*, f^*\rangle - \langle f^*, \mathbb{E}X_j\rangle - \langle \mathbb{E}X_i, f^*\rangle + \mathbb{E}\langle X_i, X_j\rangle = 0.$$

The diagonal terms are identically distributed:

$$\begin{aligned}\mathbb{E}\Big\|f^* - \frac{1}{N}\sum_j X_j\Big\|^2 &= \frac{1}{N}\mathbb{E}\|f^* - X_1\|^2 = \frac{1}{N}\mathbb{E}[\|f^*\|^2 - 2\langle f^*, X_1\rangle + \|X_1\|^2] \\ &= \frac{1}{N}(\mathbb{E}\|X_1\|^2 - \|f^*\|^2) \le \frac{B^2}{N},\end{aligned}$$

by $\mathbb{E}\langle f^*, X_1\rangle = \|f^*\|^2$, $\|X_1\| \le B$. Some realization $g_i = X_i(\omega)$ is at most the mean: the claim.
By the triangle inequality,

$$\Big\|f - \frac{1}{N}\sum_i g_i\Big\|_H \le \|f - f^*\|_H + \Big\|f^* - \frac{1}{N}\sum_i g_i\Big\|_H < \varepsilon + \frac{B}{\sqrt{N}}.$$

$\varepsilon < (\sqrt{c} - B)/\sqrt{N}$ makes this $< \sqrt{c/N}$ (as $c > B^2$); square. ∎

It remains to put every $f \in \Gamma_C$ in the closed convex hull of affine Heaviside functions.

5.2 Barron functions lie in a convex hull of half-space indicators

Normalized norm on the unit ball:

$$\|g\|_{L_0^2(B_1^d)} = \Big(\frac{1}{|B_1^d|}\int_{B_1^d} |g(\mathbf{x})|^2\, d\mathbf{x}\Big)^{1/2},$$

and scaled half-space indicators

$$G_C = \Big\{\mathbf{x} \in B_1^d \mapsto \gamma\, \mathbf{1}_{\mathbb{R}_+}(\langle \mathbf{a}, \mathbf{x}\rangle + b) : \ \mathbf{a} \in \mathbb{R}^d,\ b \in \mathbb{R},\ |\gamma| \le 2C\Big\},$$

$\mathbf{1}_{\mathbb{R}_+}(t) = \mathbf{1}\{t \ge 0\}$ (Heaviside). Every $g \in G_C$ has $\|g\|_{L_0^2(B_1^d)} \le |\gamma| \le 2C$.

Theorem 8.16 (Lemma 2, Barron 1993)

Let $d \in \mathbb{N}$, $C > 0$ and $f \in \Gamma_C$. Then

$$f|_{B_1^d} - f(\mathbf{0}) \in \overline{\mathrm{co}}(G_C),$$

where the closure is taken in $L_0^2(B_1^d)$.

Proof. $K = \overline{\mathrm{co}}(G_C)$ is closed convex in the Hilbert space $L^2_0(B^d_1)$; $h = f|_{B^d_1} - f(\mathbf{0})$. (The lecture sketched Step 1 and a Riemann-sum idea; parts marked [Supplement] fill the gaps after Barron.)

Step 1 (integral representation). $f, \hat{f} \in L^1(\mathbb{R}^d)$, so by Fourier inversion

$$f(\mathbf{x}) - f(\mathbf{0}) = \int_{\mathbb{R}^d} \hat{f}(\boldsymbol{\xi})(e^{2\pi i\langle \mathbf{x},\boldsymbol{\xi}\rangle} - 1)\, d\boldsymbol{\xi}.$$

With phase $\hat{f}(\boldsymbol{\xi}) = |\hat{f}(\boldsymbol{\xi})|e^{i\kappa(\boldsymbol{\xi})}$,

$$f(\mathbf{x}) - f(\mathbf{0}) = \int_{\mathbb{R}^d} |\hat{f}(\boldsymbol{\xi})|(e^{i(2\pi\langle \mathbf{x},\boldsymbol{\xi}\rangle + \kappa(\boldsymbol{\xi}))} - e^{i\kappa(\boldsymbol{\xi})})\, d\boldsymbol{\xi}.$$

and taking real parts (f real),

$$f(\mathbf{x}) - f(\mathbf{0}) = \int_{\mathbb{R}^d} \frac{\cos\left(2\pi\langle \mathbf{x},\boldsymbol{\xi}\rangle + \kappa(\boldsymbol{\xi})\right) - \cos\kappa(\boldsymbol{\xi})}{|2\pi\boldsymbol{\xi}|}\, |2\pi\boldsymbol{\xi}|\, |\hat{f}(\boldsymbol{\xi})|\, d\boldsymbol{\xi}.$$

(the integrand vanishes at $\boldsymbol{\xi} = \mathbf{0}$). Let $C_f = \int |2\pi\boldsymbol{\xi}||\hat{f}(\boldsymbol{\xi})|\, d\boldsymbol{\xi} < C$ and $d\Lambda(\boldsymbol{\xi}) = C_f^{-1}|2\pi\boldsymbol{\xi}||\hat{f}(\boldsymbol{\xi})|\, d\boldsymbol{\xi}$, a probability measure (lecture: C for C_f; if $C_f = 0$, $h = 0 \in K$). With

$$q_{\boldsymbol{\xi}}(\mathbf{x}) = \frac{\cos\left(2\pi\langle \mathbf{x},\boldsymbol{\xi}\rangle + \kappa(\boldsymbol{\xi})\right) - \cos\kappa(\boldsymbol{\xi})}{|2\pi\boldsymbol{\xi}|}, \qquad k_{\boldsymbol{\xi}} = C_f q_{\boldsymbol{\xi}},$$

we obtain

$$f(\mathbf{x}) - f(\mathbf{0}) = C_f \int_{\mathbb{R}^d} q_{\boldsymbol{\xi}}(\mathbf{x})\, d\Lambda(\boldsymbol{\xi}) = \int_{\mathbb{R}^d} k_{\boldsymbol{\xi}}(\mathbf{x})\, d\Lambda(\boldsymbol{\xi}). \tag{5}$$

Step 2 (each $k_{\boldsymbol{\xi}}$ lies in K). [Supplement]

$k_{\mathbf{0}} = 0 \in G_C$ ($\gamma = 0$). For $\boldsymbol{\xi} \neq \mathbf{0}$, $\mathbf{u} = \boldsymbol{\xi}/|\boldsymbol{\xi}|$, $t = \langle \mathbf{u}, \mathbf{x}\rangle \in [-1, 1]$, $k_{\boldsymbol{\xi}}(\mathbf{x}) = r(t)$ with

$$r(t) = C_f \frac{\cos\left(2\pi|\boldsymbol{\xi}|t + \kappa\right) - \cos\kappa}{2\pi|\boldsymbol{\xi}|}, \qquad r(0) = 0, \qquad |r'(t)| = C_f|\sin\left(2\pi|\boldsymbol{\xi}|t + \kappa\right)| \le C_f,$$

so $|r(s) - r(t)| \le C_f|s - t|$.

Staircase. $t_k = k/m$ ($k = 0, \ldots, m$),

$$s_m(t) = \sum_{k=1}^{m} \alpha_k \mathbf{1}_{\mathbb{R}_+}(t - t_k) + \sum_{k=1}^{m} \alpha'_k \mathbf{1}_{\mathbb{R}_+}(-t - t_k), \qquad \alpha_k = r(t_k) - r(t_{k-1}),\ \ \alpha'_k = r(-t_k) - r(-t_{k-1}).$$

For $t \in [t_j, t_{j+1})$, $t \ge 0$, the second sum is 0 and the first telescopes to $r(t_j)$; symmetrically for $t < 0$. The grid point is within $1/m$, so

$$\sup_{t\in[-1,1]} |r(t) - s_m(t)| \le \frac{C_f}{m}.$$

Coefficients. Each $|\alpha_k|, |\alpha'_k| \le C_f/m$, so $S := \sum_k(|\alpha_k| + |\alpha'_k|) \le 2C_f \le 2C$.

Convex combination. $\mathbf{1}_{\mathbb{R}_+}(\langle \pm\mathbf{u}, \mathbf{x}\rangle - t_k)$ are half-space indicators; call the $2m$ coefficients β_j, indicators e_j. If $S = 0$, $s_m = 0 \in G_C$; otherwise

$$s_m(\langle \mathbf{u}, \cdot\rangle) = \sum_j \frac{|\beta_j|}{S}\Big(S\ \mathrm{sign}(\beta_j)\, e_j\Big),$$

weights $|\beta_j|/S$ summing to 1, brackets in G_C ($S \le 2C$): $s_m(\langle \mathbf{u}, \cdot\rangle) \in \mathrm{co}(G_C)$.

Limit. $\|g\|_{L^2_0(B^d_1)} \le \sup_{B^d_1} |g|$ (an average against a probability measure), so $\|k_{\boldsymbol{\xi}} - s_m(\langle \mathbf{u}, \cdot\rangle)\|_{L^2_0} \le C_f/m \to 0$, and $k_{\boldsymbol{\xi}} \in K$.

Step 3 (the average stays in K). The lecture's Riemann sum $\sum_{\boldsymbol{\theta}} k_{\boldsymbol{\theta}} \Lambda(I_{\boldsymbol{\theta}})$ over cubes $I_{\boldsymbol{\theta}}$ ($\boldsymbol{\theta} \in \frac{1}{n}\mathbb{Z}^d$) needs truncation and continuity of $\boldsymbol{\xi} \mapsto k_{\boldsymbol{\xi}}$ (the phase may jump where $\hat{f} = 0$); the following avoids both. [Supplement]

Projection. h has a nearest point p in the closed convex K:

$$\langle h - p,\, k - p \rangle \le 0 \qquad \text{for all } k \in K.$$

Averaging. By (5), $h - p = \int (k_{\boldsymbol{\xi}} - p)\, d\Lambda(\boldsymbol{\xi})$. Pair with $h - p$ and use Fubini ($|k_{\boldsymbol{\xi}}| \le C_f$ since $|r(t)| \le C_f |t|$; L^2 functions on the ball are integrable); by Step 2,

$$\|h - p\|_{L^2_0}^2 = \int_{\mathbb{R}^d} \langle h - p,\, k_{\boldsymbol{\xi}} - p \rangle \, d\Lambda(\boldsymbol{\xi}) \le 0.$$

Hence $h = p \in K$. ■

5.3 Completing the proof of Theorem 1

Proof of Barron's theorem. Lemma 8.16 and Lemma 8.15 ($H = L^2_0(B^d_1)$, $B = 2C$) give, for $c > 4C^2$, $|\gamma_i| \le 2C$, $\mathbf{a}_i$, b_i with

$$\frac{1}{|B^d_1|} \int_{B^d_1} \Big| f(\mathbf{x}) - f(\mathbf{0}) - \frac{1}{N} \sum_{i=1}^{N} \gamma_i \mathbf{1}_{\mathbb{R}_+}\big(\langle \mathbf{a}_i, \mathbf{x} \rangle + b_i\big) \Big|^2 d\mathbf{x} < \frac{c}{N}.$$

$\sigma(\lambda t) \to \mathbf{1}_{\mathbb{R}_+}(t)$ for $t \neq 0$, and the hyperplanes $\{\langle \mathbf{a}_i, \mathbf{x} \rangle + b_i = 0\}$ are null (for $\mathbf{a}_i = \mathbf{0}$, $b_i = 0$ the term is the constant γ_i, also given by $b_i = 1$). [Supplement]By dominated convergence, $\sigma\big(\lambda(\langle \mathbf{a}_i, \mathbf{x} \rangle + b_i)\big)$ with large λ keeps the error below c/N:

$$f_N(\mathbf{x}) = f(\mathbf{0}) + \sum_{i=1}^{N} \frac{\gamma_i}{N}\, \sigma\big(\lambda \langle \mathbf{a}_i, \mathbf{x} \rangle + \lambda b_i\big)$$

has N neurons and output bias $f(\mathbf{0})$. ■

6 Generalization and Rademacher Complexity

6.1 Population and empirical risk

With iid $\{(\mathbf{x}_i, y_i)\}_{i=1}^n \sim P$, loss ℓ and class $\mathcal{F}$ we want

$$\arg\min_{f \in \mathcal{F}} \; \mathbb{E}_{(\mathbf{x},y)\sim P}\big[\ell(y, f(\mathbf{x}))\big] \quad \text{(population risk)},$$

but can only compute

$$\arg\min_{f \in \mathcal{F}} \; \frac{1}{n} \sum_{i=1}^{n} \ell(y_i, f(\mathbf{x}_i)) \quad \text{(empirical risk)}.$$

Generalization error is controlled by $\sup_{f \in \mathcal{F}} \big| \frac{1}{n} \sum_i f(\mathbf{z}_i) - \mathbb{E} f \big|$ over the losses $\mathbf{z} = (\mathbf{x}, y) \mapsto \ell(y, f(\mathbf{x}))$.

6.2 Rademacher complexity

Definition 8.17: Rademacher variables and complexity

Random variables $\xi_1, \dots, \xi_n$ are **Rademacher** if they are iid with $P(\xi_i = 1) = P(\xi_i = -1) = \tfrac{1}{2}$. For a class $\mathcal{F}$ and iid samples $\mathbf{z}_1, \dots, \mathbf{z}_n$, the **Rademacher complexity** is

$$\operatorname{Rad}_n(\mathcal{F}) = \mathbb{E}\Big[\sup_{f \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^{n} \xi_i f(\mathbf{z}_i)\Big],$$

the expectation being over both the ξ_i and the $\mathbf{z}_i$.

It measures how well the class fits random signs.

Theorem 8.18: uniform deviation bound

By symmetrization, $\mathbb{E}\sup_{f \in \mathcal{F}} \left|\frac{1}{n}\sum_i f(\mathbf{z}_i) - \mathbb{E}f\right| \le 2\operatorname{Rad}_n(\mathcal{F} \cup -\mathcal{F})$. If moreover every $f \in \mathcal{F}$ takes values in $[0,1]$, then with probability at least $1 - \delta$

$$\sup_{f \in \mathcal{F}} \Big|\frac{1}{n}\sum_{i=1}^{n} f(\mathbf{z}_i) - \mathbb{E}f\Big| \le 2\operatorname{Rad}_n(\mathcal{F} \cup -\mathcal{F}) + \sqrt{\frac{\log(2/\delta)}{2n}}.$$

(Lecture: "$\le 2\operatorname{Rad}(\mathcal{F}) + 1/\sqrt{2n}$", confidence factor suppressed.)

6.3 Two-layer networks

For $\mathcal{F} = \{f_{\boldsymbol{\theta}}(\mathbf{x}) = \sum_j \beta_j \sigma(\mathbf{w}_j^\top \mathbf{x})\}$ with $\sum_j |\beta_j| \le Q$, $\|\mathbf{w}_j\| \le W$, inputs $\|\mathbf{x}\| \le R$ and σ 1-Lipschitz with $\sigma(0) = 0$ (e.g. tanh), the contraction lemma gives

$$\operatorname{Rad}_n(\mathcal{F}) \le \frac{2QWR}{\sqrt{n}} = O\Big(\frac{1}{\sqrt{n}}\Big).$$

It depends on Q, not the number of neurons. With Barron: approximation $1/N$, generalization $1/\sqrt{n}$, both free of d.

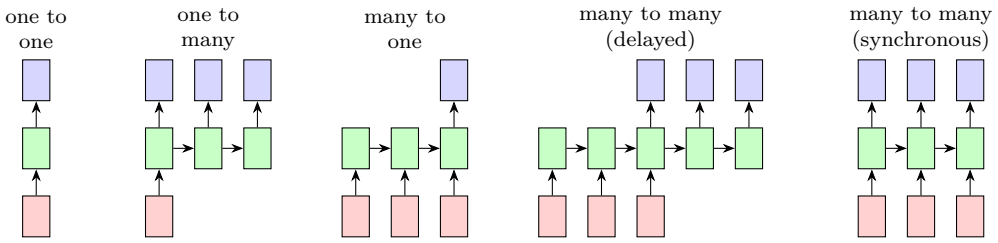
Chapter 9. Recurrent Neural Networks and LSTM

Lectures 10/28, 11/2: RNNs, backpropagation through time, vanishing and exploding gradients, LSTM, language modeling, image captioning (naive)

1 Why Recurrent Networks

For variable-length **sequences**, a recurrent network reads one element at a time and updates an **internal (hidden) state**. Patterns:

Pattern	Input → output	Examples
one to one	single → single	vanilla NN / FFN
one to many	single → sequence	image to video, image captioning
many to one	sequence → single	video classification, action recognition
many to many (delayed)	read all, then emit	translation (English to Korean), video to caption
many to many (synchronous)	emit at every step	video segmentation, video editing, style transfer



Inputs (red), states (green), outputs (blue). Delayed: inputs at $t = 1, 2, 3$, outputs at $t = 4, 5, 6$.

2 The Recurrence

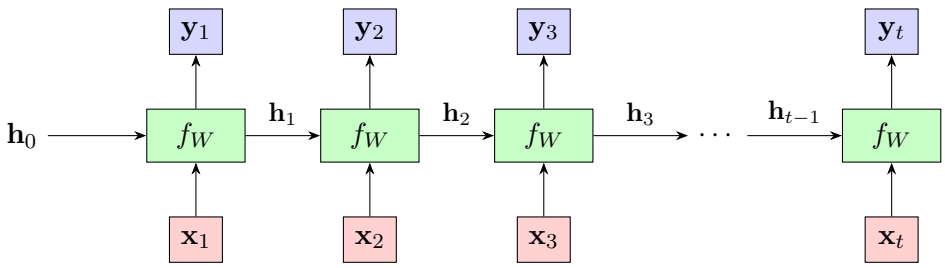
Definition 9.1: recurrent neural network

For inputs $\mathbf{x}_1, \mathbf{x}_2, \ldots,$

$$\mathbf{h}_t = f_W(\mathbf{h}_{t-1}, \mathbf{x}_t), \qquad \mathbf{y}_t = f_{W_{hy}}(\mathbf{h}_t), \qquad t \geq 1,$$

from $\mathbf{h}_0$ (usually $\mathbf{0}$), with f_W, $f_{W_{hy}}$ and their parameters **the same at every time step**.

Unrolled, this is an ordinary graph with one parameter-sharing copy per step, so any length can be processed.



The unrolled RNN; f_W and $f_{W_{hy}}$ are shared.

Definition 9.2: vanilla RNN

With an elementwise activation σ (typically tanh),

$$\mathbf{h}_t = \sigma(U\mathbf{h}_{t-1} + W\mathbf{x}_t + \mathbf{b}), \qquad \hat{\mathbf{y}}_t = W^{\text{out}}\mathbf{h}_t, \qquad \mathbf{h}_0 = \mathbf{0}.$$

$$\mathbf{h}_1 = \sigma(W\mathbf{x}_1 + \mathbf{b}), \quad \hat{\mathbf{y}}_1 = W^{\text{out}}\mathbf{h}_1; \qquad \mathbf{h}_2 = \sigma(U\mathbf{h}_1 + W\mathbf{x}_2 + \mathbf{b}), \quad \hat{\mathbf{y}}_2 = W^{\text{out}}\mathbf{h}_2.$$

(The next section, following the handwritten notes, swaps the letters: W recurrent, U input.)

3 Backpropagation Through Time

3.1 The idea

$L = \sum_{t=1}^{\tau} L_t$, and L_t depends on every earlier step through the state. **Backpropagation through time (BPTT)** is backpropagation on the unrolled graph (one forward and one backward pass over the sequence); shared parameters sum the contributions of all copies.

Remark 9.3: truncated BPTT

Carry the state through the whole sequence but backpropagate only through a recent window.

3.2 A worked two-step example

Scalar RNN:

$$h_t = \tanh(Wh_{t-1} + Ux_t + b), \qquad Y_t = Vh_t + c, \qquad \theta = (W, U, V, b, c),$$

with inputs $(x_1, \dots, x_\tau)$, outputs $(Y_1, \dots, Y_\tau)$, targets $(Z_1, \dots, Z_\tau)$ and $L = \sum_t L_t$, $L_t = \frac{1}{2}(Y_t - Z_t)^2$; the chain rule acts within and across steps. Forward pass, $\tau = 2$:

$$\begin{aligned} a_1 &= Wh_0 + Ux_1 + b, \quad h_1 = \tanh a_1, \quad Y_1 = Vh_1 + c, \\ a_2 &= Wh_1 + Ux_2 + b, \quad h_2 = \tanh a_2, \quad Y_2 = Vh_2 + c, \end{aligned} \qquad L = \tfrac{1}{2}(Y_1 - Z_1)^2 + \tfrac{1}{2}(Y_2 - Z_2)^2.$$

Local derivatives. $\tanh' = 1 - \tanh^2$; ∂ is the derivative in the explicit occurrence:

$$\frac{\partial h_t}{\partial W} = (1 - h_t^2)h_{t-1}, \qquad \frac{\partial h_t}{\partial U} = (1 - h_t^2)x_t, \qquad \frac{\partial h_t}{\partial b} = 1 - h_t^2, \qquad \frac{\partial h_2}{\partial h_1} = (1 - h_2^2)W.$$

Through Y_t,

$$\frac{\partial L_t}{\partial h_t} = \frac{\partial L_t}{\partial Y_t}\frac{\partial Y_t}{\partial h_t} = (Y_t - Z_t)V.$$

Output parameters.

$$\frac{\partial L}{\partial V} = \sum_{t=1}^{2}(Y_t - Z_t)h_t, \qquad \frac{\partial L}{\partial c} = \sum_{t=1}^{2}(Y_t - Z_t).$$

Recurrent weight. L_1 sees W via h_1; L_2 via h_2 directly and via $h_1 \to h_2$:

$$\frac{\partial L}{\partial W} = \frac{\partial L_1}{\partial h_1}\frac{\partial h_1}{\partial W} + \frac{\partial L_2}{\partial h_2}\frac{\partial h_2}{\partial W} + \frac{\partial L_2}{\partial h_2}\frac{\partial h_2}{\partial h_1}\frac{\partial h_1}{\partial W},$$

that is,

$$\frac{\partial L}{\partial W} = (Y_1 - Z_1)V(1 - h_1^2)h_0 + (Y_2 - Z_2)V(1 - h_2^2)h_1 + (Y_2 - Z_2)V(1 - h_2^2)W(1 - h_1^2)h_0.$$

Input weight and bias [Supplement]. Same three paths, h_{t-1} replaced by x_t and 1:

$$\frac{\partial L}{\partial U} = (Y_1 - Z_1)V(1-h_1^2)x_1 + (Y_2 - Z_2)V(1-h_2^2)x_2 + (Y_2 - Z_2)V(1-h_2^2)W(1-h_1^2)x_1,$$
$$\frac{\partial L}{\partial b} = (Y_1 - Z_1)V(1-h_1^2) + (Y_2 - Z_2)V(1-h_2^2) + (Y_2 - Z_2)V(1-h_2^2)W(1-h_1^2).$$

With $h_0 = 0$ the first and third terms of $\partial L/\partial W$ vanish; the others keep all three.

3.3 General formulation

Input $\tau(X) = (u_1, \dots, u_L)$, $u_\ell \in \mathbb{R}^m$; update $q_{\tilde\theta} : \mathbb{R}^n \times \mathbb{R}^m \to \mathbb{R}^n$, e.g. $q_{\tilde\theta}(h,u) = \phi(Wh + Uu + c)$ with $\tilde\theta = (W, U, c) \in \mathbb{R}^p$:

$$h_0 = \mathbf{0}, \qquad h_\ell = q_{\tilde\theta}(h_{\ell-1}, u_\ell) \quad (\ell = 1, \dots, L), \qquad f_\theta(X) = Ah_L + b,$$

with $A \in \mathbb{R}^{1\times n}$, $b \in \mathbb{R}$, $h_\ell \in \mathbb{R}^n$ and $\theta = (A, b, \tilde\theta)$. For a loss $\mathcal{L}(f_\theta(X), Y)$,

$$\frac{\partial \mathcal{L}}{\partial \theta} = \frac{\partial \mathcal{L}}{\partial f}(f_\theta(X), Y)\,\frac{\partial f_\theta(X)}{\partial \theta}, \qquad \frac{\partial f_\theta(X)}{\partial A} = h_L^\top, \quad \frac{\partial f_\theta(X)}{\partial b} = 1, \quad \frac{\partial f_\theta(X)}{\partial \tilde\theta} = A\frac{dh_L}{d\tilde\theta}.$$

$dh_L/d\tilde\theta$ is total; $\partial_{\tilde\theta} q_{\tilde\theta}(h_{\ell-1}, u_\ell)$ holds $h_{\ell-1}$ fixed; $J_k = \partial_h q_{\tilde\theta}(h_{k-1}, u_k) \in \mathbb{R}^{n\times n}$.

Theorem 9.4: BPTT formula

For $\ell \le L$,

$$\frac{\partial h_L}{\partial h_\ell} = J_L J_{L-1} \cdots J_{\ell+1}, \qquad \frac{dh_L}{d\tilde\theta} = \sum_{\ell=1}^{L} \frac{\partial h_L}{\partial h_\ell}\,\partial_{\tilde\theta} q_{\tilde\theta}(h_{\ell-1}, u_\ell),$$

(the empty product for $\ell = L$ is the identity). Consequently

$$\frac{\partial \mathcal{L}(f_\theta(X), Y)}{\partial \tilde\theta} = \sum_{\ell=1}^{L} \frac{\partial \mathcal{L}(f_\theta(X), Y)}{\partial h_\ell}\,\partial_{\tilde\theta} q_{\tilde\theta}(h_{\ell-1}, u_\ell), \qquad \frac{\partial \mathcal{L}}{\partial h_\ell} = \frac{\partial \mathcal{L}}{\partial h_L}\frac{\partial h_L}{\partial h_\ell} = \frac{\partial \mathcal{L}}{\partial h_L} J_L \cdots J_{\ell+1}.$$

Proof. Step 1 (state Jacobian). For $k > \ell$, $\partial h_k/\partial h_\ell = J_k\,\partial h_{k-1}/\partial h_\ell$; iterate from I.

Step 2 (one-step recursion). Directly and through $h_{\ell-1}$:

$$\frac{dh_\ell}{d\tilde\theta} = \partial_{\tilde\theta} q_{\tilde\theta}(h_{\ell-1}, u_\ell) + J_\ell \frac{dh_{\ell-1}}{d\tilde\theta}, \qquad \frac{dh_0}{d\tilde\theta} = 0.$$

Step 3 (unrolling). Induction on L; $L = 1$ is Step 2. From $L-1$, Steps 2 and 1 give

$$\frac{dh_L}{d\tilde\theta} = \partial_{\tilde\theta} q_{\tilde\theta}(h_{L-1}, u_L) + \sum_{\ell=1}^{L-1} J_L \frac{\partial h_{L-1}}{\partial h_\ell}\,\partial_{\tilde\theta} q_{\tilde\theta}(h_{\ell-1}, u_\ell) = \sum_{\ell=1}^{L} \frac{\partial h_L}{\partial h_\ell}\,\partial_{\tilde\theta} q_{\tilde\theta}(h_{\ell-1}, u_\ell).$$

Step 4 (loss). $\mathcal{L}$ sees $\tilde\theta$ only through h_L; multiply by $\partial\mathcal{L}/\partial h_L$. ■

The sum over ℓ is the parameter-sharing sum, each copy weighted by its influence on the loss.

4 Vanishing and Exploding Gradients

4.1 RNNs are very deep networks

$\partial h_L/\partial h_\ell = J_L \cdots J_{\ell+1}$ multiplies up to hundreds of Jacobians: factors above 1 explode (**exploding gradients**), below 1 vanish (**vanishing gradients**, the bigger problem). Scalar tanh RNN (Section 3.2): factor $(1-h_t^2)W$ [Supplement].

Theorem 9.5: a sufficient condition for vanishing gradients [Supplement]

Let $q_{\tilde{\theta}}(h,u) = \tanh(Wh + Uu + c)$ and let $\|\cdot\|$ be the spectral norm. Then

$$\left\| \frac{\partial h_L}{\partial h_\ell} \right\| \le \|W\|^{L-\ell}.$$

If $\|W\| < 1$ the signal decays geometrically in $L - \ell$.

Proof. Step 1. $J_k = \operatorname{diag}(1 - \tanh^2(a_k))\, W = \operatorname{diag}(1 - h_k^2)\, W$, $a_k = W h_{k-1} + U u_k + c$.

Step 2. Diagonal entries in $[0,1]$: $\|J_k\| \le \|W\|$.

Step 3. Submultiplicativity and Theorem 9.4: $\|J_L \cdots J_{\ell+1}\| \le \prod_{k=\ell+1}^{L} \|J_k\| \le \|W\|^{L-\ell}$. ■

$\|W\| > 1$ need not explode: saturated units ($h_k^2 \approx 1$) shrink the factors. [Supplement]

4.2 Exploding gradients and gradient clipping

When the direction is fine but the length too large, **gradient clipping** with threshold $v > 0$:

$$g \;\leftarrow\; \min\Big(1, \frac{v}{\|g\|}\Big) g = \begin{cases} g, & \|g\| \le v, \\ \dfrac{v}{\|g\|}\, g, & \text{otherwise.} \end{cases}$$

same direction, length $\min(\|g\|, v)$ (the slide's max means min). At a cliff an unclipped step jumps away from the valley.

4.3 Vanishing gradients

$\partial \mathcal{L}/\partial h_L$ may be moderate while

$$\frac{\partial \mathcal{L}}{\partial h_\ell} = \frac{\partial \mathcal{L}}{\partial h_L} \frac{\partial h_L}{\partial h_{L-1}} \frac{\partial h_{L-1}}{\partial h_{L-2}} \cdots \frac{\partial h_{\ell+1}}{\partial h_\ell}$$

is tiny; since

$$\frac{\partial \mathcal{L}}{\partial u_\ell} = \frac{\partial \mathcal{L}}{\partial h_\ell} \frac{\partial h_\ell}{\partial u_\ell},$$

small changes of u_ℓ barely affect $\mathcal{L}$ (loosely, u_ℓ is forgotten; large changes may still matter), and the model cannot learn what to keep at time ℓ.

5 Long Short-Term Memory

Motivation. An additively updated, gated memory cell lets gradients cross long time spans.

5.1 The LSTM cell

Vanilla: $h_t = \tanh\left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right)$. The LSTM adds a gated **cell state** c_t (long memory).

Definition 9.6: LSTM

With σ the logistic sigmoid and $\odot$ elementwise multiplication,

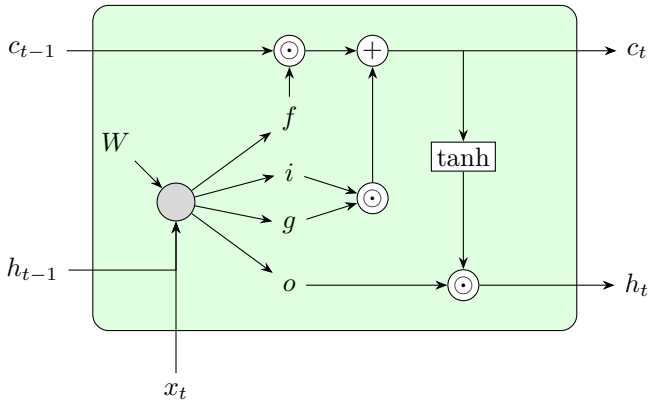
$$\begin{aligned} i_t &= \sigma(W_i x_t + U_i h_{t-1} + b_i), & f_t &= \sigma(W_f x_t + U_f h_{t-1} + b_f), \\ o_t &= \sigma(W_o x_t + U_o h_{t-1} + b_o), & g_t &= \tanh(W_g x_t + U_g h_{t-1} + b_g), \\ c_t &= f_t \odot c_{t-1} + i_t \odot g_t, & h_t &= o_t \odot \tanh(c_t). \end{aligned}$$

- Input gate i_t: how much of g_t to write.
- Forget gate f_t: how much of c_{t-1} to keep.
- Output gate o_t: how much of the cell to reveal in h_t.
- Candidate ("gate gate") g_t: new content.

Compactly,

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix},$$

W is $4h \times 2h$ ($4h \times (h+m)$ for input dimension m), output split into four blocks. Other notation: $[h_{t-1}, x_t]$, $\tilde{C}_t = g_t$, $* = \odot$.



The LSTM cell: W acts on the stacked (h_{t-1}, x_t); the cell state runs along the top.

5.2 Gradient flow through the cell

Theorem 9.7: the cell path

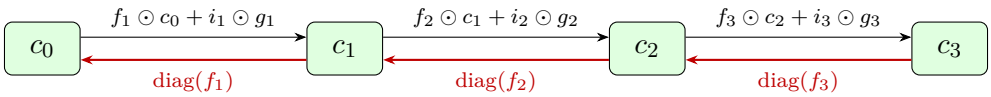
Holding the gates fixed, $\dfrac{\partial c_t}{\partial c_{t-1}} = \operatorname{diag}(f_t)$. Along the path that goes only through cell states,

$$\frac{\partial c_L}{\partial c_\ell} = \operatorname{diag}\Big(\prod_{k=\ell+1}^{L} f_k\Big) \qquad \text{(product taken entrywise)}.$$

Proof. Step 1. In $c_t = f_t \odot c_{t-1} + i_t \odot g_t$ only the first term contains c_{t-1}, as $\operatorname{diag}(f_t) c_{t-1}$.

Step 2. Chain the diagonal Jacobians. ∎

Vanilla factors $\operatorname{diag}(1 - h_t^2)W$ repeat W and can only shrink; the LSTM factor is the learned forget gate. With $f_t \approx 1$ the gradient passes almost unchanged ("uninterrupted gradient flow"); $f = 1$, $i = 0$ preserves the cell indefinitely.



Backward (red) along the cell states multiplies only by forget gates.

Remark 9.8: what the LSTM does not guarantee

Other paths (through h_{t-1} and gates) remain and f_t may be far from 1: no guarantee, just an easier route to long-range dependencies.

6 Applications

Language modeling and image captioning; also SIRD epidemic ODEs, where an LSTM learns the time-varying infection rate α.

6.1 Language modeling

Generation, translation and summarization all model token sequences with f_θ.

Definition 9.9: one-hot encoding (tokenization)

Token t of an n-token vocabulary is $\mathbf{x}_t \in \mathbb{R}^n$ with a single 1 at its index.

Definition 9.10: RNN language model

Sentence generation is next-token prediction. At step t:

$$\begin{aligned} \mathbf{z}_t &= W_e \mathbf{x}_t + \mathbf{b}_e, & W_e &\in \mathbb{R}^{d\times n}, \\ \mathbf{h}_t &= \mathrm{LSTM}(\mathbf{z}_t, \mathbf{h}_{t-1}, \mathbf{c}_{t-1}; \theta), & \mathbf{h}_t &\in \mathbb{R}^d, \\ \hat{\mathbf{x}}_{t+1} &= W_d \mathbf{h}_t + \mathbf{b}_d, & W_d &\in \mathbb{R}^{n\times d}. \end{aligned}$$

(word embedding, recurrent state, vocabulary scores), from $\mathbf{x}_0 =$ `<SOS>` to `<EOS>`.

Training. Read `<SOS>`, $\mathbf{x}_1, \ldots, \mathbf{x}_{T-1}$, predict the next token, with word-level loss

$$\sum_{t=1}^{T} \mathcal{L}(\hat{\mathbf{x}}_t, \mathbf{x}_t), \qquad \mathcal{L}(\hat{\mathbf{x}}_t, \mathbf{x}_t) = -\log \mathrm{softmax}(\hat{\mathbf{x}}_t)_{k_t},$$

(k_t the true index). Since

$$p_\theta(\mathbf{x}_1, \ldots, \mathbf{x}_T) = \prod_{t=1}^{T} p_\theta(\mathbf{x}_t \mid \mathbf{x}_0, \ldots, \mathbf{x}_{t-1}).$$

this is maximum likelihood (Chapter 2).

Inference. From scores $S(\mathbf{x}_t)$:

- Greedy (deterministic): $\mathbf{x}_t = \arg\max S(\mathbf{x}_t)$.
- Probabilistic (stochastic): sample $\mathbf{x}_t \sim S(\mathbf{x}_t)$ from the softmax probabilities.

Feed back until `<EOS>`. Greedy always gives the same sentence; sampling varies.

Machine translation is the delayed many-to-many pattern.

6.2 Image captioning

Describe image $\mathbf{x}$ by sentence $\mathbf{y}$:

1. Recognize the visual patterns in the image with a CNN g_ϕ (Chapter 7).
2. Generate a sentence conditioned on the image with an RNN language model.

The model is

$$f_\theta(\mathbf{y} \mid g_\phi(\mathbf{x})) :$$

features $g_\phi(\mathbf{x})$ condition the language model ("man in black shirt is playing guitar"). Variants: naive, and with attention (Chapter 10).

Naive captioning. The last FC feature of an ImageNet CNN (4096 VGG, 2048 ResNet; $224 \times 224 \times 3$ input) is linearly mapped to the embedding dimension and fed as the first word, then `<sos>`, then generation as in Definition 9.10. Train on (image, sentence) pairs with word-level cross-entropy.

Example 9.11: the captioning loss (lecture example)

$\mathbf{y}_t$ one-hot truth, $\hat{\mathbf{y}}_t$ predicted distribution. Vocabulary {man, plays, guitar, `<EOS>`, `<SOS>`}, target "man plays guitar": inputs `<SOS>`, man, plays, guitar, targets $\mathbf{e}_1, \mathbf{e}_2, \mathbf{e}_3, \mathbf{e}_4 \in \mathbb{R}^5$. If $\hat{\mathbf{y}}_3 = (0.05, 0.1, 0.8, 0.04, 0.01)$, then

$$\mathcal{L}_3 = -\sum_{k=1}^{5} y_{3,k} \log \hat{y}_{3,k} = -\log 0.8 \approx 0.223,$$

and $\mathcal{L} = \sum_{k=1}^{4} \mathcal{L}_k$.

A practical issue. Previous words are at dependency distance 1, the image only at step one, so a strong language model may ignore the image. Remedies:

- **Shortcuts.** Feed the image feature at every step (distance 1, but still ignorable).
- **Attention.** Different regions for different words (Chapter 10); uniform weights recover the global feature.

Chapter 10. Attention and Transformers

Lectures 11/2, 11/4: attention-based captioning, spatial and channel attention, inductive bias, scaled dot-product, cross- and self-attention, positional encoding, masking, multi-head attention, Transformer encoder, linearized attention, layer normalization, Vision Transformer, GELU, Swin Transformer, precipitation nowcasting

1 Attention-Based Image Captioning

Motivation. Each generated word re-weights the image regions it describes.

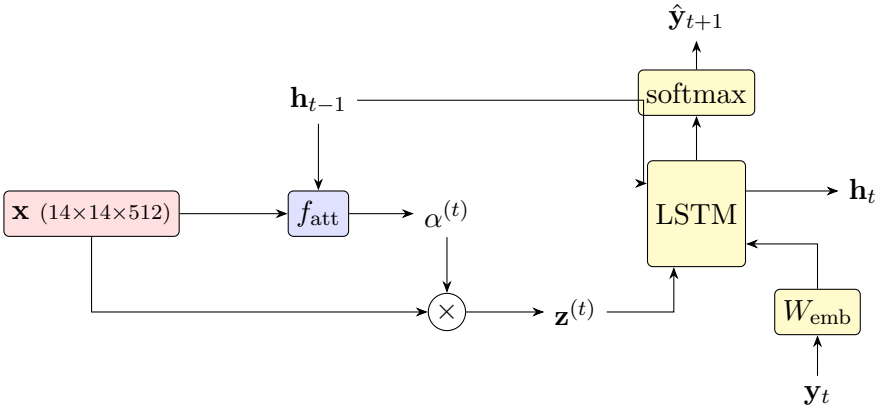
Keep VGG's conv5 map $\mathbf{x} \in \mathbb{R}^{14\times14\times512}$ ($224 \times 224 \times 3$ input): $L = 196$ vectors $\mathbf{x}_{i,j} \in \mathbb{R}^{512}$. Each word t has an **attention map** $\alpha^{(t)} \in \mathbb{R}^{14\times14}$, nonnegative with $\sum_{i,j} \alpha^{(t)}_{i,j} = 1$ ($\alpha^{(1)}$ for "a", $\alpha^{(2)}$ for "bird", ...), summarizing the image as $\mathbf{z}^{(t)} = \sum_{i,j} \alpha^{(t)}_{i,j} \mathbf{x}_{i,j} \in \mathbb{R}^{512}$.

Definition 10.1: attention-based captioning (Show, Attend and Tell)

At step t, with previous hidden state $\mathbf{h}_{t-1}$ and current input word $\mathbf{y}_t$:

$$
\begin{aligned}
e^{(t)} &= f_{\text{att}}(\mathbf{x}, \mathbf{h}_{t-1}) && \text{(alignment scores, one per location)},\\
\alpha^{(t)}_{i,j} &= \frac{\exp(e^{(t)}_{i,j})}{\sum_{i',j'} \exp(e^{(t)}_{i',j'})} && \text{(attention weights)},\\
\mathbf{z}^{(t)} &= \sum_{i,j} \alpha^{(t)}_{i,j} \mathbf{x}_{i,j} && \text{(context vector)},\\
\mathbf{h}_t &= \text{LSTM}(\mathbf{h}_{t-1}, W_{\text{emb}}\mathbf{y}_t, \mathbf{z}^{(t)}) && \text{(modified LSTM)},\\
\hat{\mathbf{y}}_{t+1} &\propto \exp\left(W^o(W_{\text{emb}}\mathbf{y}_t + W^h\mathbf{h}_t + W^z\mathbf{z}^{(t)})\right) && \text{(next-word distribution)}.
\end{aligned}
$$

The context vector enters every LSTM gate; f_{att} is a small MLP; the cell repeats once per word.



$(\mathbf{x}, \mathbf{h}_{t-1}) \mapsto \alpha^{(t)}$, *and* $\mathbf{z}^{(t)}$ *enters the LSTM.*

$\mathbf{z}^{(t)}$ is a weighted average; uniform weights $1/L$ give the naive model with average pooling.

Soft versus hard attention. Soft attention (above) is differentiable and trained by backpropagation. **Hard attention** samples $s_t \sim \text{Categorical}(\alpha^{(t)})$, $\mathbf{z}^{(t)} = \mathbf{x}_{s_t}$ (one spot per word, vs. soft spreading over bird and water); with no gradient through sampling it needs score-function (REINFORCE) estimators.

2 Spatial and Channel Attention

Weights on the axes of $U \in \mathbb{R}^{H \times W \times C}$:

- **Spatial attention** weights the $H \times W$ locations (Section 1).
- **Channel attention** weights the C channels.

Definition 10.2: squeeze-and-excitation block (SENet)

Given $U = [\mathbf{u}_1, \ldots, \mathbf{u}_C]$ with $\mathbf{u}_c \in \mathbb{R}^{H \times W}$:

$$
\begin{aligned}
s_c &= \frac{1}{HW} \sum_{i=1}^{H} \sum_{j=1}^{W} u_c(i,j) && \text{(squeeze: global average pooling)}, \\
\mathbf{e} &= \sigma\big(W_2 \, \mathrm{ReLU}(W_1 \mathbf{s})\big) && \text{(excitation)}, \\
\tilde{\mathbf{u}}_c &= e_c \, \mathbf{u}_c && \text{(scale)},
\end{aligned}
$$

with $W_1 \in \mathbb{R}^{(C/r) \times C}$ and $W_2 \in \mathbb{R}^{C \times (C/r)}$.

Sigmoid weights $e_c \in (0, 1)$ need not sum to 1. The block adds $2C^2/r$ weights (r e.g. 16), inside residual (SE-ResNet) or mobile (ANTNets) blocks.

3 Inductive Bias

Motivation. Each architecture builds in assumptions; attention has the fewest.

Four families:

- MLP: linear layers and nonlinear functions.
- CNN: convolution, pooling, nonlinearities and an MLP head.
- RNN: MLPs for the recurrent update, plus gating functions.
- Attention and Transformers: dot-product attention plus MLPs.

Data and models can be mixed (1D CNN for sequences, RNN or MLP on pixels).

Definition 10.3: inductive bias

The **inductive bias** of a learner is the set of assumptions it uses to predict outputs for inputs it has not observed.

Mostly **locality** and **invariance**:

- Convolution: 2D grid, translation equivariance (Chapter 7).
- RNN: 1D chain, first-order Markov ($\mathbf{h}_t$ sees the past only via $\mathbf{h}_{t-1}$).

Efficient, but not transferable across modalities; one model for images, point clouds, audio and language (a extbffoundation model, in practice the Transformer) needs few structural assumptions.

4 The General Attention Layer

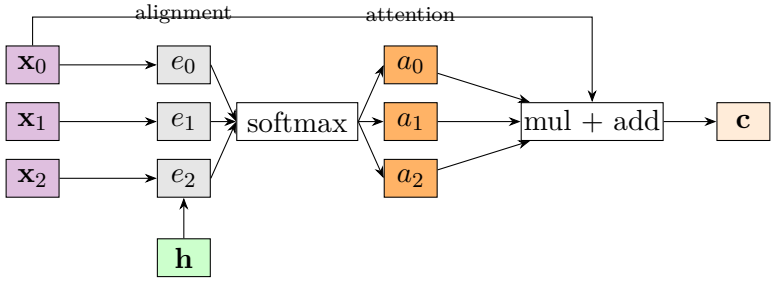
4.1 One query

Definition 10.4: attention with one query

Inputs: vectors $\mathbf{x}_1, \dots, \mathbf{x}_N \in \mathbb{R}^D$ (stacked as $\mathbf{x} \in \mathbb{R}^{N \times D}$) and a query $\mathbf{h} \in \mathbb{R}^D$.

$$\begin{aligned} \text{alignment:} \quad & e_i = f_{\text{att}}(\mathbf{h}, \mathbf{x}_i) = \frac{\mathbf{h} \cdot \mathbf{x}_i}{\sqrt{D}}, \\ \text{attention:} \quad & \mathbf{a} = \text{softmax}(\mathbf{e}), \\ \text{output:} \quad & \mathbf{c} = \sum_{i=1}^{N} a_i \mathbf{x}_i \in \mathbb{R}^D \quad \text{(context vector)}. \end{aligned}$$

An $H \times W \times D$ map gives $N = HW$ vectors. Reordering them leaves the sum $\mathbf{c}$ unchanged (**permutation invariance**).



One query $\mathbf{h}$*: scores, softmax, weighted sum.*

4.2 Why divide by $\sqrt{D}$

For independent entries of $\mathbf{h}, \mathbf{x}$ with mean 0, variance 1, $z = \mathbf{h} \cdot \mathbf{x} = \sum_{m=1}^{D} h_m x_m$ has $\mathbb{E}z = 0$, $\text{Var}\, z = \sum_m \text{Var}(h_m x_m) = D$. Unscaled logits of sd $\sqrt{D}$ make the softmax nearly one-hot, with almost no gradient through the weights; dividing by $\sqrt{D}$ restores unit variance.

4.3 Many queries, keys and values

Definition 10.5: multiple queries

Queries $\mathbf{q}_1, \dots, \mathbf{q}_M \in \mathbb{R}^D$ ($\mathbf{q} \in \mathbb{R}^{M \times D}$), one context vector each:

$$e_{i,j} = \frac{\mathbf{q}_j \cdot \mathbf{x}_i}{\sqrt{D}}, \qquad \mathbf{a}_{\cdot,j} = \text{softmax}(\mathbf{e}_{\cdot,j}), \qquad \mathbf{y}_j = \sum_i a_{i,j} \mathbf{x}_i.$$

$\mathbf{x}$ plays two roles (alignment, output); a separate linear layer for each adds expressiveness and changes dimensions.

Definition 10.6: cross-attention layer

With learnable $W_k \in \mathbb{R}^{D \times D_k}$ and $W_v \in \mathbb{R}^{D \times D_v}$ and queries $\mathbf{q} \in \mathbb{R}^{M \times D_k}$:

$$\mathbf{k} = \mathbf{x} W_k, \quad \mathbf{v} = \mathbf{x} W_v, \quad e_{i,j} = \frac{\mathbf{q}_j \cdot \mathbf{k}_i}{\sqrt{D_k}}, \quad \mathbf{a}_{\cdot,j} = \text{softmax}(\mathbf{e}_{\cdot,j}), \quad \mathbf{y}_j = \sum_i a_{i,j} \mathbf{v}_i \in \mathbb{R}^{D_v}.$$

Definition 10.7: self-attention layer

With $\mathbf{q} = \mathbf{x}W_q$, $W_q \in \mathbb{R}^{D\times D_k}$, the layer maps N inputs to N outputs (**self-attention**); rows as tokens,

$$\mathrm{SA}(\mathbf{x}) = \mathrm{Attn}(\mathbf{x}W_q, \mathbf{x}W_k, \mathbf{x}W_v) = \mathrm{softmax}\Big(\frac{\mathbf{x}W_q(\mathbf{x}W_k)^\top}{\sqrt{D_k}}\Big)\mathbf{x}W_v,$$

softmax by rows: the Transformer's **scaled dot-product attention** $\mathrm{Attn}(Q,K,V) = \mathrm{softmax}(QK^\top/\sqrt{d_k})V$.

$A = \mathrm{softmax}(QK^\top/\sqrt{d_k}) \in \mathbb{R}^{n\times n}$ is row-stochastic, so each output averages all values: a contextual encoding with whole-sequence receptive field (**feature aggregation**).

The four steps. $\mathbf{x} \in \mathbb{R}^{n\times d}$, $W_q, W_k \in \mathbb{R}^{d\times d_k}$, $W_v \in \mathbb{R}^{d\times d_v}$; $Q = \mathbf{x}W_q$ (**query**, "the question"), $K = \mathbf{x}W_k$ (**key**, "the index"), $V = \mathbf{x}W_v$ (**value**, "the information"):

(i) $A = QK^\top \in \mathbb{R}^{n\times n}$, $A_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j$: how much token i uses token j;

(ii) $A' = QK^\top/\sqrt{d_k}$: $\mathrm{Var}\sum_{m=1}^{d_k} q_m k_m = d_k$ as above;

(iii) $\alpha_{ij} = \mathrm{softmax}(A')_{ij}$, row by row;

(iv) $\mathrm{output}_i = \sum_j \alpha_{ij}\mathbf{v}_j$.

Cross versus self. Cross-attention takes queries from elsewhere (a decoder), keys and values from X; self-attention takes all three from X. Captioning (Section 1) is cross-attention (query: sentence state; keys, values: image features). In Definition 10.4 keys and values coincide.

5 Permutation Symmetry

For a permutation matrix P, cross-attention with fixed queries is **permutation invariant** (keys and values reorder together), and self-attention is **permutation equivariant**, $\mathrm{SA}(P\mathbf{x}) = P\ \mathrm{SA}(\mathbf{x})$, since $\mathbf{q}_i \cdot \mathbf{k}_j$ depends only on the two tokens. "Dog bites man" and "man bites dog" agree up to reordering: order must be supplied separately.

6 Positional Encoding

Motivation. Attach a position vector to each token, since self-attention cannot see positions.

$\mathbf{p}_j = \mathrm{pos}(j) \in \mathbb{R}^d$ is concatenated (slides) or added, $\mathbf{x}_j + \mathbf{p}_j$ (Transformer). Options:

1. **Learn a lookup table**: $T \times d$ parameters for $t \in [0, T)$.
2. **Design a fixed function**.

Intuition: in a binary counter each bit is periodic in the position (periods $2, 4, \ldots$).

Definition 10.8: sinusoidal positional encoding

For even d and $k = 0, 1, \ldots, \frac{d}{2} - 1$, let $\omega_k = 1/10000^{2k/d}$ and

$$\mathrm{PE}(t)_{2k} = \sin(\omega_k t), \qquad \mathrm{PE}(t)_{2k+1} = \cos(\omega_k t).$$

Equivalently, $\mathrm{PE}(t) = \big(\sin\omega_0 t, \cos\omega_0 t,\ \ldots,\ \sin\omega_{d/2-1}t, \cos\omega_{d/2-1}t\big)$.

Unlike a table it is defined for every t; frequencies fall geometrically from 1 to about $1/10000$, fast to slow like counter bits.

Example 10.9: $d = 4$ (lecture example)

$\omega_0 = 1$, $\omega_1 = 1/10000^{2/4} = 0.01$: $\mathrm{PE}(t) = (\sin t, \cos t, \sin 0.01t, \cos 0.01t)$ and

$$\mathrm{PE}(0) = (0, 1, 0, 1), \qquad \mathrm{PE}(1) \approx (0.841, 0.540, 0.010, 1.000),$$
$$\mathrm{PE}(2) \approx (0.909, -0.416, 0.020, 1.000).$$

With $\mathbf{z}_j = \mathbf{x}_j + \mathbf{p}_j$ and $\mathbf{x}_{\text{dog}} = (1, 0, 2, 0)$,

$$\mathbf{x}_{\text{dog}} + \mathrm{PE}(0) = (1, 1, 2, 1), \qquad \mathbf{x}_{\text{dog}} + \mathrm{PE}(2) \approx (1.909, -0.416, 2.020, 1.000),$$

so "dog" at positions 0 and 2 differ, and "dog bites man" is no longer a reordering of "man bites dog".

7 Masked Self-Attention

Generation must not see future words. Training on "man plays guitar", the Transformer predicts all next tokens in parallel:

position	input	target
0	`<SOS>`	man
1	man	plays
2	plays	guitar
3	guitar	`<EOS>`

Position 1 could copy its target "plays" from position 2, so query j may attend only to $0, \ldots, j$: future scores are set to $-\infty$ ($e^{-\infty} = 0$).

Definition 10.10: masked (causal) self-attention

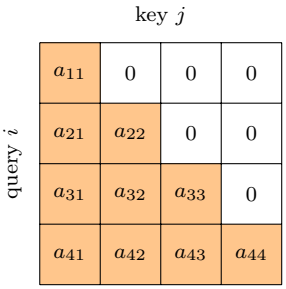
With $\mathbf{q}_i, \mathbf{k}_j, \mathbf{v}_j$ as in Definition 10.7, query i uses the scores

$$e_{i,j} = \begin{cases} \mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{D_k}, & j \le i, \\ -\infty, & j > i, \end{cases} \qquad \mathbf{y}_i = \sum_j a_{i,j} \mathbf{v}_j, \quad \mathbf{a}_{i,\cdot} = \mathrm{softmax}(\mathbf{e}_{i,\cdot}).$$

The slides put the key index first, $e_{i,j} = \mathbf{q}_j \cdot \mathbf{k}_i / \sqrt{D_k}$ ($i \le j$), $-\infty$ ($i > j$):

$$E = \begin{pmatrix} e_{0,0} & e_{0,1} & e_{0,2} \\ -\infty & e_{1,1} & e_{1,2} \\ -\infty & -\infty & e_{2,2} \end{pmatrix}$$

(column j = query j), the transpose of the pattern below. Rows still sum to 1; position i uses only tokens $1, \ldots, i$.



Causal attention: each query sees itself and earlier tokens.

8 Multi-Head Attention

Definition 10.11: multi-head attention

With h heads, each with its own projections $W_q^{(r)}, W_k^{(r)} \in \mathbb{R}^{d\times d_k}$ and $W_v^{(r)} \in \mathbb{R}^{d\times d_v}$,

$$\text{head}_r = \text{Attn}\left(QW_q^{(r)}, KW_k^{(r)}, VW_v^{(r)}\right), \qquad \text{MHA}(Q,K,V) = [\text{head}_1, \ldots, \text{head}_h]\, W_o,$$

with $W_o \in \mathbb{R}^{hd_v\times d}$. Usually $d_k = d_v = d/h$.

Each head (MatMul, Scale, optional Mask, SoftMax, MatMul) can track a different relation (previous word, subject).

9 The Transformer Encoder

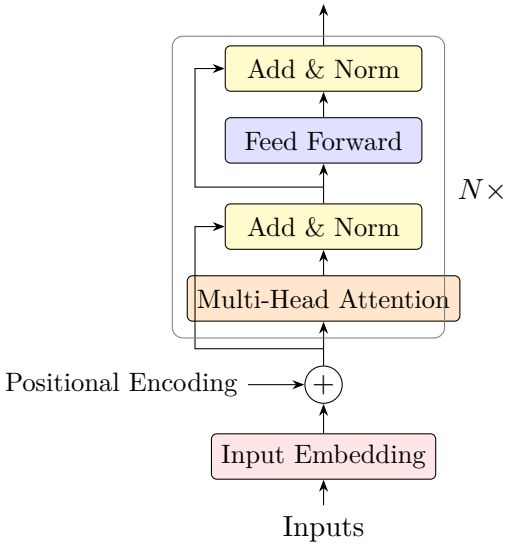
Motivation. Self-attention and position-wise MLPs, each with skip connection and layer norm (Vaswani et al., 2017).

Definition 10.12: Transformer encoder

1. Tokenize and embed: $\mathbf{x} \in \mathbb{R}^{n\times d}$.
2. Add positions: $X_0 = \mathbf{x} + \text{PE}$.
3. Each of N layers:

$$Z = \text{LN}\left(X + \text{MHA}(X,X,X)\right), \qquad X' = \text{LN}\left(Z + \text{FFN}(Z)\right),$$

   $\text{FFN}(\mathbf{z}) = W_2\,\text{ReLU}(W_1\mathbf{z} + \mathbf{b}_1) + \mathbf{b}_2$ per token.
4. Layers are chained.



One encoder layer (boxed), repeated N times; side arrows are skips.

Positions enter only through PE. Cost is dominated by $QK^\top \in \mathbb{R}^{n\times n}$: $O(n^2 d)$ time, $O(n^2)$ memory.

10 Linearization of the Transformer

Motivation. Replace the softmax similarity by a kernel feature map, so attention costs $O(N)$ instead of $O(N^2)$.

Recap. $X = [\mathbf{x}_1, \dots, \mathbf{x}_N]^\top \in \mathbb{R}^{N\times D_x}$.

Step 1. Project: $Q = XW_Q^\top$, $K = XW_K^\top$, $V = XW_V^\top$, with learnable $W_Q, W_K \in \mathbb{R}^{D\times D_x}$, $W_V \in \mathbb{R}^{D_v\times D_x}$; $Q = [\mathbf{q}_1, \dots, \mathbf{q}_N]^\top$, similarly K, V.

Step 2. Output $\hat{V} = [\hat{\mathbf{v}}_1, \dots, \hat{\mathbf{v}}_N]^\top$ with

$$\hat{\mathbf{v}}_i = \sum_{j=1}^{N} \mathrm{softmax}_j\Big(\frac{\mathbf{q}_i^\top \mathbf{k}_j}{\sqrt{D}}\Big)\mathbf{v}_j \iff \hat{V} = \mathrm{softmax}\Big(\frac{QK^\top}{\sqrt{D}}\Big)V = AV.$$

Bottleneck. Storing $A \in \mathbb{R}^{N\times N}$ takes $O(N^2)$ memory and computing AV costs $O(N^2)$ time.

Similarity form. With $s_{ij} = \mathbf{q}_i^\top \mathbf{k}_j/\sqrt{D}$, $\alpha_{ij} = \mathrm{softmax}_j(s_{ij}) = \exp(s_{ij})/\sum_{l=1}^N \exp(s_{il})$, so

$$\hat{\mathbf{v}}_i = \sum_{j=1}^{N} \alpha_{ij}\mathbf{v}_j = \frac{\sum_{j=1}^N \exp(s_{ij})\mathbf{v}_j}{\sum_{l=1}^N \exp(s_{il})} = \frac{\sum_{j=1}^N \mathrm{sim}(\mathbf{q}_i, \mathbf{k}_j)\mathbf{v}_j}{\sum_{j=1}^N \mathrm{sim}(\mathbf{q}_i, \mathbf{k}_j)}, \qquad \mathrm{sim}(\mathbf{q}, \mathbf{k}) := \exp\Big(\frac{\mathbf{q}^\top \mathbf{k}}{\sqrt{D}}\Big).$$

Definition 10.13: linearized (kernel) attention

Replace $\mathrm{sim}(\mathbf{q}_i, \mathbf{k}_j)$ by a kernel that is an inner product of a feature map,

$$k(\mathbf{q}_i, \mathbf{k}_j) = \phi(\mathbf{q}_i)^\top \phi(\mathbf{k}_j), \qquad \phi : \mathbb{R}^D \to \mathbb{R}^r,$$

e.g. $\phi(\mathbf{x}) = \mathrm{ELU}(\mathbf{x}) + 1$ entrywise: $\phi(\mathbf{x})_l = x_l + 1$ $(x_l \geq 0)$, e^{x_l} $(x_l < 0)$. Then

$$\hat{\mathbf{v}}_i^\top \approx \tilde{\mathbf{v}}_i^\top = \frac{\sum_j k(\mathbf{q}_i, \mathbf{k}_j)\mathbf{v}_j^\top}{\sum_j k(\mathbf{q}_i, \mathbf{k}_j)} = \frac{\phi(\mathbf{q}_i)^\top \sum_{j=1}^N \phi(\mathbf{k}_j)\mathbf{v}_j^\top}{\phi(\mathbf{q}_i)^\top \sum_{j=1}^N \phi(\mathbf{k}_j)}.$$

Cost. The sums do not depend on i; set

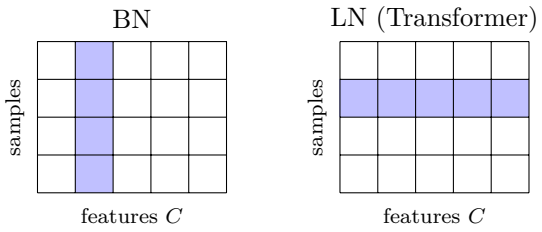
$$S = \sum_{j=1}^{N} \phi(\mathbf{k}_j)\mathbf{v}_j^\top \in \mathbb{R}^{r\times d_v}, \qquad \mathbf{z} = \sum_{j=1}^{N} \phi(\mathbf{k}_j) \in \mathbb{R}^r \quad \Longrightarrow \quad \tilde{\mathbf{v}}_i = \frac{S^\top \phi(\mathbf{q}_i)}{\phi(\mathbf{q}_i)^\top \mathbf{z}}.$$

S costs $O(Nrd_v)$, $\mathbf{z}$ costs $O(Nr)$, and all $S^\top\phi(\mathbf{q}_i)$, $\phi(\mathbf{q}_i)^\top\mathbf{z}$ cost $O(Nrd_v + Nr)$: **linear in** N. In matrix form $A \approx \phi(Q)\phi(K)^\top$ (rows normalized), a product of two thin factors, so $\phi(K)^\top V$ is formed first and the $N \times N$ matrix never appears.

11 Layer Normalization

BN (Chapter 6) uses batch statistics per channel; LN normalizes each sample by itself.

- **BN**: per channel, over batch N and positions H, W.
- **LN in CNNs**: per sample, over C, H, W.
- **LN in Transformers**: per sample and token, over the C features.



Shaded cells share one mean and variance.

Definition 10.14: layer normalization

For $\mathbf{x} \in \mathbb{R}^d$, let $\mu = \frac{1}{d}\sum_k x_k$ and $\sigma^2 = \frac{1}{d}\sum_k (x_k - \mu)^2$. Then

$$\hat{\mathbf{x}} = \frac{\mathbf{x} - \mu\mathbf{1}}{\sqrt{\sigma^2 + \epsilon}}, \qquad \mathrm{LN}(\mathbf{x}) = \boldsymbol{\gamma} \odot \hat{\mathbf{x}} + \boldsymbol{\beta},$$

with learnable $\boldsymbol{\gamma}, \boldsymbol{\beta} \in \mathbb{R}^d$ and a small $\epsilon > 0$.

Example 10.15: BN versus LN (lecture example)

CNN with $B = 2$, $C = 2$, $H = 1$, $W = 2$ ($\epsilon = 0$): channel 1, 2 of $X^{(1)}$ are $(1, 3)$, $(5, 7)$; of $X^{(2)}$, $(9, 11)$, $(13, 15)$.

(i) BN (per channel over the batch): $\mu_1 = \frac{1+3+9+11}{4} = 6$, $\sigma_1^2 = 17$; $\mu_2 = 10$, $\sigma_2^2 = 17$. The first entry 1 of $X^{(1)}$ becomes $(1 - 6)/\sqrt{17}$.

(ii) LN (per image): $X^{(1)}$ has entries $\{1, 3, 5, 7\}$, $\mu^{(1)} = 4$, $(\sigma^{(1)})^2 = 5$; $\mu^{(2)} = 12$, $(\sigma^{(2)})^2 = 5$. The entry 1 becomes $(1 - 4)/\sqrt{5}$.

(iii) LN in a Transformer (per token): token $1 = (1, 5)$ has $\mu = 3$, $\sigma^2 = 4$, so $(1, 5) \mapsto (-1, 1)$; likewise token $2 = (3, 7) \mapsto (-1, 1)$.

12 Why Transformers

Built for translation, the Transformer scales well with data and underlies BERT, GPT-2, Megatron-LM, T5, Turing-NLG (up to 17 billion parameters by early 2020). Its weak inductive bias costs on small data and pays off on large data; the loss keeps falling as the parameter count grows (extbfscaling law). Most large-scale models since (BERT and GPT 2018, GPT-3 2020, ChatGPT 2022, GPT-4 and LLaMA 2023) are Transformers.

13 Vision Transformer

Motivation. An image becomes a sequence of patch tokens, and the Transformer encoder is applied unchanged (Dosovitskiy et al., 2021).

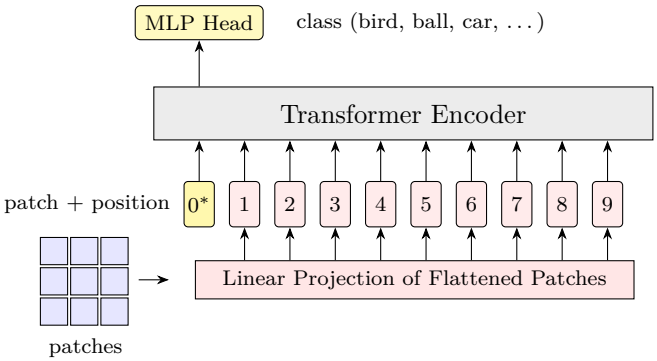
Definition 10.16: Vision Transformer (ViT)

Split an image into N non-overlapping $P \times P$ patches $\mathbf{x}_p^1, \dots, \mathbf{x}_p^N \in \mathbb{R}^{P^2C}$ (tokenization).

1. **Patch embedding**: $\mathbf{x}_p^i E$, $E \in \mathbb{R}^{(P^2C)\times D}$, from the flattened patch (or from ConvNet features: *hybrid*).
2. **Class token**: a learnable $\mathbf{x}_{\text{class}}$ is prepended (the *global token*, as BERT's [class] token).
3. **Position embedding** $E_{pos} \in \mathbb{R}^{(N+1)\times D}$ is added (learnable, or a fixed 2D periodic function).

$$
\begin{aligned}
\mathbf{z}_0 &= [\mathbf{x}_{\text{class}}; \mathbf{x}_p^1 E; \dots; \mathbf{x}_p^N E] + E_{pos}, \\
\mathbf{z}'_\ell &= \mathrm{MSA}\big(\mathrm{LN}(\mathbf{z}_{\ell-1})\big) + \mathbf{z}_{\ell-1}, \qquad \ell = 1, \dots, L, \\
\mathbf{z}_\ell &= \mathrm{MLP}\big(\mathrm{LN}(\mathbf{z}'_\ell)\big) + \mathbf{z}'_\ell, \qquad \ell = 1, \dots, L, \\
\mathbf{y} &= \mathrm{LN}(\mathbf{z}_L^0).
\end{aligned}
$$

LN comes before every block, a residual after every block; the MLP has two layers with GELU. The classifier (MLP head) acts on $\mathbf{y}$, the output of the class token.



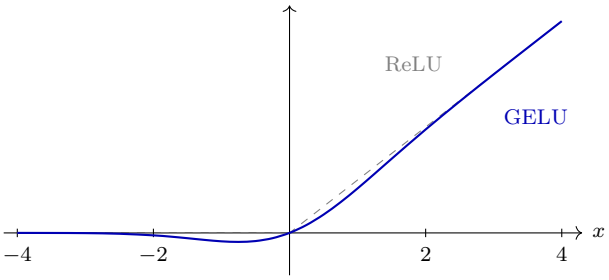
ViT: patch tokens plus the class token 0; the head reads only the class token.*

13.1 GELU

Definition 10.17: GELU

With Φ the standard normal CDF,

$$\mathrm{GELU}(x) = x\,\Phi(x) \approx 0.5\,x\Big(1 + \tanh\big(\sqrt{2/\pi}\,(x + 0.044715\,x^3)\big)\Big).$$



GELU (solid) is smooth and slightly negative for $x < 0$ (minimum ≈ -0.17 at $x \approx -0.75$); ReLU dashed.

13.2 ViT versus CNN

- **Locality**: CNN features are local; ViT attends globally (non-local).
- **Translation equivariance**: CNN yes; ViT not built in.
- **Fine-tuning at higher resolution**: replace the prediction head and learn it; 2D-interpolate the pre-trained position embeddings.
- **Experiments**: pre-trained on ImageNet, large ViTs are worse than ResNets (BiT); pre-trained on ImageNet-21k or JFT-300M they win, and larger ViTs overtake smaller ones as data grow. In few-shot evaluation ResNets do better on small pre-training sets but plateau sooner.

14 Swin Transformer (Advanced)

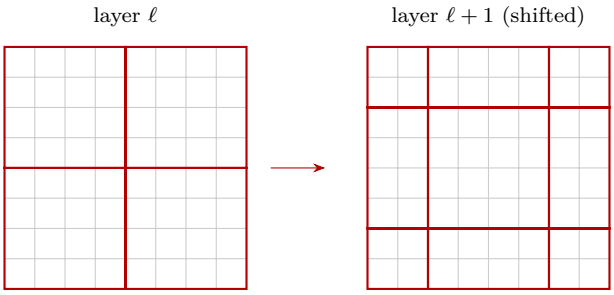
Motivation. Hierarchical feature maps and cost linear in image size, via attention inside local windows (Liu et al., 2021).

For an $h \times w$ map with C channels and $M \times M$ windows,

$$\Omega(\mathrm{MSA}) = 4hwC^2 + 2(hw)^2C, \qquad \Omega(\text{W-MSA}) = 4hwC^2 + 2M^2hwC\ :$$

global attention is quadratic in hw, window attention linear.

- **Patch merging** ($\sim$ pooling): split into 2×2 groups, stack each group depth-wise, combine; the next stage has half the resolution and larger receptive windows (from $4\times$ to $8\times$ to $16\times$ down-sampling).
- **Shifted window**: layer $\ell + 1$ shifts the window grid, so attention crosses the boundaries of the windows of layer ℓ (non-overlapping windows alone never interact).



Self-attention runs inside each red window; shifting the grid links neighbouring windows.

15 Application: Radar-Based Precipitation Nowcasting

- Extreme rainfall is becoming more frequent and intense with climate change (monthly counts of normal, heavy, extreme and very extreme events, 2014 vs 2023).
- Extreme rain is **high-frequency**: small regions with high intensity.
- Model: an encoder–decoder on radar echo sequences ($T \times H \times W$) with 3D (Video) Swin Transformer blocks and patch merging in the encoder, skip connections, cubic dual upsampling in the decoder, a temporal extractor, and the FACL loss.
- Video Swin: attention within windows, shifted windows, and patch merging to larger windows, now over space and time.
- At $t + 10$ min it keeps sharper high-intensity cells than SimVP, Earthformer, AlphaPre and CasCast.

Chapter 11. Generative Models and Autoencoders

Lecture 11/9: supervised vs unsupervised learning, generative modeling, PixelRNN and PixelCNN, latent variable models, autoencoders, autoencoders for generation, the variational lower bound

1 Supervised versus Unsupervised Learning

	supervised	unsupervised
data	pairs (x, y), y a label	x only, no labels
goal	learn a map $x \mapsto y$	learn hidden structure of the data
examples	classification, regression, detection, segmentation, captioning	clustering, dimensionality reduction (PCA, 3-d $\to$ 2-d), density estimation

2 Generative Modeling

Motivation. Given training data, generate new samples from the same distribution.

Training data $\sim p_{\text{data}}(x)$; learning gives $p_{\text{model}}(x)$, sampling gives new x. Objectives:

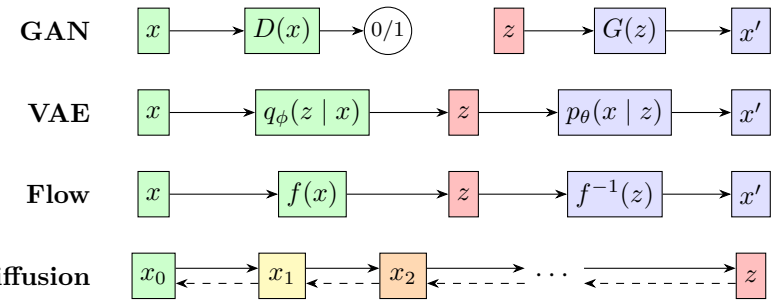
1. learn $p_{\text{model}}(x)$ approximating $p_{\text{data}}(x)$;
2. sample new x from $p_{\text{model}}(x)$.

As density estimation:

- **Explicit**: define and solve for $p_{\text{model}}(x)$ explicitly.
- **Implicit**: learn a model that samples from $p_{\text{model}}(x)$ without defining it.

Four families. x: training data, x': generated sample, z: latent variable with a simple, tractable distribution.

- **GAN**: adversarial training; generator $G(z) = x'$, discriminator $D(x) \in \{0, 1\}$.
- **VAE**: maximize a variational lower bound; encoder $q_\phi(z \mid x)$, decoder $p_\theta(x \mid z)$.
- **Flow-based**: invertible transform $z = f(x)$, $x' = f^{-1}(z)$.
- **Diffusion**: gradually add Gaussian noise $x_0 \to x_1 \to \cdots \to z$, then reverse.



Green: encoding / real data; blue: generation; red: latent variable with a simple distribution.

3 Autoregressive Models

An RNN models $p(x_i \mid x_1, \ldots, x_{i-1})$ (Chapter 9), so $p(x) = \prod_i p(x_i \mid x_1, \ldots, x_{i-1})$.

- **PixelRNN** (van den Oord et al., 2016): generate pixels from a corner; the dependency on previous pixels is an RNN (LSTM). Sequential generation is slow in both training and inference.
- **PixelCNN** (van den Oord et al., 2016): the dependency is a CNN over the context region (**masked convolution**). Training is faster (context values are known from training images, so convolutions run in parallel); generation is still slow: a 32×32 image needs 1024 forward passes.

Pros: explicit likelihood $p(x)$, easy to optimize, good samples. Con: sequential generation is slow. Improvements (PixelCNN++, Salimans et al., 2017): gated convolutions, shortcuts, discretized logistic loss, multi-scale, training tricks.

4 Latent Variable Models

Motivation. Only a small portion of the data space holds valid data, so a much more compact representation should exist.

Data are fixed-dimensional vectors $x \in \mathbb{R}^d$: $d = \text{width} \times \text{height} \times 3$ for images, $d = (\text{sentence length}) \times (\#\text{words})$ for sentences. A random point of $\mathbb{R}^d$ is almost never valid data. Hence a compact representation $z \in \mathbb{R}^m$ with $d \gg m$:

- **observable variable** x: observed directly from data;
- **latent variable** z: hidden, to be inferred from data.

Learning a representation ($x \to z \to$ output):

- Supervised (paired data (x, y)): compare the prediction for x with the known output,

$$\arg\min_{\psi,\phi} \sum_{(x,y)\in D} \mathcal{L}(p_\psi(g_\phi(x)), y).$$

- Unsupervised (data x only): model $p(x)$ by maximum likelihood (Chapter 2),

$$\arg\max_{\theta\in\Theta} \sum_i \log p_\theta(x^{(i)}).$$

Definition 11.1: latent variable model

A latent variable z (compact representation) generates the observation x (original representation), $p(x, z) = p_\theta(x \mid z)\, p(z)$:

1. sample $z \sim p(z)$ from a simple distribution;
2. generate $x \sim p_\theta(x \mid z)$, modeled by a neural network.

Why latent variables. They make data easy to interpret and manipulate.

- Attribute vector: $z = (\text{angle}, \text{age}, \text{gender}, \text{color}, \ldots, \text{hair})$; sample z, generate a face conditioned on it.

- **Latent interpolation**: $z = \alpha z_1 + (1-\alpha) z_2$ morphs one face into another.
- Changing one generative factor (e.g. hairstyle) changes only that attribute of the image.

Remaining questions. How to learn the model with no supervision on z? How to sample z (its distribution is unknown)? How to design the generator?

5 Autoencoders

Motivation. Learn an encoder mapping data to a compact z, much smaller than x yet keeping its most useful information, by reconstruction.

Definition 11.2: autoencoder

Encoder $q_\phi(z \mid x)$ maps x to the feature z; decoder $p_\theta(x \mid z)$ maps z back to the reconstruction $\hat{x}$. Both are trained on the reconstruction loss

$$\arg\min_{\theta,\phi} \|x - \hat{x}\|_2^2.$$

Ideally it learns the identity, yet the representation is useful: x must be squashed into a small z that still suffices for reconstruction. Learning is purely unsupervised. Example: 4-layer conv encoder, 4-layer upconv decoder.

Autoencoder for generation. Discard the encoder, sample a feature z, decode. Issue: the prior is unknown, and encoded points scatter irregularly in z-space. Fix: force the encoder distribution toward a simple prior,

$$\min_{\phi} \; D_{\mathrm{KL}}\big(q_\phi(z \mid x) \,\|\, p(z)\big), \qquad p(z) = \mathcal{N}(0, I),$$

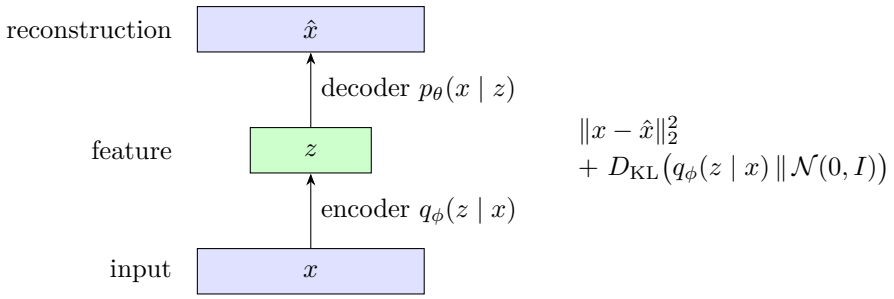
so $q_\phi(z \mid x) \approx \mathcal{N}(0, I)$ and $z \sim \mathcal{N}(0, I)$ can be sampled.

Definition 11.3: autoencoder as a generative model

Training:

$$\arg\min_{\theta,\phi} \sum_i \|x^{(i)} - \hat{x}^{(i)}\|_2^2 + D_{\mathrm{KL}}\big(q_\phi(z \mid x^{(i)}) \,\|\, p(z)\big).$$

Inference: sample $z^{(i)} \sim \mathcal{N}(0, I)$, then generate $\hat{x}^{(i)} \sim p_\theta(x \mid z^{(i)})$.



Autoencoder; the KL term makes sampling $z \sim \mathcal{N}(0, I)$ a valid way to generate.

What this objective does exactly is derived next from maximum likelihood.

6 Variational Autoencoder

Maximize $\sum_i \log p_\theta(x^{(i)})$ with the latent variable model $z \sim p(z)$, $x \sim p_\theta(x \mid z)$. For $x = x^{(i)}$,

$$\log p(x) = \log \int p(x, z)\, dz = \log \int p_\theta(x \mid z)\, p(z)\, dz.$$

Theorem 11.4: variational lower bound

$$\log p(x^{(i)}) \;\geq\; \mathbb{E}_{q_\phi(z|x^{(i)})}\big[\log p_\theta(x^{(i)} \mid z)\big] - D_{\mathrm{KL}}\big(q_\phi(z \mid x^{(i)}) \,\|\, p(z)\big).$$

The left side is the likelihood to maximize; the right side is the **variational lower bound**, so maximizing it maximizes the likelihood indirectly. The objective of the latent variable generative model:

$$\arg\max_{\theta,\phi} \; \frac{1}{N} \sum_{i=1}^{N} \mathbb{E}_{q_\phi(z|x^{(i)})}\big[\log p_\theta(x^{(i)} \mid z)\big] - D_{\mathrm{KL}}\big(q_\phi(z \mid x^{(i)}) \,\|\, p(z)\big).$$